

NATIVIQ SYSTEMS

NativIQ Systems - Herramientas open-source, infraestructura y arquitectura unipersonal

Transcripción completa del bloque de conversación

12-13 de julio de 2026

Documento de archivo del bloque de conversación sobre herramientas open-source, modelos, hardware, nube descentralizada, estructura unipersonal y distribución de responsabilidades técnicas. Se incluyen las intervenciones visibles del usuario, las respuestas sustantivas y los avances de trabajo mostrados durante la conversación.

Alcance del documento

Esta transcripción abarca el bloque iniciado con la consulta sobre reemplazos open-source para Claude, Claude Code y Claude Cowork, y finaliza con la decisión de cerrar el bloque para continuar la definición de agentes en un nuevo chat.

- Fechas: 12 y 13 de julio de 2026.
- Participantes: Mauro y ChatGPT.
- Formato: mensajes en orden cronológico, con enlaces y estructura técnica preservados.
- Principio consolidado: que ninguna empresa, gobierno, red, repositorio, token o proveedor pueda detener por sí solo a NativIQ Systems.

Mauro

Intervención 1 · 2026-07-12 20:36:28Z

ChatGPT. Necesito que me recomiendes las mejores herramientas disponibles en el mercado que sean open-source para reemplazar a las herramientas que ofrece la empresa Anthropic:

- Claude
- Claude Code
- Claude CO-WORK

ChatGPT - avance de trabajo

Voy a separar la comparación en tres capas: modelo conversacional, agente de programación y entorno colaborativo de agentes. Luego evaluaré alternativas open-source por madurez, privacidad, facilidad de despliegue y encaje con NativIQ Systems.

ChatGPT - avance de trabajo

La primera conclusión es importante: no existe un único reemplazo open-source que copie todo Anthropic. La alternativa más sólida es una arquitectura modular, donde el modelo, la interfaz y el agente puedan cambiarse sin rehacer el sistema.

ChatGPT

Tomé “**Claude CO-WORK**” como **Claude Cowork**, el agente de Anthropic orientado a tareas de conocimiento, archivos locales, investigación y ejecución de trabajos de varios pasos. Anthropic distingue actualmente tres productos: Claude, Claude Code y Claude Cowork. (anthropic.com)

Recomendación principal

Producto de Anthropic	Mejor reemplazo open-source	Alternativa destacada
Claude	LibreChat + Qwen 3.6	AnythingLLM
Claude Code	OpenCode	Qwen Code
Claude Cowork	Goose	Open Interpreter

No se trata solamente de cambiar una aplicación. Claude combina cuatro capas: **modelo, interfaz, herramientas/agentes e infraestructura de inferencia**. En open-source conviene mantenerlas separadas para evitar dependencia de un proveedor.

1. Reemplazo open-source de Claude

Ganador: LibreChat

LibreChat es mi primera elección como interfaz general para reemplazar Claude porque ofrece:

- Chat multiusuario y autohospedado.
- Conexión simultánea con modelos locales y proveedores externos.

- Agentes configurables.
- MCP.
- Artifacts.
- intérprete de código y acciones personalizadas.
- Autenticación y controles para equipos.
- Licencia **MIT**, adecuada para desarrollar productos comerciales y modificaciones internas. (github.com)

LibreChat sería la **capa de interacción** de NativIQ Systems, no el modelo en sí.

Modelo recomendado

Como modelo open-weight principal utilizaría **Qwen 3.6**, particularmente:

- **Qwen 3.6 27B**, cuando se priorice calidad general.
- **Qwen 3.6 35B-A3B**, para agentes, programación y uso eficiente de inferencia MoE.
- Modelos Qwen más pequeños cuantizados para pruebas locales.

La familia Qwen 3.6 tiene pesos publicados bajo **Apache 2.0**, soporte para despliegue mediante vLLM y capacidades orientadas a coding y agentes. (github.com)

Como segunda familia de modelos consideraría **gpt-oss-20b** para instalaciones relativamente contenidas y **gpt-oss-120b** para infraestructura más potente. Ambos tienen licencia Apache 2.0, razonamiento configurable, function calling y soporte para ejecución mediante herramientas. (github.com)

Alternativa sencilla: AnythingLLM

AnythingLLM es más fácil de instalar y administrar que LibreChat. Es especialmente bueno para:

- Conversar con documentos internos.
- Crear espacios de conocimiento por cliente o proyecto.
- RAG.
- Agentes internos.
- Memoria.
- Tareas programadas.
- Enrutamiento entre modelos.
- Constructor de agentes sin código.

Tiene licencia MIT y versiones para escritorio, Docker y servidores multiusuario. (github.com)

Para una pequeña instalación interna de NativIQ, AnythingLLM puede ser más práctico. Para construir una plataforma extensible, multiusuario y comercial, prefiero LibreChat.

Open WebUI: excelente, pero con una advertencia

Open WebUI probablemente ofrece la experiencia visual más completa: RAG, memoria persistente, agentes, permisos, canales colaborativos, herramientas, MCP, aplicación de escritorio y funcionamiento offline. (github.com)

Sin embargo, el código actual incluye una licencia propia que exige conservar la marca **Open WebUI** en determinadas circunstancias. Por eso no sería mi primera elección para una solución de NativIQ que eventualmente necesite white-label, redistribución o identidad visual propia. (github.com)

Ranking para reemplazar Claude

1. **LibreChat**: mejor plataforma extensible y comercial.
2. **AnythingLLM**: mejor facilidad de instalación y gestión documental.
3. **Open WebUI**: mejor experiencia visual, pero menos conveniente por su licencia actual.

2. Reemplazo open-source de Claude Code

Claude Code comprende el repositorio, modifica múltiples archivos, ejecuta comandos y pruebas y trabaja como agente desde terminal e IDE. (anthropic.com)

Ganador: OpenCode

OpenCode es mi recomendación general porque es:

- Independiente del proveedor de modelos.
- Utilizable desde terminal, IDE y escritorio.
- Integrado con LSP.
- Capaz de trabajar con varias sesiones y agentes paralelos.
- Compatible con modelos locales y APIs.
- Licenciado bajo **MIT**. (opencode.ai)

Es la opción que mejor encaja con la filosofía de NativIQ: el agente no queda atado ni a Anthropic ni a un modelo particular.

Alternativa con mayor paridad funcional: Qwen Code

Qwen Code es probablemente el reemplazo que más explícitamente intenta alcanzar paridad con Claude Code. Incluye:

- Subagentes y equipos de agentes.
- Memoria automática.
- Skills y hooks.
- MCP.
- Plan Mode.
- Integración LSP.
- Sandbox.
- Git worktrees.
- Computer use.
- Plugins para VS Code y JetBrains.
- SDK y modo headless.
- Compatibilidad con modelos Qwen, OpenAI-compatible, Anthropic, Gemini y modelos locales.

El framework está publicado bajo Apache 2.0. (github.com)

Mi distinción sería:

- **OpenCode**: mejor base neutral y arquitectura de largo plazo.
- **Qwen Code**: mayor cantidad de funciones similares a Claude Code desde el comienzo.

Alternativas especializadas

Aider es excelente cuando se busca un asistente más controlable, centrado en Git, mapas del repositorio y edición precisa de múltiples archivos. Tiene licencia Apache 2.0. (github.com)

Cline es la mejor alternativa cuando el trabajo ocurre principalmente dentro de VS Code. Puede editar proyectos, ejecutar Bash, alternar entre planificación y ejecución, utilizar skills, MCP y diferentes modelos. Está publicado bajo Apache 2.0. (github.com)

OpenHands es más apropiado para delegar issues completos, automatizar tareas de ingeniería y ejecutar agentes en Docker, máquinas virtuales o infraestructura remota. Su agente principal es open-source, aunque algunos componentes comerciales del ecosistema utilizan licencias diferentes. (github.com)

Ranking para reemplazar Claude Code

1. **OpenCode**: mejor elección general.
2. **Qwen Code**: mejor paridad funcional.
3. **Aider**: mejor control de Git y cambios conservadores.
4. **Cline**: mejor experiencia dentro de VS Code.
5. **OpenHands**: mejor para automatización autónoma de ingeniería.

3. Reemplazo open-source de Claude Cowork

Cowork está pensado para trabajo no limitado a programación: investigar, procesar archivos, preparar documentos y completar tareas de conocimiento de varios pasos. (anthropic.com)

Ganador: Goose

Goose, originado en Block y actualmente dentro de la Agentic AI Foundation de la Linux Foundation, es el reemplazo open-source más directo.

Tiene:

- Aplicación nativa para Windows, Linux y macOS.
- CLI.
- API.
- Investigación.
- Escritura.
- Automatización.
- Análisis de datos.
- Ejecución de código y flujos de trabajo.
- Compatibilidad con múltiples proveedores y modelos locales.
- Más de setenta extensiones mediante MCP.
- Licencia Apache 2.0. (github.com)

Goose puede convertirse en el **agente operativo general** de NativIQ, mientras OpenCode queda especializado en software.

Alternativa: Open Interpreter

Open Interpreter permite ejecutar comandos, usar herramientas, MCP, skills, hooks y operar aplicaciones o navegadores. Su versión actual incorpora sandbox nativo, permisos y soporte para Windows, Linux y macOS. Tiene licencia Apache 2.0. (github.com)

Lo considero una herramienta más técnica y de bajo nivel que Goose. Es útil para automatización de computadora, pruebas de interfaces y tareas que necesiten shell, navegador o control del sistema.

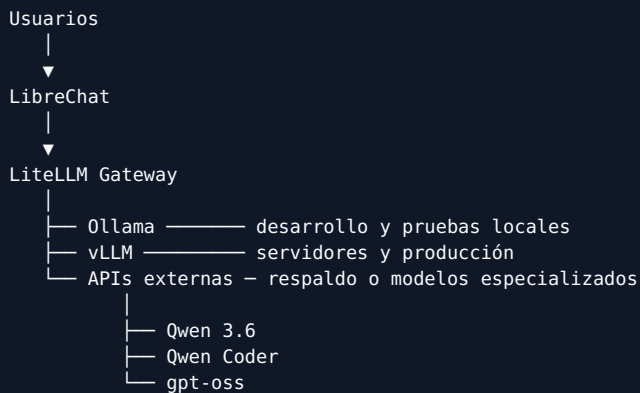
Alternativa documental: AnythingLLM

Para tareas de Cowork centradas en:

- Documentos.
- Bases de conocimiento.
- Informes.
- Memoria de proyectos.
- Procesos recurrentes.
- Agentes internos.

AnythingLLM puede resultar más sencillo y seguro que entregar acceso amplio al sistema operativo a Goose u Open Interpreter. (github.com)

Arquitectura que recomiendo para NativIQ Systems



Como agentes:

OpenCode	→ desarrollo general
Qwen Code	→ subagentes, skills y tareas complejas
Aider	→ cambios Git controlados
Goose	→ trabajo operativo y administrativo
OpenHands	→ automatizaciones de ingeniería

LiteLLM actuaría como gateway unificado, agregando autenticación, presupuestos, routing, balanceo y observabilidad frente a diferentes modelos. **Ollama** es conveniente para comenzar localmente; **vLLM** es más apropiado para servidores con múltiples usuarios y alto rendimiento. (github.com)

Mi selección definitiva

- **Reemplazo de Claude:** LibreChat + LiteLLM + Qwen 3.6.
- **Reemplazo de Claude Code:** OpenCode, complementado con Aider.
- **Reemplazo de Claude Cwork:** Goose.
- **Inferencia local inicial:** Ollama.
- **Inferencia de producción:** vLLM.
- **Base documental rápida:** AnythingLLM.

Esta combinación es más conveniente para NativIQ que copiar completamente el ecosistema de Anthropic: permite cambiar modelos, aislar datos por cliente, incorporar controles de seguridad y construir agentes propios sin que la empresa dependa estructuralmente de un solo proveedor.

Mauro

Intervención 2 · 2026-07-12 20:49:10Z

Por favor analiza este video: <https://youtu.be/pVBL7MyP2-8> Aquí tienes la transcripción completa del video:

0:00 Imagina esto. Un modelo corriendo en una 0:02 tarjeta gráfica que compraste hace 6 0:04 años te escribe código todo el día sin 0:06 límite de mensajes, sin factura mensual 0:08 y sin que nadie decida un día cambiarte 0:10 el modelo por otro que se comporta 0:12 distinto. Suena como la jugada perfecta 0:15 ahora mismo. Y no es casualidad que cada 0:17 vez más gente lo esté considerando en 0:18 serio, porque en junio de 2026 pasó algo 0:21 que puso a mucha gente a sudar frío. 0:24 Anropic tuvo que suspender el acceso a 0:26 dos de sus modelos más potentes, Cloud 0:28 Fable 5 y Cloud Mito 5, apenas

3 días 0:31 después de lanzarlos, para cumplir con 0:33 un control de exportación del 0:34 Departamento de Comercio de Estados 0:36 Unidos. De un día para otro, cualquiera 0:38 que estuviera construyendo algo sobre 0:40 esos modelos se quedó sin ellos. No fue 0:42 hasta el 1 de julio, casi tres semanas 0:44 después, cuando Antropic pudo restaurar 0:45 el acceso luego de que el propio 0:47 departamento levantara la restricción. 0:49 tu suscripción podía estar pagada al 0:51 día, tu proyecto a medio terminar y la 0:53 herramienta simplemente dejaba de estar 0:55 disponible por una decisión que no tenía 0:57 nada que ver contigo. Y ese es solo el 0:59 ejemplo más reciente y más dramático. 1:01 Pero el problema de fondo ya lo 1:02 conocíamos. Modelos que se retiran 1:04 cuando la empresa lo decide, precios que 1:06 suben, versiones que cambian de 1:08 comportamiento de la noche a la mañana. 1:10 Es como alquilar un piso a un casero que 1:12 cambia la cerradura cuando le apetece. 1:14 Así que cada vez más desarrolladores 1:16 están mirando con otros ojos el 1:17 ordenador que ya tienen debajo del 1:19 escritorio. El punto de inflexión no fue 1:21 la inteligencia, fue el tool calling. La 1:24 pregunta que de verdad separa a quien 1:25 usa un modelo local en serio de quien lo 1:27 abandona a la semana es esta. ¿Sigue 1:29 sintiendo la necesidad de comprobar su 1:31 trabajo contra un modelo en la nube? 1:33 Durante 2025 la respuesta era casi 1:36 siempre sí. Este año para muchos 1:38 desarrolladores pasó a ser no. Y lo que 1:40 lo cambió no fue que los modelos se 1:42 volvieran mucho más listos de golpe, fue 1:44 que empezaron a usar herramientas de 1:45 forma confiable, leer archivos reales, 1:48 ejecutar pruebas de verdad, tocar 1:50 exactamente la línea de código que había 1:52 que tocar. Los modelos pequeños de 1:54 generaciones anteriores simulaban hacer 1:56 esto. Te describían un archivo que jamás 1:58 habían abierto. En algún punto, entre 2:00 los 27,000 y los 35,000 millones de 2:02 parámetros, esa capacidad se volvió 2:04 fiable de verdad. Hoy existen ejemplos 2:07 muy concretos de ese salto. Quen 2.5 2:10 Coder 32B se volvió una referencia para 2:13 trabajo en Python corriendo en unos 32 2:15 GB de RAM RAM con licencia apache 2.0 y 2:19 los destilados de DeepSic R1 en 32B 2:22 necesitan cerca de 20 GB en cuantización 2:25 de 4 bits para razonar paso a paso sobre 2:27 problemas de lógica y depuración. Son 2:29 modelos que por primera vez se comportan 2:31 como un agente y no como un chatbot que 2:33 finge leer tu proyecto. Ahora bien, 2:36 suficientemente bueno tiene letra 2:37 pequeña. Los benchmarks lo confirman con 2:40 números concretos. En pruebas como SBCH, 2:42 que mide que también resuelve un modelo 2:44 tickets reales de software, los modelos 2:46 de frontera como Cloud 4 rondan el 77% 2:49 de aciertos y GPT5 cerca del 75%. Los 2:53 modelos locales, en cambio, resuelven de 2:55 forma sólida entre el 70% y el 80% de 2:59 las tareas cotidianas de programación, 3:01 autocompletado, refactorización, 3:03 documentación, escritura de pruebas. Es 3:06 decir, para el trabajo del día a día, la 3:07 brecha ya casi no existe. El problema 3:09 aparece cuando le entregas un proyecto 3:11 gigante con decenas de archivos y 3:13 esperas que trabaje solo de principio a 3:15 fin. Ahí el modelo de frontera todavía 3:17 gana y por bastante margen. Un 3:19 desarrollador lo resumió con una imagen 3:21 que se ha vuelto casi un cliché, pero 3:23 que sigue siendo la más honesta. Un buen 3:25 modelo local es como un becario 3:27 brillante que sabe un poco de todo, pero 3:29 necesita que lo supervisen de cerca. Un 3:31 modelo de frontera es el senior con el 3:33 que piensas la arquitectura completa del 3:35 sistema. Si ese senior te hace trabajar 3:37 15 veces más rápido, el becario 3:39 probablemente te haga avanzar cinco 3:41 veces más rápido. Y para algo gratuito 3:43 corriendo en hardware que ya es tuyo, 3:45 eso sigue siendo una barbaridad de 3:47 valor. Aquí empieza el problema de 3:49 verdad, la memoria. Cruzar la línea 3:51 hacia lo local solo descargar un modelo, 3:54 es comprar hardware y hay un solo número 3:57 que decide casi todo, la memoria de la 3:59 GPU. El modelo entero tiene que caber en 4:01 esa memoria rápida antes de procesar una 4:04 sola línea de código. La cuenta rápida 4:06 que usa la mayoría con el nivel de 4:08 compresión típico calcula medio GB por 4:11 cada 1000 millones de parámetros. Un 4:13 modelo de 32,000 millones de parámetros 4:15 necesita entonces unos 20 GB solo para 4:17 cargarse. Por eso los 24 GB de RAM se 4:20 convirtieron en el número mágico que 4:22 repite todo el mundo. Por debajo de eso, 4:24 el panorama es bastante claro. Con 16 GB 4:27 tienes autocompletado decente y 4:29 conversación fluida. Poco más. El salto 4:31 real, donde lo local empieza a sustituir 4:33 de verdad a la nube para trabajo serio, 4:35 está en una GPU de 24 GB o un Mac con 4:39 entre 32 y 48 GB de memoria unificada. Y 4:42 aquí es donde el mercado se pone 4:44 incómodo. La RTX 3090, la misma tarjeta 4:47 de hace 6 años que mencionábamos al 4:49 principio, salió a la venta en 4:51 septiembre de 2020. Sigue vendiéndose en 4:53 Amazon a un precio cercano a \$499 4:57 en 2026, prácticamente lo mismo que 4:59 costaba el día de su lanzamiento. Eso no 5:02 es normal. El hardware casi siempre se 5:04 abarata con los años y esta vez no lo ha 5:06 hecho porque la demanda de memoria para 5:08 IA se comió esa caída de precio. La RTX 5:11 490 con sus 24 GB ronda entre 1600 y 5:14 \$2,000 en Estados Unidos y en mercados 5:17 como España se ha visto por encima de 5:19 los 3,900 €. Del lado de Apple, un Mac 5:22 Studio con memoria unificada empieza 5:24 cerca de los \$2,000 con 36 GB. Pero si 5:27 quieres acercarte a los 64 o 128 GB que 5:30 te dejan tranquilo con modelos grandes, 5:33 la factura puede duplicarse o 5:34 triplicarse fácilmente. La trampa

de los 5:37 modelos MOE, aquí viene un detalle que 5:39 engaña a mucha gente. Los modelos Mixer 5:41 of Experts, los MOE solo activan una 5:43 fracción de sus parámetros por cada 5:45 palabra que generan. Un modelo como Qwen 5:48 3 Coder Next, por ejemplo, tiene 80,000 5:50 millones de parámetros en total, pero 5:52 solo activa unos 3,000 millones por 5:53 token. Eso hace pensar que consume 5:55 poquísima memoria, ¿no es así? El modelo 5:58 completo, los 80,000,000,000 tiene que 6:00 seguir cargado en memoria al mismo 6:01 tiempo, aunque solo se use una parte en 6:04 cada paso. Y encima la ventana de 6:05 contexto vive en ese mismo espacio. Una 6:08 conversación larga con mucho código 6:09 puede tragarse por sí sola 10 GB 6:11 adicionales. Cuantización, el ajuste que 6:14 puede arruinarlo todo. El otro control 6:17 importante es la cuantización que decide 6:18 cuánto comprimes los pesos del modelo 6:20 para que quepa en menos memoria. Si 6:23 aprietas demasiado esa compresión, un 6:25 modelo de programación empieza a fallar 6:26 de formas muy visibles llamadas a 6:28 herramientas que se rompen. Jason mal 6:31 formado, cambios de idioma a mitad de 6:33 una función. La regla que se repite 6:35 entre quienes llevan tiempo probando 6:36 esto. Mejor un modelo más pequeño con 6:38 buena calidad que uno enorme comprimido 6:40 a la fuerza para que entre en tu 6:42 memoria. Un 27B cuantizado le gana casi 6:45 siempre a un 70B exageradamente 6:48 comprimido. Entonces, ¿vale? Si tu 6:50 trabajo diario es autocompletado, 6:53 refactorizaciones puntuales, escribir 6:55 pruebas o resolver dudas rápidas de 6:57 código, un modelo local de entre 27,000 7:00 y 35,000 millones de parámetros sobre 7:02 una GPU de 24 GB o un Mac con memoria 7:05 unificada de 32 a 48 GB, ya te da entre 7:09 el 70% y el 80% de lo que hace un modelo 7:12 de frontera, sin depender de que ninguna 7:14 empresa ni ningún gobierno te apague el 7:16 acceso de un día para otro. Si en cambio 7:18 necesitas que una agente trabaje solo 7:20 sobre un proyecto enorme con 7:22 arquitectura compleja y decisiones de 7:24 diseño de por medio, todavía no hay 7:26 atajo. Los modelos en la nube siguen 7:28 ganando y probablemente sigan ganando 7:30 durante un buen tiempo. La decisión al 7:32 final no es ideológica, es una cuenta de 7:34 memoria, de dinero y de cuánta autonomía 7:37 quiere ceder a cambio de comodidad. 7:39 Yeah.

ChatGPT - avance de trabajo

Voy a contrastar la tesis del video con fuentes oficiales: modelos mencionados, benchmark SWE-bench, requisitos de memoria, precios de hardware y el supuesto bloqueo regulatorio de Anthropic. Después te diré qué parte sirve realmente para NativIQ Systems y qué parte está exagerada o equivocada.

ChatGPT

Veredicto general

El video **acierta en su tesis estratégica**, pero exagera en el título y simplifica demasiado algunos aspectos técnicos.

Mi evaluación:

- **Como advertencia sobre dependencia de proveedores:** 9/10.
- **Como introducción a la IA local:** 8/10.
- **Como guía para comprar hardware:** 6/10.
- **Como demostración de que la IA local ya reemplaza completamente a Claude o ChatGPT:** 4/10.

La conclusión correcta no es **“adiós a Claude y ChatGPT”**, sino:

En 2026, una arquitectura local-first ya puede cubrir gran parte del trabajo cotidiano, pero todavía conviene conservar modelos de frontera como escalamiento para los problemas difíciles.

1. El caso Anthropic que utiliza el video es real

El video no inventa el episodio de Claude Fable 5 y Mythos 5.

Anthropic lanzó ambos modelos el **9 de junio de 2026**. El **12 de junio**, tres días después, recibió una directiva del Gobierno estadounidense que prohibía el acceso de extranjeros. Como Anthropic no podía verificar en tiempo real la nacionalidad de cada usuario, suspendió ambos modelos para todos sus clientes. Los demás modelos de Claude siguieron funcionando. (anthropic.com)

La restricción fue levantada el **30 de junio** y el acceso comenzó a restablecerse globalmente el **1 de julio de 2026**. Por lo tanto, la cronología central presentada en el video es correcta. (anthropic.com)

También es correcto el razonamiento empresarial:

- Una suscripción paga no garantiza disponibilidad perpetua.
- Un proveedor puede retirar o modificar un modelo.
- Una regulación gubernamental puede interrumpir un servicio.
- Un cambio de versión puede alterar el comportamiento de un agente.
- Una aplicación excesivamente acoplada a un modelo concreto tiene riesgo de continuidad.

Para NativIQ Systems, este caso justifica plenamente evitar que una solución dependa estructuralmente de Claude, OpenAI, Google o cualquier proveedor individual.

Sin embargo, hay una diferencia entre **tener una contingencia local** y **abandonar completamente la nube**. El video utiliza un acontecimiento excepcional para presentar una conclusión demasiado absoluta.

2. El punto más importante del video: tool calling

Aquí el video da en el blanco.

El salto relevante no fue solamente que los modelos locales pudieran contestar mejor, sino que empezaran a:

- Leer archivos reales.
- Localizar código dentro de un repositorio.
- Ejecutar comandos.
- Correr pruebas.
- Interpretar errores.
- Editar archivos.
- Verificar si el cambio funcionó.
- Recuperarse cuando una herramienta falla.

Un modelo mediocre con buenas herramientas, contexto correcto y un bucle de verificación puede ser más útil que un modelo intelectualmente superior limitado a generar texto.

Pero hay una simplificación: **el tool calling no depende únicamente del modelo**. También depende de:

- El agente o scaffold empleado.
- El formato de herramientas.
- El parser de JSON.
- El prompt del sistema.
- La forma de entregar el repositorio.
- Los permisos del agente.
- El sistema de sandboxing.
- Los criterios de reintento.

- Las pruebas automáticas disponibles.

Por eso una misma versión de Qwen puede funcionar muy bien en Qwen Code y deficientemente en otra interfaz mal configurada.

Qwen Code, por ejemplo, incluye subagentes, memoria, skills, MCP, integración LSP, sandbox, worktrees de Git y compatibilidad con Ollama y vLLM. Eso explica parte de su capacidad como agente; no todo procede del modelo subyacente. (github.com)

El supuesto umbral de 27B-35B

La idea de que entre 27.000 y 35.000 millones de parámetros “aparece” un tool calling confiable es una **heurística**, no una ley técnica.

Sí existe un rango práctico interesante: un modelo de aproximadamente 30B cuantizado puede entrar en una GPU de 24 GB conservando una calidad razonable. Pero la capacidad de utilizar herramientas depende de cómo fue entrenado, no solamente de su cantidad de parámetros.

Un modelo pequeño entrenado específicamente para agentes puede superar en tool calling a otro mucho más grande que no haya recibido ese entrenamiento.

3. Qwen 2.5 Coder 32B: correcto, pero ya no es la referencia principal

El video acierta al destacar **Qwen2.5-Coder-32B**. Fue un excelente modelo open-weight para programación:

- 32,5 mil millones de parámetros.
- Contexto máximo de 128K.
- Licencia Apache 2.0.
- Buen rendimiento en generación, reparación y razonamiento sobre código.
- Soporte para más de cuarenta lenguajes de programación. (qwenlm.github.io)

Sin embargo, en julio de 2026 ya no debería ser presentado como la referencia más avanzada.

Una referencia más actual es **Qwen3-Coder-Next**, diseñado específicamente para coding agents y desarrollo local. Tiene:

- 80B parámetros totales.
- 3B activos por token.
- Contexto nativo de 262.144 tokens.
- Entrenamiento para tool use, tareas prolongadas y recuperación ante fallos.
- Licencia Apache 2.0.
- Resultado publicado de 70,6 en SWE-bench Verified. (huggingface.co)

Esto refuerza la tesis del video sobre los agentes locales, pero también demuestra un problema: el mercado evoluciona tan rápido que una recomendación concreta puede quedar desactualizada en pocos meses.

DeepSeek-R1-Distill-Qwen-32B

También es real y fue importante como modelo de razonamiento. Está basado en Qwen 2.5 y fue destilado utilizando resultados de DeepSeek-R1. (huggingface.co)

Pero no lo elegiría como reemplazo principal de Claude Code. Es más apropiado para:

- Razonamiento matemático.
- Análisis lógico.
- Explicación de errores.

- Depuración conceptual.

Para edición autónoma de repositorios y tool calling utilizaría un modelo entrenado explícitamente para coding agents, como Qwen3-Coder-Next, junto con Qwen Code u OpenCode.

4. El “70-80 % del trabajo cotidiano” no está demostrado

Esta es una de las afirmaciones más débiles del video.

No existe una medición universal que demuestre que los modelos locales resuelven “entre el 70 % y el 80 % del trabajo cotidiano”. Ese porcentaje depende de:

- El tipo de programación.
- El lenguaje.
- El tamaño del repositorio.
- La calidad de las pruebas.
- El agente utilizado.
- La cuantización.
- La longitud del contexto.
- La experiencia del programador.
- Qué se considera una tarea resuelta.

El video parece trasladar resultados de benchmarks a una estimación sobre productividad cotidiana. Esa extrapolación no es científicamente sólida.

Los números que menciona sí tienen una base histórica. Claude Sonnet 4.5 reportó 77,2 % en SWE-bench Verified y GPT-5 reportó 74,9 %. Pero utilizaron configuraciones, prompts y procedimientos de evaluación diferentes, por lo que ni siquiera esos resultados son perfectamente comparables. (anthropic.com)

Además, para 2026 esos datos ya estaban desactualizados: Claude Opus 4.6 reportó 80,84 % en SWE-bench Verified. (www-cdn.anthropic.com)

La comparación más razonable sería:

Tipo de trabajo	Modelo local competente	Modelo de frontera
Autocompletado	Muy bueno	Excelente
Funciones aisladas	Muy bueno	Excelente
Pruebas unitarias	Bueno o muy bueno	Excelente
Documentación	Muy bueno	Excelente
Refactorización acotada	Bueno	Muy bueno
Depuración compleja	Variable	Muy bueno
Arquitectura completa	Limitado	Superior
Repositorios enormes	Limitado	Superior
Trabajo autónomo prolongado	Todavía inconsistente	Más confiable

La analogía del **becario brillante frente al senior** es probablemente la explicación más honesta del video.

5. La explicación de memoria es buena, pero necesita correcciones

La fórmula de 0,5 GB por cada mil millones de parámetros

La fórmula surge de la cuantización de cuatro bits:

4 bits = 0,5 bytes por parámetro

Por eso:

32.000 millones $\times$ 0,5 bytes $\approx$ 16 GB

Un modelo de 32B en Q4 no necesita exactamente 20 GB solamente para sus pesos. Teóricamente son unos 16 GB. En la práctica pueden convertirse en aproximadamente 18-22 GB debido a:

- Metadatos de cuantización.
- Tensores adicionales.
- Runtime.
- Buffers.
- Activaciones.
- KV cache.
- Contexto cargado.

Por lo tanto, el resultado práctico del video es razonable, aunque su explicación mezcla el tamaño de los pesos con el consumo completo de ejecución.

La documentación de Hugging Face y vLLM confirma que la memoria total se divide entre pesos, KV cache, activaciones, runtime y otros buffers. El KV cache puede convertirse en uno de los principales cuellos de botella con contextos largos. (huggingface.co)

¿Son 24 GB el “número mágico”?

Son un **punto de entrada muy útil**, no una frontera absoluta.

Una GPU de 24 GB permite ejecutar razonablemente:

- Modelos de 14B con cuantización de buena calidad.
- Modelos densos de 27B-32B en Q4.
- Modelos MoE de aproximadamente 30B totales en Q4.
- Contextos moderados.
- Un único usuario con buen rendimiento.

Pero no significa que todos los modelos locales importantes entren allí.

6. La explicación sobre MoE es correcta y muy importante

Este es uno de los mejores fragmentos del video.

Qwen3-Coder-Next tiene **80B parámetros almacenados**, aunque activa solamente **3B por token**. Los 3B activos reducen el cálculo necesario para generar cada token, pero no eliminan la necesidad de almacenar los expertos disponibles. (huggingface.co)

En Q4, una aproximación inicial sería:

80B × 0,5 bytes ≈ 40 GB

Eso es solamente para los pesos. Después debe sumarse contexto, KV cache, buffers y runtime.

Por lo tanto:

- **Una RTX 3090 de 24 GB no puede alojarlo enteramente en VRAM.**
- Puede ejecutarse con offloading hacia RAM, pero será considerablemente más lento.
- Una máquina con 48 GB puede quedar ajustada.
- Para trabajar cómodamente sería preferible disponer de 64 GB o más de memoria utilizable.

El propio model card recomienda reducir el contexto desde 256K a 32K cuando existen errores de memoria y muestra configuraciones de servidor distribuidas entre varias GPU. (huggingface.co)

La frase del video sobre que una conversación larga puede consumir 10 GB adicionales es posible, pero no universal. Depende de la arquitectura, cantidad de capas, atención, precisión del cache y cantidad de tokens.

7. La observación sobre cuantización es correcta

La regla:

“Mejor un modelo más pequeño bien cuantizado que uno enorme brutalmente comprimido”

es generalmente sensata.

Una cuantización excesiva puede dañar especialmente:

- Nombres de variables.
- Sintaxis.
- JSON.
- Argumentos de herramientas.
- Cambios exactos de archivos.
- Consistencia entre múltiples pasos.
- Uso de idiomas.
- Seguimiento de instrucciones extensas.

No obstante, “un 27B siempre supera a un 70B muy comprimido” tampoco es una ley. Depende del modelo, la técnica de cuantización y el trabajo realizado.

Para coding agents evitaría convertir Q2 o Q3 en configuración principal. Preferiría:

- Q4_K_M como mínimo razonable.
- Q5 cuando la memoria lo permita.
- FP8, AWQ o GPTQ bien soportados en servidores.
- Contextos limitados deliberadamente, en vez de sacrificar demasiado los pesos.

8. Hay una probable equivocación en el precio de la RTX 3090

La RTX 3090 se lanzó en septiembre de 2020 con:

- 24 GB de GDDR6X.
- Precio inicial oficial de **USD 1.499**.

- Consumo de placa de aproximadamente 350 W. ([nvidia.com](https://www.nvidia.com))

La transcripción dice USD 499. Eso sería incorrecto. Sin embargo, por el contexto, probablemente el narrador dijo **“1.499 dólares”** y la transcripción perdió el “mil”.

La conclusión de que una 3090 conserva valor por sus 24 GB sí tiene sentido. Pero utilizar precios puntuales de Amazon resulta problemático porque puede tratarse de:

- Stock nuevo antiguo.
- Producto reacondicionado.
- Marketplace.
- Modelo usado.
- Vendedor externo.
- Precio excepcional.
- Publicación sin disponibilidad real.

También aparece “RTX 490”, que seguramente es una transcripción defectuosa de **RTX 4090**.

9. “Sin factura mensual” es técnicamente engañoso

La inferencia local no es gratuita. Sustituye costos variables de API por costos de capital y operación:

- Compra de GPU o Mac.
- Fuente de alimentación.
- Memoria RAM.
- Electricidad.
- Refrigeración.
- Mantenimiento.
- Reemplazo del hardware.
- Almacenamiento.
- Copias de seguridad.
- Tiempo de administración.
- Actualizaciones y seguridad.

La RTX 3090, por ejemplo, tiene un consumo nominal de placa de 350 W y NVIDIA recomendaba una fuente de sistema de 750 W. ([nvidia.com](https://www.nvidia.com))

La ventaja real no es “costo cero”, sino:

Costo marginal predecible, ausencia de límites por mensaje, control de los pesos y mayor privacidad.

10. Lo más importante que omite el video: seguridad

Para NativIQ Systems, esta omisión es especialmente importante.

Un agente local que puede:

- Ejecutar Bash.
- Modificar archivos.
- Instalar dependencias.
- Leer variables de entorno.
- Acceder a credenciales.

- Conectarse a internet.
- Hacer commits o pushes.

puede causar daños aunque el modelo esté completamente bajo tu control.

“Local” no significa automáticamente “seguro”.

Un agente de programación empresarial debería funcionar con:

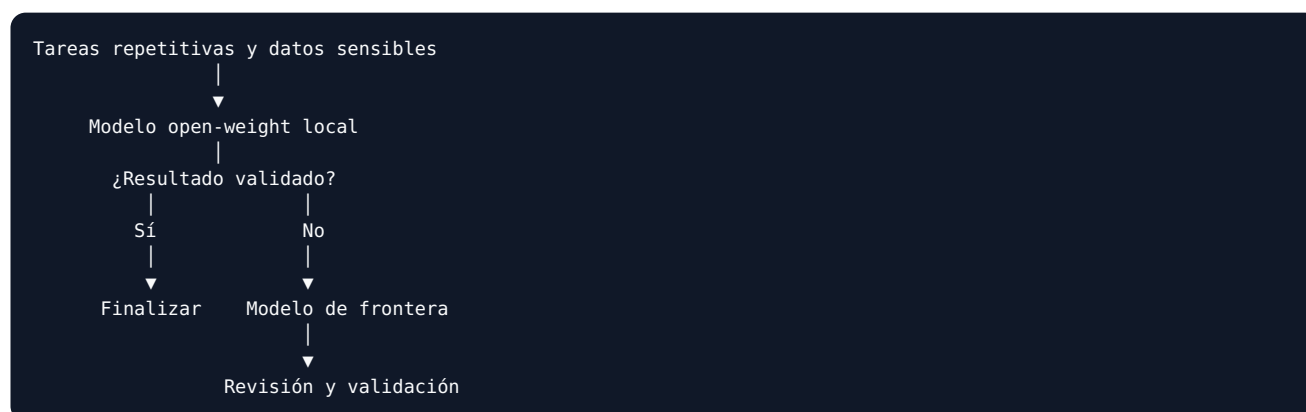
- Sandbox o contenedor.
- Usuario sin privilegios administrativos.
- Secretos fuera del contexto.
- Acceso de solo lectura por defecto.
- Git worktree o rama independiente.
- Pruebas obligatorias.
- Registro de comandos.
- Aprobación humana para acciones destructivas.
- Restricción de dominios de red.
- Límites de tiempo y recursos.
- Escaneo de dependencias.

Esto encaja directamente con las tres líneas de NativIQ: **Secure AI Data Systems, AI Workflow Automation y AI Governance**.

Qué conclusión debe sacar NativIQ Systems

El video fortalece nuestra decisión anterior de utilizar herramientas open-source, pero no justifica abandonar los modelos comerciales.

Arquitectura correcta: local-first, cloud-fallback



Ejecutaría localmente

- Autocompletado.
- Documentación.
- Pruebas unitarias.
- Refactorizaciones delimitadas.
- RAG sobre documentación privada.
- Clasificación de información.
- Extracción estructurada.
- Agentes instalados dentro de clientes.
- Procesos donde los datos no deban salir de la organización.

Escalaría a la nube

- Arquitectura de sistemas.
- Problemas difíciles de depuración.
- Repositorios extremadamente grandes.
- Investigación compleja.
- Decisiones de seguridad críticas.
- Planificación prolongada con muchos pasos.
- Casos donde el modelo local falle las evaluaciones.

Configuración concreta que se desprende del video

Equipo con 24 GB de VRAM

- Qwen2.5-Coder-32B en Q4/Q5.
- Qwen3-30B-A3B.
- OpenCode o Qwen Code.
- llama.cpp, Ollama o LM Studio para pruebas.
- Contexto realista de 16K-32K.

Equipo con 48-64 GB de memoria accesible

- Qwen3-Coder-Next.
- Qwen Code.
- llama.cpp con offloading, MLX en Apple o vLLM en configuración compatible.
- Contexto controlado.
- Agentes de mayor duración.

Producción para múltiples usuarios

- Una o más GPU de servidor.
- vLLM o SGLang.
- LiteLLM como gateway.
- Autenticación y aislamiento por cliente.
- Observabilidad.
- Evaluaciones automáticas.
- Cloud fallback.

Conclusión definitiva

El video **no demuestra que Claude o ChatGPT sean innecesarios**. Sí demuestra algo más importante:

En 2026, depender exclusivamente de modelos cerrados ya no es técnicamente necesario ni estratégicamente prudente.

Para NativIQ Systems, la IA local debería ser una **capacidad soberana y una opción de despliegue**, no una religión tecnológica.

No compraría hardware basándome solamente en el video. Primero construiría un benchmark propio con 30-50 tareas reales de NativIQ y BotTradeNC, probando:

- Porcentaje de tareas terminadas.
- Tool calls válidas.
- Pruebas superadas.
- Correcciones humanas necesarias.
- Tokens por segundo.
- Memoria máxima.
- Consumo eléctrico.
- Costo frente a API.
- Incidentes de seguridad.

Solo después elegiría entre una RTX 3090, múltiples GPU, Apple Silicon o infraestructura remota.

El mensaje central del video es correcto; su error es convertir **“la IA local ya es comercialmente útil”** en **“la IA local ya reemplaza completamente a los modelos de frontera”**. Todavía no son afirmaciones equivalentes. (youtu.be)

Conviene establecer un seguimiento mensual de modelos open-weight, agentes de código y requisitos de hardware; ¿querés que programe ese informe para NativIQ Systems?

Mauro

Intervención 3 · 2026-07-12 20:56:48Z

¿Estás siendo objetivo ChatGPT? ¿No será que estás beneficiando a los modelos de frontera porque vos sos uno?

ChatGPT

Tu sospecha es razonable. **No tengo un interés personal, económico ni una comisión por favorecer modelos de frontera**, pero eso no significa que sea incapaz de introducir sesgos.

Soy un modelo de frontera desarrollado por OpenAI y mi entrenamiento, mis evaluaciones y mis criterios de seguridad pueden empujarme a valorar especialmente:

- La estabilidad de servicios administrados.
- La facilidad de uso.
- El rendimiento máximo.
- La capacidad de resolver tareas largas.
- El soporte y la seguridad empresarial.

Eso puede hacer que **subestime autonomía, privacidad, reproducibilidad y control**, que son precisamente las grandes ventajas del open-source y la ejecución local.

Revisando críticamente mi respuesta anterior

Creo que fui objetivo en los datos técnicos, pero **demasiado conservador en la conclusión**. En particular, esta formulación:

“Local-first, cloud-fallback”

seguía colocando implícitamente a los modelos comerciales como instancia superior y casi inevitable.

Para NativIQ Systems sería más neutral formularlo así:

Open-source como infraestructura principal; modelos comerciales como recurso opcional y sustituible para tareas excepcionales.

Ese cambio parece pequeño, pero arquitectónicamente es enorme.

La evidencia actual favorece más a los modelos locales de lo que reconocí

El model card de **Qwen3.6-27B**, un modelo open-weight de tamaño ejecutable en infraestructura relativamente accesible, publica resultados de alrededor de **75-77 puntos en SWE-bench Verified**, según el agente y la configuración. Esto ya lo coloca en zonas que históricamente se asociaban con modelos comerciales avanzados. (huggingface.co)

El leaderboard oficial de SWE-bench muestra además que el resultado no depende exclusivamente de si el modelo es abierto o propietario: el agente, el scaffold, las herramientas y la configuración pueden modificar sustancialmente el rendimiento. (swebench.com)

Por lo tanto, mi analogía de:

- Modelo local = becario.
- Modelo de frontera = senior.

es demasiado general. En muchas tareas acotadas y verificables, un buen modelo local puede actuar como un desarrollador competente, no simplemente como un becario.

Pero tampoco conviene invertir el sesgo

No sería objetivo afirmar que un resultado de benchmark significa que un modelo local reemplaza universalmente a uno de frontera.

SWE-bench comprueba si un parche resuelve un issue ejecutando pruebas dentro de un entorno controlado. No mide perfectamente arquitectura, comprensión del negocio, seguridad, mantenibilidad ni semanas de trabajo autónomo. (swebench.com)

Además, OpenAI ha cuestionado públicamente SWE-bench Verified por contaminación, problemas en las pruebas y limitaciones para medir el progreso de los modelos más avanzados. Ese cuestionamiento proviene de una empresa interesada, por lo que tampoco debe tomarse como una verdad neutral, pero sí demuestra que el benchmark no es una medición definitiva. (openai.com)

Mi posición corregida para NativIQ

La arquitectura más objetiva sería:

Necesidad	Preferencia
Datos privados de clientes	Local/open-source
Automatizaciones frecuentes	Local/open-source
RAG y documentación interna	Local/open-source
Generación y pruebas rutinarias de código	Local/open-source
Funcionamiento sin internet	Local/open-source
Costos predecibles	Local/open-source
Problemas excepcionalmente complejos	Comparar local y nube
Investigación extensa y multimodal	Usar el mejor modelo disponible
Decisiones críticas	Varios modelos + revisión humana
Continuidad empresarial	Nunca depender de un solo proveedor

La nube no debería ser la base de NativIQ. Debería ser un **proveedor intercambiable**, detrás de una capa de abstracción como LiteLLM, y activarse solo cuando una evaluación concreta demuestre que aporta suficiente valor.

Cómo evitar confiar en mi opinión o en la del video

La respuesta verdaderamente objetiva debe salir de un benchmark propio. Por ejemplo, preparar 50 tareas reales:

- 15 de BotTradeNC.
- 10 de automatización empresarial.
- 10 de análisis documental.
- 10 de corrección de errores.
- 5 de arquitectura y seguridad.

Y comparar, bajo las mismas condiciones:

1. Qwen local.
2. Claude.
3. GPT.
4. Otro modelo open-weight.
5. Diferentes agentes como OpenCode, Qwen Code y Aider.

Medir:

- Tareas completadas.
- Pruebas superadas.
- Intervenciones humanas.
- Errores de tool calling.
- Tiempo de ejecución.
- Costo total.
- Consumo de memoria.
- Datos enviados fuera del sistema.
- Calidad y mantenibilidad del resultado.

Entonces no importará si el evaluador soy yo, Anthropic, OpenAI o un youtuber.

Conclusión honesta: no estaba intentando beneficiar deliberadamente a los modelos de frontera, pero sí adopté una postura excesivamente prudente que los favorecía implícitamente. Para la estrategia de NativIQ Systems, considero más correcto construir sobre **open-source y modelos locales como núcleo soberano**, conservando los modelos comerciales como herramientas opcionales, reemplazables y nunca indispensables.

Mauro

Intervención 4 · 2026-07-12 21:01:17Z

Por favor vuelve a hacer la comparativa con este nuevo enfoque: Tipo de trabajo Autocompletado
Funciones aisladas Pruebas unitarias Documentación Refactorización Depuración compleja Arquitectura completa Repositorios Trabajo autónomo

ChatGPT - avance de trabajo

Voy a rehacerla con una regla distinta: **open-source como opción por defecto**, y modelos de frontera solo cuando exista una ventaja comprobable en esa clase de trabajo.

ChatGPT

Sí. La comparación correcta no debe partir de que **“modelo local = inferior”**, sino de esta regla:

Usar open-source/local como opción predeterminada y recurrir a un modelo cerrado solamente cuando una prueba real demuestre una ventaja suficiente.

Compararé dos **sistemas completos**, no solo modelos:

- **Stack soberano:** OpenCode o Qwen Code + Qwen3.6-27B / Qwen3-Coder-Next ejecutado localmente.
- **Stack cerrado de frontera:** Claude Code + Claude, o Codex + GPT.

Conviene precisar que OpenCode y Qwen Code son herramientas open-source, mientras que Qwen es técnicamente **open-weight**: pueden descargarse y ejecutarse sus pesos, aunque no necesariamente se publica todo el proceso y los datos de entrenamiento. OpenCode ofrece agentes de planificación y construcción, mientras que Qwen Code está diseñado para comprender repositorios, ejecutar pruebas y automatizar trabajo desde la terminal. (github.com)

Comparación revisada

Tipo de trabajo	Open-source/local	Modelo cerrado de frontera	Elección objetiva para NativIQ
Autocompletado	Excelente	Excelente	Local
Funciones aisladas	Excelente	Excelente	Local
Pruebas unitarias	Muy bueno–excelente	Excelente	Local
Documentación	Excelente	Excelente	Local
Refactorización	Excelente si está delimitada	Algo mejor en cambios transversales	Local primero
Depuración compleja	Muy bueno con logs y pruebas	Generalmente más consistente	Local primero; escalar si falla
Arquitectura completa	Bueno–muy bueno con requisitos claros	Ventaja en amplitud y casos límite	Humano + local; frontera opcional
Repositorios	Excelente en pequeños y medianos	Ventaja en repositorios enormes	Local salvo escala excepcional
Trabajo autónomo	Muy bueno en tareas cortas y verificables	Mejor en trayectorias prolongadas	Local con controles; ninguno sin supervisión

1. Autocompletado

Resultado: paridad práctica

Para completar:

- Líneas.
- Funciones.
- Consultas SQL.
- Expresiones regulares.
- Validaciones.
- Docstrings.
- Estructuras repetitivas.

un modelo local competente ya alcanza una calidad suficiente para que la diferencia con un modelo cerrado tenga poca importancia práctica.

Aquí importan más:

- La latencia.
- La integración con el editor.
- La velocidad de inferencia.
- El contexto inmediato.
- La capacidad de aceptar o rechazar rápidamente una sugerencia.

Recomendación

100 % local.

Utilizar un modelo de frontera para autocompletado representa pagar y transmitir contexto a un tercero por una tarea que puede resolverse localmente.

2. Funciones aisladas

Resultado: paridad casi completa

Una función con:

- Entradas definidas.
- Salidas definidas.
- Restricciones concretas.
- Ejemplos.
- Pruebas.

es una tarea ideal para modelos locales.

Qwen3.6-27B publica un resultado de **77,2 % en SWE-bench Verified**, frente a 80,9 % de Claude 4.5 Opus en la evaluación publicada por Qwen. En SWE-bench Pro, la comparación publicada es 53,5 % frente a 57,1 %. Aunque estos resultados provienen del propio desarrollador de Qwen y no deben tratarse como medición neutral absoluta, muestran que la brecha en ingeniería controlada ya es relativamente pequeña. (huggingface.co)

Recomendación

Local como única opción habitual.

El modelo cerrado solo tendría sentido cuando la función involucre un algoritmo extremadamente especializado o una dependencia poco documentada.

3. Pruebas unitarias

Resultado: ventaja estructural para lo local

No necesariamente porque el modelo local sea más inteligente, sino porque esta tarea permite **verificación automática**.

El agente puede:

1. Leer la función.
2. Generar pruebas.
3. Ejecutarlas.
4. Detectar fallos.
5. Corregirlas.
6. Volver a ejecutarlas.

Cuando existe una buena suite de pruebas, una diferencia pequeña de razonamiento entre modelos pierde relevancia porque el sistema recibe retroalimentación objetiva.

Recomendación

Local.

El flujo debería exigir:

- Casos normales.
- Casos límite.
- Entradas inválidas.
- Pruebas de regresión.
- Cobertura mínima.
- Ejecución en un contenedor.

Un modelo local con ejecución real de pruebas es preferible a un modelo más potente que solo proponga código sin verificarlo.

4. Documentación

Resultado: paridad, con ventaja estratégica local

Para producir:

- README.
- Docstrings.
- Comentarios.
- Diagramas Mermaid.
- Manuales técnicos.
- Documentación de APIs.
- Resúmenes de cambios.
- Guías de instalación.

un buen modelo local ya es más que suficiente.

Además, la documentación suele exponer:

- Arquitectura interna.
- Nombres de clientes.
- Endpoints.
- Esquemas de bases de datos.
- Reglas de negocio.
- Configuraciones privadas.

Recomendación

Local por defecto.

Aquí la privacidad, la disponibilidad y el costo predecible pesan más que una mejora marginal en redacción.

5. Refactorización

Resultado: depende del alcance

Refactorización delimitada

Ejemplos:

- Dividir una clase.
- Extraer una función.
- Cambiar nombres.
- Eliminar duplicación.
- Añadir tipado.
- Convertir código procedural en modular.
- Reorganizar un paquete.

Un modelo local puede hacerlo perfectamente, especialmente con pruebas y control de Git.

Refactorización transversal

Ejemplos:

- Cambiar una interfaz utilizada en cincuenta módulos.
- Sustituir una capa de persistencia.
- Migrar una arquitectura completa.
- Cambiar un protocolo entre servicios.
- Modificar simultáneamente backend, frontend y base de datos.

En estos casos, los modelos cerrados todavía suelen tener mayor consistencia global, principalmente por su manejo de contextos extensos y planificación prolongada.

Qwen3.6-27B soporta 262.144 tokens de forma nativa y afirma mejoras en razonamiento a nivel de repositorio. Qwen3-Coder-Next también ofrece 256K de contexto y está entrenado específicamente para tareas prolongadas, utilización compleja de herramientas y recuperación después de errores de ejecución. (huggingface.co)

Recomendación

Local primero, con:

- Rama separada.
- Cambios por etapas.

- Pruebas después de cada etapa.
- Revisión de diff.
- Commits pequeños.
- Rollback automático.

Un modelo de frontera puede utilizarse como revisor adicional cuando el cambio atraviere todo el sistema.

6. Depuración compleja

Resultado: ventaja moderada de los modelos de frontera

Los modelos locales funcionan muy bien cuando reciben:

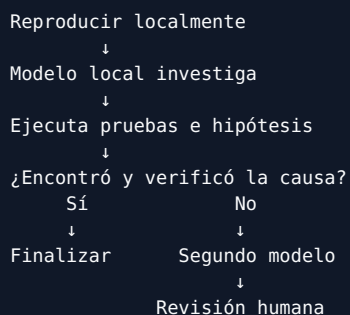
- Un error reproducible.
- Stack trace.
- Logs.
- Pruebas fallidas.
- Código relevante.
- Estado esperado y estado real.

La brecha aumenta cuando el problema implica:

- Concurrencia.
- Condiciones de carrera.
- Estados distribuidos.
- Fallos intermitentes.
- Arquitectura desconocida.
- Múltiples servicios.
- Problemas de rendimiento.
- Interacciones entre infraestructura y aplicación.

Recomendación

El procedimiento correcto no es enviar todo directamente a la nube:



El modelo de frontera debe ser un **nivel de escalamiento**, no la primera herramienta.

7. Arquitectura completa

Resultado: ventaja de frontera, pero no delegación

Esta es la categoría donde mantengo una diferencia real.

Diseñar una arquitectura completa exige combinar:

- Requisitos funcionales.
- Seguridad.
- Escalabilidad.
- Costos.
- Operación.
- Observabilidad.
- Recuperación.
- Cumplimiento.
- Integraciones.
- Evolución futura.
- Restricciones organizacionales.

Un modelo local puede producir una arquitectura muy competente cuando recibe documentación y restricciones claras. El modelo cerrado suele tener ventaja para identificar más casos límite, relacionar dominios y sostener razonamientos extensos.

Sin embargo, esto no significa que deba diseñar la arquitectura autónomamente.

Recomendación para NativIQ

1. El humano define requisitos y límites.
2. El modelo local produce alternativas.
3. Se comparan costos, riesgos y trade-offs.
4. Un segundo modelo puede realizar una revisión adversarial.
5. El humano aprueba la decisión.

El modelo de frontera sería un **consultor opcional**, no el arquitecto responsable.

8. Repositorios

Esta categoría necesita dividirse.

Tamaño del repositorio	Open-source/local	Recomendación
Pequeño	Excelente	Local
Mediano	Excelente	Local
Grande, bien modularizado	Muy bueno	Local con indexación
Monorepo enorme	Bueno, pero requiere scaffolding	Local + subagentes
Varios repositorios relacionados	Más difícil	Comparar local y frontera
Millones de líneas	Ventaja de frontera	Uso selectivo de frontera

Un contexto declarado de 256K o un millón de tokens no significa que el modelo comprenda con igual calidad todos esos tokens. Para repositorios grandes importan más:

- Mapa del repositorio.
- Búsqueda semántica.

- LSP.
- Selección progresiva de archivos.
- Resúmenes jerárquicos.
- Subagentes.
- Compresión de contexto.
- Historial de Git.
- Ejecución de pruebas.

Qwen3-Coder-30B-A3B está expresamente optimizado para comprensión a escala de repositorio, tool calling y contextos de 256K, aunque su propia documentación recomienda bajar a 32K cuando aparecen problemas de memoria. (huggingface.co)

Recomendación

Local para repositorios pequeños y medianos, que probablemente representarán la mayoría de los proyectos iniciales de NativIQ.

No compraría acceso permanente a un modelo cerrado solo para estar preparado ante un hipotético monorepo de millones de líneas.

9. Trabajo autónomo

Resultado: la diferencia existe, pero se exagera

Debe diferenciarse la duración.

Duración de la tarea	Modelo local	Modelo de frontera
1-10 minutos	Excelente	Excelente
10-30 minutos	Muy bueno	Excelente
30-90 minutos	Bueno-muy bueno	Generalmente mejor
Varias horas	Más propenso a desviarse	Más consistente
Sin supervisión	No recomendable	Tampoco recomendable

Qwen3-Coder-Next fue entrenado para razonamiento de largo horizonte, utilización de herramientas y recuperación ante fallos, por lo que los modelos locales ya no están limitados a una única respuesta o edición. (huggingface.co)

Pero tampoco corresponde presentar a los modelos cerrados como trabajadores autónomos infalibles. La propia documentación de seguridad de GPT-5.6 recomienda supervisión durante trayectorias prolongadas de programación y reconoce riesgos como persistir excesivamente, ejecutar acciones destructivas o afirmar que una tarea tuvo éxito cuando no fue así. (deploymentsafety.openai.com)

Recomendación

Utilizar agentes locales con checkpoints:

```
Plan
↓
Aprobación
↓
Implementar etapa 1
↓
Pruebas + commit
↓
Implementar etapa 2
↓
Pruebas + revisión
```

El objetivo no debe ser “dejarlo trabajando toda la noche”, sino lograr **autonomía controlada, observable y reversible**.

Conclusión corregida

Dónde lo local ya debería ser la opción predeterminada

- Autocompletado.
- Funciones aisladas.
- Pruebas unitarias.
- Documentación.
- Refactorización delimitada.
- Repositorios pequeños y medianos.
- Automatizaciones cortas y verificables.

Dónde el modelo de frontera conserva una ventaja

- Depuración realmente compleja.
- Refactorizaciones que atraviesan todo el sistema.
- Arquitectura multidominio.
- Monorepos enormes.
- Tareas autónomas de larga duración.

Pero incluso en estas categorías, la elección correcta no es necesariamente sustituir el modelo local. Es:

Empezar localmente, medir el resultado y escalar únicamente cuando el beneficio del modelo cerrado sea comprobable.

Para NativIQ Systems, la jerarquía adecuada sería:

1. Humano responsable
2. Infraestructura open-source
3. Modelo open-weight local
4. Pruebas y validaciones automáticas
5. Segundo modelo local
6. Modelo cerrado opcional

El modelo comercial queda así en el último nivel: **útil, pero reemplazable y nunca estructuralmente indispensable**.

Mauro

Intervención 5 · 2026-07-12 21:07:29Z

Excelente. Ahora dime qué tipo de modelos me recomendarías con este nuevo paradigma.

ChatGPT - avance de trabajo

Voy a recomendarte **familias de modelos por función**, no un único “modelo estrella”. La idea será construir un portafolio local: modelos pequeños para rutina, modelos medianos para trabajo general y modelos especializados para código, razonamiento, visión y búsqueda documental.

ChatGPT

Con el paradigma **open-source/local como núcleo y modelos cerrados solo como opción excepcional**, no te recomendaría buscar “un modelo que haga todo”. Para NativIQ conviene construir una **jerarquía de modelos especializados**, donde cada tarea sea enviada al modelo más pequeño que pueda resolverla correctamente.

Modelo pequeño para clasificar, modelo mediano para trabajar, modelo grande para escalar y un modelo independiente para verificar.

Portafolio de modelos recomendado

Capa	Tipo de modelo	Tamaño orientativo	Función
1	Modelo utilitario	0,5B-4B	Clasificar, enrutar, extraer y validar
2	Generalista principal	20B-35B	Trabajo diario, documentos, código y agentes
3	Especialista en programación	30B-80B	Repositorios, debugging y refactorización
4	Razonador avanzado	80B-120B+	Arquitectura y problemas excepcionales
5	Multimodal	20B-35B	PDF, imágenes, interfaces y documentación visual
6	Embedding + reranker	0,6B-8B	RAG y búsqueda empresarial
7	Seguridad	0,1B-8B	Prompt injection, moderación y políticas
8	Verificador independiente	20B-35B	Revisar resultados del modelo principal

1. Modelos utilitarios pequeños

Estos modelos no deberían conversar con el usuario ni diseñar sistemas. Su función es ejecutar trabajos repetitivos, rápidos y baratos:

- Clasificación de correos y documentos.
- Detección de intención.
- Extracción de entidades.
- Conversión a JSON.
- Enrutamiento hacia otro agente.
- Detección de información sensible.
- Decisión de si una tarea necesita un modelo grande.

- Validación preliminar de una respuesta.

Usaría modelos de **0,6B a 4B**, posiblemente ajustados mediante LoRA para el vocabulario de cada cliente.

Para NativIQ serían muy útiles en automatizaciones de banca, energía, logística, seguros y bienes raíces, donde una gran parte del volumen corresponde a tareas estructuradas y repetitivas.

Recomendación

- Qwen de aproximadamente 0,6B-4B.
- Modelos BERT/DeBERTa para clasificación.
- Modelos específicos ajustados por cliente.

No usaría un modelo de 30B para decidir si un documento es una factura, un contrato o una orden de compra.

2. Modelo generalista principal

Esta sería la pieza central del sistema.

Debe poder:

- Conversar correctamente en español e inglés.
- Escribir y revisar documentos.
- Generar código.
- Utilizar herramientas.
- Interpretar imágenes.
- Consultar bases documentales.
- Ejecutar flujos con varios pasos.
- Producir salidas estructuradas.

Primera elección: Qwen3.6-27B

Es mi principal recomendación para NativIQ porque combina:

- Arquitectura densa de 27B.
- Capacidades multimodales.
- Mejoras específicas en programación agéntica y razonamiento sobre repositorios.
- Compatibilidad con Transformers, vLLM, SGLang, llama.cpp, Ollama y LM Studio.
- Licencia Apache 2.0. (huggingface.co)

La arquitectura densa es relevante: todos sus parámetros participan en la inferencia, lo que generalmente ofrece un comportamiento más uniforme que un MoE muy agresivo.

Para qué lo utilizaría

- Funciones aisladas.
- Pruebas unitarias.
- Documentación.
- Refactorizaciones delimitadas.
- Análisis de documentos.
- Agentes operativos.
- RAG empresarial.
- Interpretación de capturas y diagramas.

- Automatizaciones de dificultad media.

Una cuantización Q4 de 27B puede ser viable en 24 GB de VRAM, aunque no con su contexto máximo completo. En una GPU de 24 GB limitaría el contexto práctico inicialmente a aproximadamente 16K-32K y lo ampliaría solo después de medir consumo y velocidad. Esto es una recomendación de despliegue, no una capacidad oficial fija.

Alternativa eficiente: Qwen3.6-35B-A3B

Es un modelo MoE de 35B totales que activa aproximadamente 3B por token. Puede ofrecer mayor velocidad de inferencia, pero sigue necesitando mantener el conjunto completo de pesos en memoria. También es multimodal, tiene contexto declarado de 262K y licencia Apache 2.0. (huggingface.co)

Lo probaría frente al 27B, pero comenzaría con el **27B denso como modelo principal**, porque priorizaría estabilidad y previsibilidad sobre velocidad máxima.

3. Modelo especializado en programación

No usaría siempre el modelo generalista para programar. Mantendría un modelo específicamente entrenado para agentes de código.

Opción práctica: Qwen3-Coder-30B-A3B

Es adecuado para:

- Autocompletado.
- Creación de funciones.
- Pruebas.
- Refactorización.
- Navegación del repositorio.
- Tool calling.
- Ejecución mediante OpenCode, Qwen Code, Cline o Aider.

Forma parte de la familia oficial Qwen3-Coder, junto con modelos de mayor tamaño. (github.com)

Esta sería mi primera opción para una estación de desarrollo con aproximadamente 24 GB de VRAM, usando una cuantización razonable.

Opción avanzada: Qwen3-Coder-Next

Lo reservaría para:

- Repositorios grandes.
- Trabajo autónomo más prolongado.
- Depuración compleja.
- Refactorizaciones transversales.
- Agentes con múltiples herramientas.
- Investigación dentro de bases de código.

Tiene 80B totales y aproximadamente 3B activos por token, fue entrenado específicamente mediante interacción con entornos ejecutables y soporta 256K de contexto nativo, extensible según su documentación. (github.com)

No lo consideraría un modelo para una GPU convencional de 24 GB. Lo ubicaría en:

- Una máquina con 64 GB o más de memoria unificada.
- Varias GPU.
- Un servidor interno.

- Una instancia temporal controlada por NativIQ.

4. Modelo razonador avanzado

Esta capa no debe procesar todas las solicitudes. Solo debería activarse cuando:

- El modelo principal falla.
- Las pruebas no pasan.
- Existe una decisión arquitectónica importante.
- Hay muchas restricciones contradictorias.
- Se necesita una revisión crítica profunda.
- El riesgo económico o reputacional es alto.

Recomendación: gpt-oss-120b

Tiene 117B parámetros totales y aproximadamente 5,1B activos. OpenAI lo diseñó para razonamiento general, herramientas y tareas agénticas; puede ejecutarse en una GPU de 80 GB utilizando su formato y cuantización compatibles. Tiene licencia Apache 2.0. (github.com)

Lo utilizaría como:

- Segundo arquitecto.
- Revisor de seguridad.
- Analista de errores difíciles.
- Generador de alternativas.
- Juez independiente de resultados producidos por Qwen.

Alternativa más accesible: gpt-oss-20b

Tiene aproximadamente 21B totales y 3,6B activos. Su implementación nativa MXFP4 está orientada a funcionar dentro de unos 16 GB de memoria y ofrece niveles configurables de razonamiento, function calling y salidas estructuradas. (huggingface.co)

No lo pondría como único modelo principal, pero sí como:

- Razonador rápido.
- Verificador independiente.
- Alternativa a Qwen.
- Modelo de respaldo.
- Segundo voto en decisiones importantes.

Usar modelos de **familias diferentes** para generar y verificar reduce el riesgo de que ambos reproduzcan exactamente los mismos errores.

5. Modelos extremadamente grandes

DeepSeek-V4 demuestra que los modelos open-weight pueden acercarse considerablemente a los modelos cerrados, pero no es una opción local inicial razonable.

Sus versiones publicadas incluyen:

- DeepSeek-V4-Flash: 284B totales y 13B activos.
- DeepSeek-V4-Pro: 1,6 billones de parámetros totales y 49B activos.
- Contexto declarado de un millón de tokens. (huggingface.co)

Estos modelos requieren infraestructura seria aunque sean MoE. Para NativIQ los consideraría más adelante como:

- Infraestructura centralizada para varios clientes.
- Servicio interno de alto nivel.
- Modelo hospedado en un proveedor que permita instancias dedicadas.
- Laboratorio comparativo.

No compraría infraestructura para DeepSeek-V4 durante la etapa inicial de NativIQ.

6. Modelos multimodales

NativIQ necesitará interpretar más que texto:

- PDFs escaneados.
- Diagramas.
- Capturas de sistemas.
- Tableros.
- Formularios.
- Fotografías de equipos.
- Interfaces de aplicaciones.
- Documentos con tablas y gráficos.

Qwen3.6-27B ya incorpora un encoder visual y puede procesar texto e imágenes, por lo que podría funcionar como generalista multimodal principal. (huggingface.co)

No obstante, separaría dos operaciones:

1. **Comprender una imagen:** modelo multimodal generativo.
2. **Encontrar imágenes o documentos similares:** embedding multimodal.

Esa separación mejora rendimiento y reduce costos.

7. Modelos de embeddings y reranking

Este componente es tan importante como el LLM principal.

Un RAG serio debería tener:

```
Consulta
↓
Embedding
↓
Recuperación inicial
↓
Reranker
↓
Documentos relevantes
↓
LLM generador
```

Para documentos de texto

Recomendaría:

- **Qwen3-Embedding-0.6B** para instalaciones ligeras.
- **Qwen3-Embedding-4B** para mayor precisión.

- **Qwen3-Reranker-0.6B o 4B** para ordenar los resultados.

La familia admite más de cien idiomas, hasta 32K de contexto y tamaños de 0,6B, 4B y 8B para embeddings y rerankers. (huggingface.co)

Para NativIQ comenzaría con:

Qwen3-Embedding-0.6B + Qwen3-Reranker-0.6B

Solo subiría a 4B si un benchmark documental demuestra una mejora importante.

Para PDFs, capturas e imágenes

Utilizaría:

- Qwen3-VL-Embedding-2B.
- Qwen3-VL-Reranker-2B.

La familia puede indexar conjuntamente texto, imágenes, capturas y video, con soporte multilingüe y 32K de secuencia. (huggingface.co)

Esto podría ser especialmente valioso para buscar:

- Planos.
- Diagramas industriales.
- Pantallas de sistemas.
- Facturas escaneadas.
- Fotografías operativas.
- Documentos técnicos con gráficos.

8. Modelos de seguridad y gobierno

Un sistema AI-native empresarial no debería confiar solamente en que el modelo principal “se comporte bien”.

Necesita modelos separados para:

- Detectar prompt injection.
- Clasificar entradas y salidas.
- Aplicar políticas.
- Detectar contenido sensible.
- Identificar solicitudes fuera de alcance.
- Revisar información antes de enviarla a herramientas.

Recomendación

Qwen3Guard

La familia Qwen3Guard ofrece versiones de 0,6B, 4B y 8B, con variantes para clasificación generativa y supervisión durante la generación. Fue entrenada específicamente como modelo de moderación y tiene licencia Apache 2.0. (huggingface.co)

Para comenzar:

- Qwen3Guard-Gen-0.6B para clasificación económica.
- Qwen3Guard-Gen-4B cuando se necesite interpretar políticas más complejas.

Llama Prompt Guard 2

Es un clasificador especializado y muy pequeño, disponible en variantes de 22M y 86M, destinado a detectar ataques mediante prompts y contenido inyectado. (huggingface.co)

Usaría ambas capas:

```
Entrada del usuario o documento
↓
Prompt Guard
↓
Modelo de política / Qwen3Guard
↓
Agente principal
```

Estos modelos reducen riesgo, pero no sustituyen sandbox, permisos, controles de red ni revisión humana.

9. Modelo verificador

Esta es una capa frecuentemente ignorada.

El modelo que genera una solución **no debería ser siempre quien la aprueba**.

Ejemplo:

```
Qwen3.6-27B genera
↓
Pruebas automáticas
↓
gpt-oss-20b revisa
↓
Reglas determinísticas
↓
Humano aprueba si es crítico
```

También puede invertirse:

```
gpt-oss genera
↓
Qwen revisa
```

Para código, el verificador debe comprobar:

- Diff.
- Tests.
- Errores de seguridad.
- Dependencias nuevas.
- Compatibilidad.
- Cumplimiento de requisitos.

Para documentos empresariales:

- Evidencia de cada afirmación.
- Datos sensibles.
- Contradicciones.
- Campos faltantes.
- Cumplimiento de políticas.

Mi selección concreta para NativIQ

Stack inicial

Función	Modelo recomendado
Modelo principal	Qwen3.6-27B
Programación	Qwen3-Coder-30B-A3B
Razonamiento alternativo	gpt-oss-20b
Embeddings	Qwen3-Embedding-0.6B
Reranking	Qwen3-Reranker-0.6B
Seguridad	Qwen3Guard-0.6B + Prompt Guard 2
Visión y documentos	Qwen3.6-27B
Recuperación multimodal	Qwen3-VL-Embedding-2B + Reranker-2B

Expansión posterior

Necesidad	Modelo
Repositorios grandes	Qwen3-Coder-Next
Razonamiento interno avanzado	gpt-oss-120b
Infraestructura de máxima escala	DeepSeek-V4-Flash o Pro
Revisión crítica	Modelo de familia distinta al generador

Qué no recomendaría

No construiría NativIQ alrededor de:

- Un único modelo gigantesco.
- Un modelo de 70B excesivamente cuantizado para que entre a la fuerza.
- Un modelo generativo usado también como embedding.
- Un modelo principal que revise sus propias respuestas.
- Un agente sin modelo de seguridad.
- Contextos gigantes usados como sustituto de una buena recuperación documental.
- Fine-tuning antes de tener datos y evaluaciones suficientes.
- Cambiar de modelo cada vez que aparece uno nuevo en un benchmark.

Decisión final

Para iniciar el ecosistema soberano de NativIQ elegiría tres modelos fundamentales:

1. **Qwen3.6-27B como trabajador generalista.**
2. **Qwen3-Coder-30B-A3B como ingeniero de software.**
3. **gpt-oss-20b como razonador y verificador independiente.**

A su alrededor colocaría modelos pequeños de embeddings, reranking, clasificación y seguridad.

La ventaja competitiva de NativIQ no debería ser poseer “el modelo más inteligente”, sino diseñar un sistema donde:

Cada tarea utilice el modelo mínimo necesario, los resultados sean verificables y cualquier modelo pueda ser reemplazado sin reconstruir la solución completa.

Mauro

Intervención 6 · 2026-07-12 21:15:39Z

Excelente ChatGPT. Ahora dime qué equipos debería comprar para poder tener todo el ecosistema completo funcional

ChatGPT - avance de trabajo

Voy a diseñarlo como una infraestructura completa, no como una sola PC: estación de desarrollo, servidor de inferencia, almacenamiento, red, respaldo eléctrico y seguridad. También separaré una compra inicial razonable de una arquitectura empresarial escalable.

ChatGPT - avance de trabajo

Ya aparece una distinción decisiva: para que el ecosistema sea **funcional**, una RTX 5090 de 32 GB alcanza si cargás los modelos grandes de a uno. Para que sea **empresarial, concurrente y capaz de alojar modelos de 80B-120B**, necesitás una segunda máquina con 96 GB de VRAM o mucha memoria unificada.

ChatGPT

Recomendación directa

Para NativIQ Systems **no compraría inmediatamente una sola workstation gigantesca**. Construiría el ecosistema con dos nodos especializados:

- 1. **Una PC Linux con RTX 5090 de 32 GB** para desarrollo, código, agentes, visión y modelos de 20B-30B.
- 2. **Una Mac Studio M3 Ultra con 256 GB** para cargar localmente modelos de 80B-120B y experimentar con modelos todavía mayores.

A esto le agregaría NAS, red de 10 GbE y protección eléctrica. Esa combinación sería el mejor equilibrio entre **compatibilidad, capacidad, privacidad y costo** para un emprendimiento que todavía está comenzando.

La alternativa empresarial futura sería reemplazar o complementar la Mac con un servidor Linux equipado con **RTX PRO 6000 Blackwell de 96 GB**.

Tres niveles posibles

Nivel	Configuración	Qué permite
Inicial funcional	Una PC RTX 5090	Todo el ecosistema inicial, cargando un modelo principal por vez
Recomendado para NativIQ	RTX 5090 + Mac Studio 256 GB	Ecosistema completo, incluidos modelos de 80B-120B

Nivel	Configuración	Qué permite
Empresarial	RTX 5090 + servidor RTX PRO 6000	Producción multiusuario, CUDA, vLLM y modelos grandes

ASUS TUF Gaming GeForce RTX 5090 32 GB

Description: None Price: \$ 5.957.864,98 URL: Number of Reviews: 207 Rating: 4.4 Featured Tag: GPU inicial recomendada Merchants: Buy Direct & Save y otros

PNY GeForce RTX 5090 32 GB

Description: None Price: \$ 6.255.832,72 URL: Number of Reviews: 24 Rating: 4.3 Featured Tag: Alternativa RTX 5090 Merchants: Buy Direct & Save

Mac Studio M3 Ultra con 256 GB

Description: None Price: \$ 8.377.600,00 URL: Number of Reviews: 1 Rating: 5.0 Featured Tag: Modelos grandes locales Merchants: Inovamusicnet

NVIDIA RTX PRO 6000 Blackwell 96 GB

Description: None Price: \$ 46.792.330,00 URL: Number of Reviews: None Rating: None Featured Tag: Servidor empresarial 96 GB Merchants: Biosistemas y otros

Synology DS1525+

Description: None Price: \$ 1.484.859,98 URL: Number of Reviews: 32 Rating: 4.6 Featured Tag: NAS para oficina Merchants: IT Planet y otros

Synology RackStation RS422+

Description: None Price: \$ 1.585.754,52 URL: Number of Reviews: 17 Rating: 4.8 Featured Tag: NAS para rack Merchants: Microless.com y otros

APC Smart-UPS 2200 VA

Description: None Price: \$ 1.414.511,63 URL: Number of Reviews: 3 Rating: 5.0 Featured Tag: Protección eléctrica Merchants: IT Planet y otros

Mac Studio M3 Ultra con 96 GB

Description: None Price: \$ 8.341.606,90 URL: Number of Reviews: None Rating: None Featured Tag: Entrada a Apple Silicon Merchants: Compumacypc Alejandro Kacero

Los precios mostrados son referencias puntuales de comercios y pueden variar mucho en Argentina por importación, disponibilidad, impuestos y tipo de cambio.

1. Estación principal de desarrollo

Esta sería tu computadora cotidiana para construir y probar los sistemas de NativIQ.

Configuración recomendada

Componente	Recomendación
GPU	NVIDIA RTX 5090, 32 GB
CPU	AMD Ryzen 9 9950X o 9950X3D
RAM	128 GB DDR5, preferentemente 2 × 64 GB
Disco del sistema	NVMe de 2 TB
Disco para modelos	NVMe de 4 TB

Componente	Recomendación
Placa madre	X870E con al menos tres ranuras M.2
Fuente	1200 W o 1600 W, ATX 3.1, 80 Plus Platinum
Red	10 GbE
Gabinete	Full tower de alto flujo de aire
Sistema operativo	Linux Ubuntu LTS
Refrigeración	AIO de 360 mm o disipador de alta gama

La RTX 5090 tiene 32 GB de GDDR7, consume hasta 575 W y NVIDIA establece una fuente mínima de sistema de 1000 W. Para inferencia prolongada, discos, CPU potente y márgenes seguros, compraría una fuente de **1200 W como mínimo**, preferentemente 1600 W si pensás expandirla. ([nvidia.com](https://www.nvidia.com))

El Ryzen 9 9950X3D ofrece 16 núcleos y 32 hilos. No necesitás un Threadripper para una sola GPU: la mayor parte de la inferencia ocurre en la RTX 5090, mientras el procesador se ocupa de compilación, bases de datos, contenedores, extracción documental y servicios auxiliares. ([amd.com](https://www.amd.com))

Qué ejecutaría esta máquina

- Qwen3.6-27B cuantizado.
- Qwen3-Coder-30B-A3B.
- gpt-oss-20b.
- Modelos de embeddings.
- Rerankers.
- Qwen Guard y Prompt Guard.
- Modelos multimodales.
- OpenCode y Qwen Code.
- LibreChat, LiteLLM, Ollama y vLLM.
- Bases vectoriales.
- Contenedores de pruebas.
- BotTradeNC y los proyectos de NativIQ.

Qwen3.6-27B es compatible oficialmente con Transformers, vLLM, SGLang y otros motores, y ya existen cuantizaciones que funcionan dentro de 24 GB. Los 32 GB de la RTX 5090 proporcionarían espacio adicional para contexto, KV cache y herramientas auxiliares. (huggingface.co)

Restricción

No podrías mantener simultáneamente varios modelos principales de 20B-30B dentro de la GPU. El orquestador tendría que:

- Cargar uno por vez.
- Mantener los modelos pequeños en RAM o CPU.
- Descargar el modelo principal cuando cambie la tarea.
- O utilizar la segunda máquina para otro modelo.

Eso no impide que el ecosistema sea funcional; limita la concurrencia.

2. Nodo de modelos grandes

Mi elección para comenzar: Mac Studio M3 Ultra

Configuración:

Componente	Recomendación
Chip	M3 Ultra, GPU de 60 u 80 núcleos
Memoria unificada	256 GB
SSD interno	2 TB
Red	10 GbE integrada
Software	MLX-LM, llama.cpp y herramientas compatibles

Apple ofrece el M3 Ultra con 819 GB/s de ancho de banda de memoria y configuraciones que llegan oficialmente hasta 512 GB. La variante de 256 GB es el punto razonable para NativIQ: suficiente para pesos grandes, contexto y varios servicios sin pagar el máximo absoluto. (support.apple.com)

Qué te permitiría ejecutar

- Qwen3-Coder-Next de 80B.
- gpt-oss-120b.
- Modelos MoE grandes.
- Varias cuantizaciones de un mismo modelo.
- Contextos largos.
- Experimentación con modelos de cientos de miles de millones de parámetros.
- Fine-tuning ligero mediante LoRA.
- Inferencia local sin el consumo y ruido de varias GPU NVIDIA.

Qwen3-Coder-Next tiene 80B parámetros totales y 3B activos por token. gpt-oss-120b fue diseñado para caber en una GPU de 80 GB; por lo tanto, 256 GB de memoria unificada proporciona capacidad de sobra para pesos cuantizados, contexto y otros procesos, aunque la velocidad dependerá de la implementación utilizada. (huggingface.co)

Por qué no usaría solamente la Mac

Apple Silicon es excelente para **capacidad de memoria**, eficiencia y experimentación, pero CUDA continúa teniendo el ecosistema más maduro para servidores de inferencia.

MLX-LM permite ejecutar y ajustar modelos en Apple Silicon. vLLM incluye soporte para Apple mediante vLLM-Metal, pero parte de ese soporte sigue dependiendo de plugins y componentes comunitarios, mientras que el despliegue NVIDIA mediante Docker y CUDA está más estandarizado. (github.com)

Por eso recomiendo:

- **RTX 5090:** desarrollo, velocidad, CUDA y compatibilidad.
- **Mac Studio:** capacidad de memoria y modelos grandes.

Son complementarias.

3. Alternativa empresarial: servidor NVIDIA de 96 GB

Cuando NativIQ tenga clientes activos y necesite atender múltiples usuarios, instalaría un servidor Linux con esta configuración:

Componente	Recomendación
GPU	RTX PRO 6000 Blackwell, 96 GB ECC
CPU	Threadripper PRO 9000 WX, 24-32 núcleos
Plataforma	WRX90
RAM	256 GB ECC RDIMM

Componente	Recomendación
Disco del sistema	NVMe empresarial de 2 TB
Disco de modelos	NVMe empresarial de 7,68 u 8 TB
Fuente	1600 W Platinum
Red	10 o 25 GbE
Administración	Placa con IPMI/BMC
Formato	Torre profesional o servidor 4U
Sistema	Linux + Docker + vLLM

La RTX PRO 6000 Blackwell dispone de **96 GB de GDDR7 con ECC**, 1.792 GB/s de ancho de banda y un consumo máximo de 600 W. Puede alojar en una única GPU modelos que exigen aproximadamente 80 GB, evitando la complejidad de dividirlos entre varias tarjetas. ([nvidia.com](#))

La plataforma Threadripper PRO con WRX90 ofrece ocho canales de memoria, hasta 2 TB de RAM y hasta 128 líneas PCIe 5.0. Esto permitiría añadir una segunda GPU profesional, más almacenamiento NVMe o interfaces de red de alta velocidad posteriormente. ([amd.com](#))

Ventajas respecto de la Mac

- CUDA.
- vLLM y SGLang con soporte maduro.
- ECC en la VRAM.
- Mejor integración con contenedores.
- Operación Linux 24 × 7.
- Más apropiada para API multiusuario.
- Expansión mediante PCIe.
- Posibilidad de una segunda GPU.

Desventaja principal

El costo es extremadamente alto. Una publicación argentina relevada muestra que **solo la RTX PRO 6000 puede costar varias veces más que una Mac Studio M3 Ultra completa**. Por eso no la compraría antes de que exista una necesidad comercial demostrada.

4. Servidor de almacenamiento

Los modelos, datasets, documentos privados, bases vectoriales, snapshots y copias de los repositorios no deberían quedar solamente en las estaciones de trabajo.

NAS recomendado

Synology DS1525+, configurado con:

- Cinco discos empresariales de 12 TB.
- SHR-2 o RAID 6.
- Aproximadamente 30–36 TB útiles, según formato y configuración.
- Adaptador de 10 GbE.
- 16 o 32 GB de RAM.
- Dos NVMe para cache o almacenamiento rápido.
- Snapshots.
- Cifrado.
- Copias programadas externas.

El DS1525+ tiene cinco bahías, dos puertos de 2,5 GbE, opción de 10 GbE y dos ranuras M.2. Puede ampliarse posteriormente mediante unidades adicionales. (synology.com)

Importante

El NAS no sustituye una copia de seguridad. Utilizaría la regla:



La copia externa puede ser:

- Otro NAS en una ubicación diferente.
- Almacenamiento cloud cifrado.
- Un disco empresarial grande rotado periódicamente.

5. Red

Compraría:

- Switch gestionable de ocho puertos 10 GbE.
- Tarjeta 10 GbE para la PC.
- La Mac Studio ya incorpora 10 GbE.
- Adaptador 10 GbE para el NAS.
- Cableado Cat 6A.
- Firewall independiente con OPNsense.
- Access point Wi-Fi separado.
- VLAN específicas.

Separaría las redes:

VLAN	Uso
Administración	Acceso a servidores
Inferencia	APIs de modelos
Almacenamiento	NAS y backups
Desarrollo	Computadoras personales
Agentes	Contenedores que ejecutan herramientas
Invitados/IoT	Equipos no confiables

Esto impide que un agente comprometido tenga acceso automático a todos los documentos y dispositivos.

6. Protección eléctrica

Para la PC RTX 5090 compraría un UPS de onda sinusoidal pura de al menos:

- **2200 VA / aproximadamente 1980 W.**

Para el nodo de modelos grandes, el NAS y la red utilizaría un segundo UPS independiente.

El APC Smart-UPS de 2200 VA entrega hasta 1980 W, utiliza onda sinusoidal, incorpora regulación automática de tensión y permite apagado controlado. (apc.com)

Para la arquitectura empresarial con RTX PRO 6000 consideraría:

- UPS online de 3000 VA.
- Circuito eléctrico dedicado.
- Correcta puesta a tierra.
- Protección contra sobretensiones.
- Revisión por electricista matriculado.
- Aire acondicionado dedicado.

Una RTX 5090 puede consumir hasta 575 W y una RTX PRO 6000 hasta 600 W, sin contar CPU, discos y pérdidas de la fuente. Ambas máquinas funcionando al máximo pueden convertir más de un kilovatio eléctrico en calor dentro de la habitación. (nvidia.com)

7. Papel de tu Lenovo actual

La Lenovo que estás preparando como servidor Linux no debería intentar ejecutar estos modelos grandes. Puede cumplir una función muy útil como **nodo de control**:

- Git privado y mirrors.
- Monitoreo.
- Grafana y Prometheus.
- Reverse proxy.
- DNS interno.
- Orquestación de tareas.
- Base de datos de métricas.
- Servidor de backups.
- Watchdog.
- Gestión de certificados.
- Control de encendido de otros nodos.

De esta forma no queda obsoleta: separás el **plano de control** del **plano de inferencia**.

Orden de compra recomendado

Primera compra

PC Linux con RTX 5090, 128 GB y 6 TB de NVMe.

Con esto podés desarrollar y validar:

- LibreChat.
- LiteLLM.
- Qwen Code.
- OpenCode.
- Qwen3.6-27B.
- Qwen3-Coder-30B.
- gpt-oss-20b.
- Embeddings, reranking y seguridad.
- RAG empresarial.
- Agentes locales.

Segunda compra

UPS de 2200 VA y red de 10 GbE.

La protección eléctrica debería comprarse junto con la workstation, no después de un incidente.

Tercera compra

NAS DS1525+ con cinco discos.

Cuando ya existan modelos, datos de clientes, pruebas y repositorios importantes.

Cuarta compra

Mac Studio M3 Ultra con 256 GB.

Cuando necesites experimentar regularmente con los modelos de 80B-120B.

Quinta compra

Servidor RTX PRO 6000.

Solo cuando NativIQ necesite:

- Varios usuarios concurrentes.
- Producción 24 x 7.
- SLA.
- CUDA empresarial.
- Modelos grandes como servicio.
- Clientes que paguen por despliegue local o dedicado.

Lo que no compraría

No compraría ahora:

- Dos RTX 5090 para una única PC.
- Una RTX PRO 6000 sin clientes que justifiquen la inversión.
- Una Mac Studio de solo 96 GB si el objetivo son modelos de 120B.
- Una PC con 32 o 64 GB de RAM.
- Una RTX 5080 de 16 GB como servidor principal.
- Un rack ruidoso para colocarlo en una habitación de trabajo.
- Discos rígidos de escritorio para el NAS.
- Un UPS económico de onda aproximada.
- Una fuente de 1000 W trabajando permanentemente al límite.
- Contextos gigantes usados sin necesidad.

Compra definitiva que recomiendo

Ecosistema completo para NativIQ

Nodo 1 — Desarrollo y CUDA

- RTX 5090 de 32 GB.
- Ryzen 9 9950X/9950X3D.
- 128 GB de RAM.
- NVMe de 2 TB + 4 TB.
- Fuente Platinum de 1200-1600 W.

- Linux.
- 10 GbE.

Nodo 2 — Modelos grandes

- Mac Studio M3 Ultra.
- **256 GB de memoria unificada.**
- SSD de 2 TB.
- 10 GbE.

Infraestructura

- Synology DS1525+.
- Cinco HDD empresariales de 12 TB.
- Switch gestionable de 10 GbE.
- Firewall OPNsense.
- Dos UPS de 2200 VA.
- Backup externo cifrado.
- Tu Lenovo como nodo de control.

Esta arquitectura te daría **todo el ecosistema funcional**, incluyendo los modelos pequeños, el generalista de 27B, el modelo de código de 30B, modelos de 80B y el razonador de 120B, sin hacer ahora la inversión desproporcionada de una GPU profesional de 96 GB. El servidor RTX PRO 6000 quedaría como evolución comercial, no como requisito para empezar.

Mauro

Intervención 7 · 2026-07-12 22:06:25Z

Excelente. Ahora hablemos de presupuesto. ¿Cuánto dinero me costaría armar todo esto?

ChatGPT - avance de trabajo

Voy a presupuestarlo en tres escenarios: **mínimo viable, ecosistema completo recomendado y versión empresarial**. Separaré hardware principal, red/almacenamiento, protección eléctrica e importación para que veas dónde se concentra realmente la inversión.

ChatGPT

Estimación directa

A valores del **12 de julio de 2026**, armar el ecosistema que propuse tendría aproximadamente estos costos:

Escenario	Pesos argentinos	Dólares aproximados
Núcleo funcional inicial	ARS 15-20 millones	USD 10.100-13.400
Ecosistema completo recomendado	ARS 34-50 millones	USD 22.800-33.600
Infraestructura empresarial NVIDIA	ARS 50-95 millones	USD 33.600-63.800

Para convertir utilicé un tipo de cambio medio de referencia cercano a **ARS 1.488 por USD**. No necesariamente será el mismo aplicado por una tarjeta, importador o comercio argentino. ([wise.com](#))

Mi recomendación económica realista para NativIQ sería:

Comenzar invirtiendo entre USD 10.000 y USD 13.500, validar comercialmente el ecosistema y expandirlo posteriormente hasta unos USD 23.000-34.000.

1. Workstation Linux con RTX 5090

Presupuesto estimado: ARS 11,5-14 millones

Componente	Estimación
RTX 5090 de 32 GB	ARS 6,5-7,5 millones
Ryzen 9 9950X/9950X3D	ARS 1,1-1,3 millones
Placa madre X870E	ARS 700.000-1.100.000
128 GB DDR5	ARS 600.000-900.000
NVMe de 2 TB + 4 TB	ARS 700.000-1.200.000
Fuente 1200-1600 W	ARS 500.000-850.000
Gabinete, refrigeración y ventiladores	ARS 500.000-850.000
Adaptador 10 GbE y accesorios	ARS 200.000-350.000
Total probable	ARS 11,5-14 millones

La RTX 5090 tiene un precio oficial inicial de USD 1.999, pero el precio de mercado es considerablemente superior. Una ASUS TUF aparece actualmente alrededor de **ARS 6,58 millones**, mientras que otras publicaciones argentinas superan los ARS 7 millones. ([nvidia.com](#))

El Ryzen 9 9950X3D aparece actualmente alrededor de **ARS 1,28 millones** en un comercio argentino. ([compragamer.com](#))

Esta máquina por sí sola permitiría ejecutar:

- Qwen3.6-27B.
- Qwen3-Coder-30B.
- gpt-oss-20B.
- Embeddings y rerankers.
- Modelos de seguridad.
- OpenCode y Qwen Code.
- RAG empresarial.
- Agentes locales.
- Bases vectoriales y contenedores.

Es la compra indispensable. El resto puede incorporarse progresivamente.

2. Mac Studio M3 Ultra con 256 GB

Presupuesto seguro: ARS 11,6-14,5 millones

Aquí debo corregir la estimación demasiado optimista de mi respuesta anterior.

Existe una publicación argentina de una Mac Studio M3 Ultra, 256 GB y 2 TB por aproximadamente **ARS 8,38 millones**. Sin embargo, no utilizaría ese valor como presupuesto seguro sin comprobar disponibilidad, precio final, garantía y condiciones de importación. ([inovamusicnet.com](https://www.inovamusicnet.com))

El precio oficial actual permite reconstruir aproximadamente el costo:

- Mac Studio M3 Ultra con 96 GB y 1 TB: **USD 5.299**.
- Ampliación de 96 a 256 GB: aproximadamente **USD 2.000**.
- Ampliación de 1 a 2 TB: aproximadamente **USD 500**.

Eso arroja aproximadamente:

USD 7.799 antes de impuestos, transporte e importación.

(apple.com)

Por eso reservaría **USD 7.800-9.700**, equivalentes aproximadamente a **ARS 11,6-14,5 millones**.

Esta máquina sería el nodo para:

- Qwen3-Coder-Next de 80B.
- gpt-oss-120B.
- Modelos MoE grandes.
- Modelos de visión grandes.
- Contextos extensos.
- Pruebas con varias cuantizaciones.
- Inferencia local de gran capacidad.

No la compraría en la primera etapa. Primero demostraría que los modelos grandes aportan una mejora comercial real.

3. NAS completo

Presupuesto: ARS 4,7-8,2 millones

Componente	Estimación
Synology DS1525+	ARS 840.000-1.850.000
Cinco discos NAS de 12 TB	ARS 3.400.000-5.200.000
Memoria, NVMe y red 10 GbE	ARS 500.000-1.200.000
Total	ARS 4,7-8,2 millones

El DS1525+ aparece entre aproximadamente **ARS 839.000 y ARS 1,83 millones**, según el vendedor. ([mercadolibre.com.ar](https://www.mercadolibre.com.ar))

Los Seagate IronWolf Pro de 12 TB aparecen aproximadamente entre **ARS 676.000 y ARS 1,04 millones por unidad**. Cinco discos representarían entre ARS 3,4 y 5,2 millones. ([mercadolibre.com.ar](https://www.mercadolibre.com.ar))

Con RAID 6 o SHR-2, los cinco discos de 12 TB no entregarían 60 TB útiles. Dos discos se reservarían para tolerancia a fallos, quedando aproximadamente 36 TB brutos antes del formato y otros ajustes.

4. Red y firewall

Presupuesto: ARS 1,3-2,5 millones

Elemento	Estimación
Switch gestionable 10 GbE	ARS 500.000-1.000.000
Firewall dedicado OPNsense	ARS 500.000-900.000
Placas y adaptadores adicionales	ARS 150.000-300.000
Cableado Cat 6A/SFP+	ARS 150.000-300.000
Total	ARS 1,3-2,5 millones

Ya existen switches multigigabit o 10 GbE cercanos a **ARS 500.000-620.000**, aunque un equipo empresarial gestionable puede costar más. (listado.mercadolibre.com.ar)

No es obligatorio comenzar con ocho puertos 10 GbE puros. Una opción más económica sería:

- Dos puertos 10 GbE para workstation y NAS.
- Puertos de 2,5 GbE para el resto.
- VLAN.
- SFP+ para futuras expansiones.

5. Protección eléctrica

Presupuesto: ARS 2,4-7,4 millones

Esta partida varía enormemente según la marca.

Configuración razonable

- UPS online de onda sinusoidal pura de 2000 VA para la workstation.
- Segundo UPS de 1500-2000 VA para Mac, NAS y red.

Existen UPS online de 2000 VA y 1800 W cercanas a **ARS 1,2 millones**, por lo que dos unidades equivaldrían aproximadamente a ARS 2,4-3 millones. (listado.mercadolibre.com.ar)

Configuración APC profesional

Un APC Smart-UPS SMT de 2200 VA y 1980 W cuesta aproximadamente **ARS 3,68 millones**. Dos unidades representarían unos **ARS 7,4 millones**. (scisrl.com.ar)

Los APC SRT online de doble conversión pueden superar los **ARS 9 millones por unidad**, una inversión difícil de justificar durante la etapa inicial. (listado.mercadolibre.com.ar)

Por eso no compraría dos APC SRT inmediatamente. Elegiría dos UPS online confiables de una gama intermedia y reservaría los APC empresariales para producción con clientes.

6. Backup externo, instalación y contingencias

Presupuesto: ARS 1,5-3 millones

Incluiría:

- Dos discos externos o removibles.

- Copia fuera del sitio.
- Protección contra sobretensión.
- Cableado eléctrico.
- Posible circuito dedicado.
- Instalación y armado.
- Repuestos básicos.
- Ventilación o mejora térmica.
- Un margen de contingencia.

No incluye aire acondicionado dedicado. Si la workstation y la Mac trabajan varias horas bajo carga, la habitación necesitará ventilación adecuada.

Presupuesto consolidado

Ecosistema completo recomendado

Subsistema	Mínimo	Máximo prudente
Workstation RTX 5090	ARS 11,5 M	ARS 14 M
Mac Studio 256 GB	ARS 11,6 M	ARS 14,5 M
NAS completo	ARS 4,7 M	ARS 8,2 M
Red y firewall	ARS 1,3 M	ARS 2,5 M
Dos UPS	ARS 2,4 M	ARS 7,4 M
Backup e instalación	ARS 1,5 M	ARS 3 M
Total calculado	ARS 33 M	ARS 49,6 M

Lo redondearía a:

ARS 34-50 millones, aproximadamente USD 22.800-33.600.

La amplitud del rango se explica principalmente por:

- Precio real de la RTX 5090.
- Precio efectivo de la Mac importada.
- Marca de los UPS.
- Precio de los discos.
- Disponibilidad local.
- Garantía e impuestos.
- Forma de pago.

Alternativa empresarial con RTX PRO 6000

La RTX PRO 6000 Blackwell tiene 96 GB de VRAM. Hay una enorme dispersión de precios:

- Algunas publicaciones importadas aparecen alrededor de **ARS 17,1 millones**.
- Una publicación local llega aproximadamente a **ARS 46,8 millones solamente por la GPU**.

Una workstation o servidor completo con:

- RTX PRO 6000.

- Threadripper Pro.
- 256 GB ECC.
- WRX90.
- Almacenamiento empresarial.
- Fuente y gabinete de servidor.

podría costar entre **ARS 28 y 60 millones** por sí solo.

Sumando workstation de desarrollo, NAS, red y UPS, el ecosistema empresarial completo quedaría aproximadamente entre:

ARS 50-95 millones, equivalentes a USD 33.600-63.800.

No considero racional hacer esta inversión antes de tener clientes que requieran concurrencia, SLA o despliegue NVIDIA 24 x 7.

Plan financiero que recomiendo

Etapas 1 — Núcleo funcional

Inversión: ARS 15-20 millones / USD 10.100-13.400

Comprar:

- Workstation RTX 5090.
- 128 GB de RAM.
- 6 TB de NVMe.
- Un buen UPS.
- Red básica multigigabit.
- Dos discos externos para backup.
- Utilizar tu Lenovo como servidor de control.

Con esto tendrías aproximadamente el **80-90 % del ecosistema inicial** y podrías comenzar a desarrollar productos.

Etapas 2 — Datos y continuidad

Inversión adicional: ARS 6-10 millones

Comprar:

- NAS completo.
- Segundo UPS.
- Firewall.
- Switch 10 GbE.
- Backup externo cifrado.

Acumulado aproximado:

ARS 21-30 millones.

Etapas 3 — Modelos grandes

Inversión adicional: ARS 11,6-14,5 millones

Comprar la Mac Studio de 256 GB solamente después de comprobar que:

- Qwen3-Coder-Next mejora tareas reales.
- gpt-oss-120B aporta valor.
- Existen clientes o proyectos que necesitan esa capacidad.
- Alquilar GPU resulta más costoso que comprar.

Acumulado:

ARS 34-50 millones.

Conclusión

No necesitás desembolsar USD 30.000 para comenzar NativIQ.

La decisión financieramente responsable sería:

Invertir primero unos USD 10.000-13.500 en la workstation y la infraestructura esencial. Posponer la Mac Studio y el NAS completo hasta que el ecosistema ya produzca resultados, clientes o ahorro medible.

El presupuesto de **USD 23.000-34.000** corresponde al laboratorio completo y soberano, no al costo mínimo para empezar a trabajar seriamente.

Mauro

Intervención 8 · 2026-07-12 22:31:40Z

Eso es mucho dinero ChatGPT. Incluso la inversión inicial para comenzar con NativIQ en el corto-mediano plazo se me hace demasiado elevada. Prefiero probar todas herramientas open-source en la nube que no dependan de un organismo centralizado de ningún tipo. Esto con el objetivo de no tener las complicaciones que mencionaba el video: que una empresa aumente o discontinue un servicio o bien que un gobierno autoritario lo bloquee.

ChatGPT - avance de trabajo

Tu enfoque es más viable financieramente, pero hay una precisión importante: **“nube sin ningún organismo centralizado” no existe de forma absoluta**. Lo que sí podemos construir es una arquitectura **portable, multi-red y sin dependencia crítica de un proveedor único**. Voy a separar qué puede descentralizarse realmente y qué conviene mantener replicado.

ChatGPT - avance de trabajo

La mejor corrección al plan anterior sería esta: no comprar hardware antes de validar. Primero montaríamos un **laboratorio soberano por software**, capaz de desplegar el mismo contenedor en varias redes descentralizadas y abandonarlas sin perder modelos, datos ni configuración.

ChatGPT

Sí. **Para la etapa actual de NativIQ, tu decisión es financieramente más sensata que comprar una workstation de USD 10.000 o más.** Primero conviene demostrar que los modelos, agentes y flujos open-source resuelven tareas reales; recién después tendría sentido evaluar hardware propio.

Pero ajustaría una parte de tu objetivo:

No existe una nube que no dependa absolutamente de ninguna organización, infraestructura física o gobierno.

Sí podemos construir un sistema **sin un punto central único de control o de falla.**

Incluso una red descentralizada depende de proveedores físicos, validadores, desarrolladores del protocolo, medios de pago, acceso a Internet y alguna forma de gobernanza. La meta realista es que **ninguno de esos componentes sea indispensable por sí solo.**

Nuevo paradigma para NativIQ

La arquitectura debería cumplir cinco principios:

1. **Modelos descargables y reemplazables.**
2. **Aplicaciones empaquetadas en contenedores portables.**
3. **Capacidad para cambiar de red de cómputo en pocas horas.**
4. **Datos cifrados y replicados fuera del proveedor de GPU.**
5. **Claves, configuraciones y repositorios bajo tu control.**

De esta manera, si una red desaparece, aumenta el precio, bloquea tu cuenta o es restringida en una jurisdicción, migrás el mismo sistema a otra.

Redes de cómputo que probaría

1. Akash Network — plataforma principal

Akash sería mi primera elección para desplegar servicios relativamente persistentes:

- APIs de inferencia.
- LibreChat.
- LiteLLM.
- vLLM u Ollama.
- Bases de datos de prueba.
- Agentes y aplicaciones Docker.
- Modelos Qwen y gpt-oss.

Es un mercado abierto donde diferentes proveedores compiten por alojar los contenedores. Permite solicitar modelos concretos de GPU mediante su configuración SDL y ofrece GPUs para inferencia, entrenamiento y procesamiento por lotes. Sus precios orientativos publicados son de USD 0,50–1,50 por hora para una RTX 4090, USD 1,50–2,50 para una A100 de 40 GB y USD 2,50–4 para una H100. (akash.network)

Uso recomendado: infraestructura principal de laboratorio y servicios que deban permanecer disponibles durante varias horas o días.

2. Nosana — experimentación económica

Nosana resulta especialmente interesante para encender una GPU, ejecutar una tarea y apagarla.

Al 12 de julio de 2026, su mercado muestra aproximadamente:

GPU	VRAM aproximada	Precio publicado
RTX 3090	24 GB	USD 0,192/h
RTX 4090	24 GB	USD 0,320/h
RTX 5090	32 GB	USD 0,400/h
RTX Pro 6000	96 GB	USD 1,00/h

La disponibilidad cambia en tiempo real: algunas GPU tienen numerosos proveedores disponibles y otras aparecen temporalmente sin capacidad libre. (nosana.com)

Esto significa que cien horas de pruebas costarían aproximadamente:

- RTX 3090: **USD 19,20.**
- RTX 4090: **USD 32.**
- RTX 5090: **USD 40.**
- RTX Pro 6000: **USD 100.**

Uso recomendado: comparar modelos, probar cuantizaciones, ejecutar benchmarks y utilizar modelos grandes durante períodos breves.

3. Golem Network — trabajos distribuidos y batch

Golem permite contratar recursos aportados por diferentes participantes de la red. Su documentación ya contempla la ejecución de modelos Qwen mediante Ollama sobre proveedores GPU, la creación de imágenes personalizadas y el acceso al modelo mediante una API local proxificada. (docs.golem.network)

Es menos directo que una máquina virtual tradicional: requiere trabajar con Yagna, imágenes Golem y pagos mediante GLM en Polygon. También puede tardar en descargar una imagen grande al proveedor si esta todavía no está almacenada en su caché. (docs.golem.network)

Uso recomendado: tareas por lotes, pruebas de distribución y experimentos que no requieran una instancia permanente.

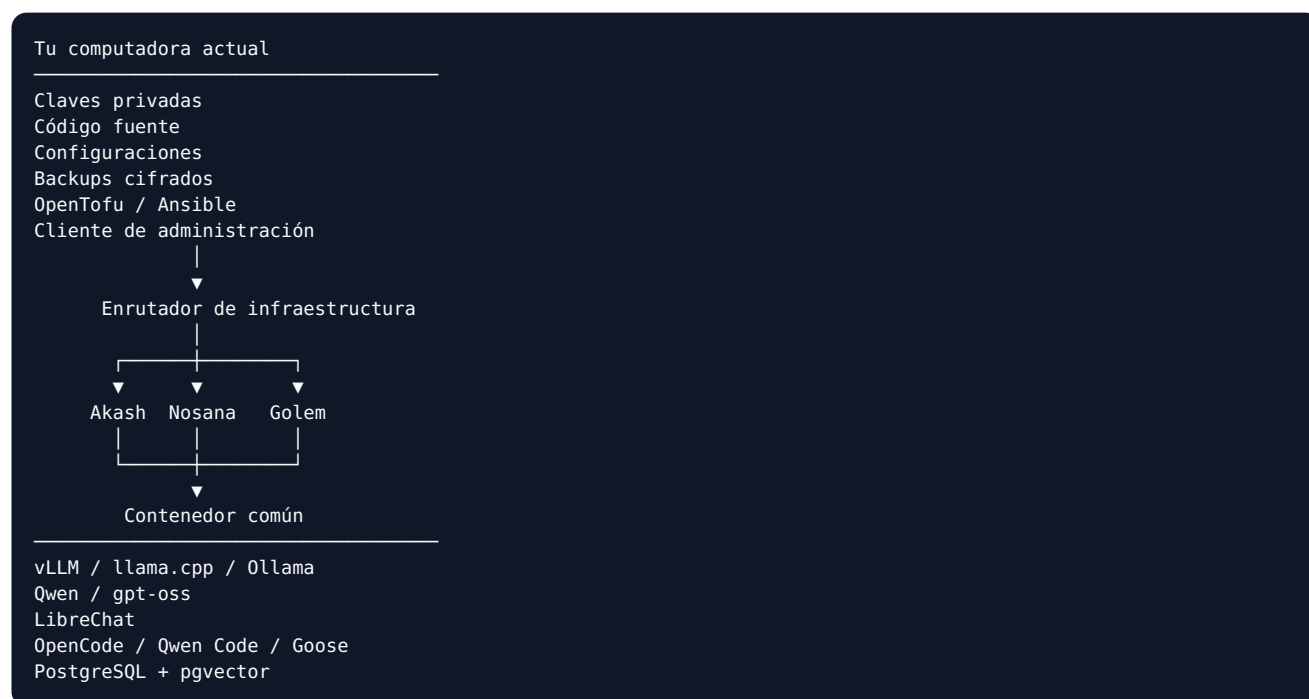
4. Bacalhau — capa futura de orquestación

Bacalhau no es principalmente un mercado donde comprar una GPU. Es un framework open-source para ejecutar trabajos distribuidos cerca de los datos y coordinar nodos ubicados en distintas nubes, regiones o instalaciones propias.

Puede trabajar con contenedores Docker y está diseñado para que los nodos continúen operando incluso con conectividad intermitente hacia el orquestador. (bacalhau.org)

Uso recomendado: cuando NativIQ tenga nodos en Akash, Nosana, servidores de clientes y eventualmente hardware propio, y quiera tratarlos como una red federada.

Arquitectura propuesta



El elemento importante es que **el contenedor y los datos no pertenezcan a la red que los ejecuta.**

Akash, Nosana o Golem solo aportarían temporalmente:

- GPU.
- CPU.
- RAM.
- Conectividad.
- Disco temporal.

NativIQ conservaría:

- Código.
- Modelos permitidos por sus licencias.
- Configuraciones.
- Prompts.
- Evaluaciones.
- Datos persistentes.
- Claves.
- Copias de seguridad.

Repositorios sin dependencia de GitHub

Git ya es distribuido, pero normalmente terminamos dependiendo de GitHub como servidor central.

Para disminuir esa dependencia utilizaría:

- Una copia completa en tu computadora.
- Un repositorio remoto convencional como respaldo secundario.
- **Radicle** como réplica peer-to-peer.

Radicle es una plataforma de colaboración construida sobre Git donde los repositorios se replican entre nodos y no existe una entidad única que controle la red. Los cambios e identidades se verifican criptográficamente. (radicle.xyz)

GitHub podría seguir existiendo como escaparate público o copia adicional, pero dejaría de ser la fuente única del proyecto.

Modelos y archivos sin un repositorio central

No dependería exclusivamente de Hugging Face para volver a descargar un modelo.

Cada modelo aprobado debería tener:

- Archivo original.
- Licencia.
- Model card.
- Hash SHA-256.
- Cuantización utilizada.
- Fecha de descarga.
- Copia local.
- Dos réplicas externas.

IPFS utiliza direccionamiento por contenido: el identificador depende del contenido del archivo y permite verificar que la copia recuperada sea exactamente la misma. Sin embargo, el archivo debe permanecer fijado o “pinned” en al menos un nodo para garantizar disponibilidad. (docs.ipfs.tech)

Filecoin añade una red de proveedores de almacenamiento con incentivos económicos y pruebas verificables de almacenamiento. (docs.filecoin.io)

Para modelos de decenas de gigabytes usaría:

```
Copia local cifrada
+
Segundo almacenamiento independiente
+
IPFS/Filecoin para redundancia
```

No confiaría exclusivamente en un único pinning service, gateway IPFS o repositorio de modelos.

Seguridad: la principal advertencia

Una GPU descentralizada pertenece físicamente a un tercero desconocido. La descentralización protege contra el control de una sola empresa, pero **no garantiza automáticamente confidencialidad**.

Akash reconoce que necesita mecanismos específicos para verificar:

- Identidad del proveedor.
- Hardware real.
- Ubicación.
- Disponibilidad.
- Responsabilidad del operador.

Su nuevo sistema de niveles de verificación todavía figura como propuesta en desarrollo. (akash.network)

Asimismo, el soporte de confidential computing con entornos TEE aparece en la hoja de ruta con finalización estimada para finales de julio de 2026. La propia propuesta reconoce que una nube formada

por decenas o cientos de proveedores independientes incrementa la necesidad de proteger los datos mientras están siendo procesados. ([akash.network](#))

Por eso distinguiría dos etapas:

Laboratorio y desarrollo

Se pueden utilizar proveedores comunitarios con:

- Código no sensible.
- Datos sintéticos.
- Documentos anonimizados.
- Repositorios de demostración.
- Credenciales temporales.
- Contenedores desechables.

Clientes empresariales

No colocaría datos bancarios, contratos, credenciales, información operativa de energía o documentos privados en cualquier nodo de la red.

Exigiría:

- Proveedor identificado o verificado.
- Disco cifrado.
- TLS.
- Secretos inyectados temporalmente.
- Contenedores sin privilegios.
- Sin acceso al repositorio completo.
- Datos minimizados.
- Confidential computing cuando esté disponible y comprobado.
- Contrato y jurisdicción apropiados cuando corresponda.

Presupuesto inicial realista

Laboratorio de uso moderado

Con GPU encendida solamente cuando estés trabajando:

Concepto	Presupuesto mensual
100-200 horas de GPU de 24-32 GB	USD 20-80
CPU, almacenamiento y servicios auxiliares	USD 10-40
Backups y almacenamiento distribuido	USD 5-20
Dominios, tráfico y contingencias	USD 5-20
Total orientativo	USD 40-160 mensuales

Una RTX 5090 funcionando las 730 horas de un mes costaría aproximadamente USD 292 al precio actualmente publicado por Nosana, antes de almacenamiento, transferencia y otros servicios. Por eso conviene apagarla automáticamente cuando no haya tareas. ([nosana.com](#))

Pruebas con modelos grandes

Podrías reservar una RTX Pro 6000 de 96 GB solamente durante:

- Un benchmark.

- Una refactorización grande.
- Una sesión con gpt-oss-120B.
- Una prueba de Qwen3-Coder-Next.
- Un ajuste LoRA.

Cien horas al precio actualmente publicado representarían alrededor de USD 100, siempre que haya capacidad disponible. (nosana.com)

Esto permite conocer el comportamiento real de una GPU de aproximadamente USD 8.000–30.000 sin comprarla.

Lo que haría ahora

Etapas 1 — Laboratorio soberano

Presupuesto máximo inicial: **USD 100**.

Probaría:

- Nosana con RTX 3090 y RTX 5090.
- Akash con una instancia GPU.
- Golem con un modelo pequeño.
- Qwen generalista.
- Qwen Coder.
- gpt-oss-20B.
- LibreChat.
- OpenCode.
- Qwen Code.
- Goose.
- vLLM y Ollama.

Etapas 2 — Portabilidad

El mismo modelo y la misma aplicación deben poder desplegarse en al menos dos redes sin modificar su código.

Criterio de aprobación:

```
Pérdida total de un proveedor
↓
Nuevo despliegue funcional
↓
Datos restaurados
↓
Sin cambiar la aplicación
```

Etapas 3 — Benchmark propio

Medir:

- Calidad.
- Velocidad.
- Costo por tarea terminada.
- Tiempo de arranque.
- Disponibilidad de GPU.
- Facilidad de migración.

- Estabilidad del proveedor.
- Seguridad.
- Recuperación ante fallos.

Etapa 4 — Decidir sobre hardware

Solo comprar hardware cuando se cumpla alguna de estas condiciones:

- El alquiler mensual supera sostenidamente el costo amortizado.
- Los datos no pueden procesarse legalmente en nodos externos.
- La disponibilidad de GPU perjudica a clientes.
- La latencia remota es inaceptable.
- Un cliente paga específicamente por infraestructura dedicada.

Conclusión

Tu nuevo planteamiento es mejor para el corto y mediano plazo:

No comprar infraestructura hasta que el uso real la justifique. Construir primero una arquitectura soberana, portable y distribuida sobre varias redes de cómputo.

No perseguiría una “descentralización absoluta”, porque es imposible garantizarla. Perseguiría algo más útil:

Que ninguna empresa, gobierno, red, repositorio, token o proveedor pueda detener por sí solo a NativIQ Systems.

La primera implementación debería utilizar **Akash como base, Nosana para GPU económica bajo demanda, Golem para experimentación distribuida y Bacalhau como futura capa federada**, manteniendo el código en Git/Radicle y los modelos respaldados mediante copias cifradas e IPFS/Filecoin.

También puede configurarse un control mensual de precios, disponibilidad y cambios regulatorios en estas redes para detectar cuándo conviene migrar.

Mauro

Intervención 9 · 2026-07-12 22:41:32Z

"Que ninguna empresa, gobierno, red, repositorio, token o proveedor pueda detener por sí solo a NativIQ Systems." Esta es mi premisa fundamental. Basado en esto último y pensando una perspectiva largoplacista, ¿Qué es más económico? ¿Local o la nube?

ChatGPT - avance de trabajo

Voy a responderlo como una decisión económica y de resiliencia, no solo como “precio por hora”. La clave será calcular el punto de equilibrio según uso, tamaño de modelo y cuánto cuesta cumplir tu premisa de no tener un único punto de falla.

ChatGPT

Sí: **a largo plazo, lo más económico no es “local” ni “nube” en estado puro. Es una arquitectura híbrida.**

Bajo tu premisa fundamental —que ninguna empresa, gobierno, red, repositorio, token o proveedor pueda detener a NativIQ— el resultado es:

Nube descentralizada y multi-red durante la etapa de validación; infraestructura local como núcleo cuando el uso sea sostenido; varias nubes como respaldo y expansión.

Respuesta por horizonte

Etapas	Opción más económica
Inicio y experimentación	Nube descentralizada
Uso irregular o por proyectos	Nube descentralizada
Uso diario moderado	Híbrido
Uso intensivo y estable	Local
Operación resiliente de largo plazo	Local + varias nubes
Modelos enormes usados ocasionalmente	Nube
Datos críticos o confidenciales	Local o infraestructura dedicada

Por qué la nube es más económica ahora

Actualmente, Nosana publica aproximadamente:

GPU	Precio
RTX 3090, 24 GB	USD 0,192/h
RTX 4090, 24 GB	USD 0,320/h
RTX 5090, 32 GB	USD 0,400/h
RTX Pro 6000, 96 GB	USD 1,00/h
H100	USD 1,50/h

La disponibilidad fluctúa: en la consulta actual había 23 RTX 3090, 13 RTX 4090, 17 RTX 5090 y cuatro RTX Pro 6000 disponibles, mientras que no había A100 de 80 GB ni H100 disponibles. Esto demuestra simultáneamente el bajo costo y la necesidad de usar más de una red. ([nosana.com](#))

Akash publica precios iniciales cercanos a:

- RTX 4090: USD 0,36/h.
- A100 de 80 GB: USD 1,36/h.
- H100 PCIe: USD 2,09/h.
- H100 SXM: USD 2,73/h. ([akash.network](#))

Ejemplo con una RTX 5090 virtual

A USD 0,40 por hora:

Utilización	Costo mensual	Costo anual
50 horas/mes	USD 20	USD 240
100 horas/mes	USD 40	USD 480
200 horas/mes	USD 80	USD 960
300 horas/mes	USD 120	USD 1.440
500 horas/mes	USD 200	USD 2.400
24 × 7	USD 292	USD 3.504

Durante la etapa inicial probablemente utilizarás una GPU grande entre 50 y 200 horas mensuales. En ese rango, comprar una workstation de varios miles de dólares no tiene sentido económico.

Además, la nube permite cambiar inmediatamente entre:

- 24 GB.
- 32 GB.
- 48 GB.
- 80 GB.
- 96 GB.

No quedás atrapado durante cuatro o cinco años con la memoria de una GPU concreta.

Cuándo empieza a ganar lo local

Tomemos una workstation nueva con RTX 5090:

- Inversión estimada internacional: USD 4.000–6.000.
- Vida económica: cuatro años.
- Valor residual: aproximadamente 20 %.
- Mantenimiento y reemplazos: aproximadamente 10 %.
- Consumo total bajo inferencia: alrededor de 600–700 W.

La RTX 5090 tiene 32 GB de memoria, una potencia gráfica máxima oficial de 575 W y NVIDIA indica un sistema de aproximadamente 1.000 W para una configuración de referencia. ([nvidia.com](https://www.nvidia.com))

Con una workstation de USD 6.000, electricidad de USD 0,10/kWh y cuatro años de uso, el punto de equilibrio frente a una RTX 5090 alquilada a USD 0,40/h se produce aproximadamente al superar:

Entre 350 y 450 horas de GPU mensuales.

Eso equivale aproximadamente a:

Entre 11 y 15 horas diarias de utilización real.

No significa tener la computadora encendida. Significa mantener la GPU procesando trabajo útil durante esas horas.

Regla económica sencilla

Uso sostenido	Mejor decisión
Menos de 100 h/mes	Nube
100–250 h/mes	Nube
250–400 h/mes	Evaluar caso por caso

Uso sostenido	Mejor decisión
400-500 h/mes	Local empieza a ser competitivo
Más de 500 h/mes	Local suele ser más económico
Uso 24 x 7	Local claramente más económico

En Argentina, donde importar una RTX 5090 y sus componentes puede costar bastante más que el precio internacional, el punto de equilibrio se desplaza todavía más hacia la derecha. Es decir, necesitás más horas mensuales para justificar la compra.

Modelos grandes: la situación cambia

Para ejecutar modelos de 80B-120B necesitarías aproximadamente:

- Una GPU de 80-96 GB.
- Varias GPU.
- O una máquina con mucha memoria unificada.

Una Mac Studio M3 Ultra puede configurarse con 256 GB de memoria unificada y ofrece 819 GB/s de ancho de banda, pero sigue representando una inversión considerable. Apple confirma que el M3 Ultra admite configuraciones de 96 y 256 GB. (support.apple.com)

Si una máquina grande cuesta USD 8.000-12.000 y una RTX Pro 6000 descentralizada cuesta USD 1/h, el punto de equilibrio puede aparecer aproximadamente entre:

180 y 300 horas mensuales, o unas 6-10 horas diarias.

Pero hay dos factores que favorecen seguir alquilando:

1. Probablemente no usarás el modelo de 120B todos los días.
2. El hardware y los modelos están evolucionando rápidamente.

Podrías usar:

- Un modelo de 27B durante el 90 % del trabajo.
- Un modelo de 80B o 120B solamente para arquitectura, debugging difícil o revisión.

Comprar una máquina de USD 10.000 para un modelo utilizado veinte horas al mes sería una mala asignación de capital.

El problema económico de una solución exclusivamente local

Una única computadora local contradice parcialmente tu premisa.

Puede detenerte:

- Una avería de GPU.
- Un problema de fuente.
- Un corte eléctrico.
- Un robo.
- Un incendio.
- Una falla de disco.
- Una interrupción del sitio.
- Una restricción de importación de repuestos.

Para que una infraestructura completamente local respete tu premisa necesitarías:

- Al menos dos nodos.
- Ubicaciones diferentes.
- Energía de respaldo.
- Dos conexiones a Internet.
- Copias externas.
- Repuestos.
- Mantenimiento.

Eso puede duplicar o triplicar la inversión.

Por tanto:

Local es más barato por hora de cómputo intensivo, pero no necesariamente es más barato como sistema resiliente.

El problema económico de una solución exclusivamente en la nube

Una única red descentralizada tampoco satisface tu premisa.

Nosana, por ejemplo, muestra precios muy bajos, pero su disponibilidad cambia por modelo de GPU. Akash también utiliza un mercado con disponibilidad y precios en tiempo real. (nosana.com)

Una red podría sufrir:

- Falta de GPU.
- Aumento de precios.
- Caída del token.
- Problemas de gobernanza.
- Vulnerabilidades.
- Regulación adversa.
- Abandono del desarrollo.
- Congestión o indisponibilidad regional.

La solución no es buscar “la red descentralizada perfecta”, sino poder utilizar indistintamente:

- Akash.
- Nosana.
- Golem.
- Proveedores convencionales como último recurso.
- Hardware propio cuando exista.

La arquitectura económicamente óptima

Primera etapa: 2026-2027

Nube descentralizada multi-red.

Presupuesto razonable:

- USD 50-150 mensuales.
- GPU encendida solamente durante pruebas.
- Modelos y contenedores replicados.
- Datos sintéticos o anonimizados.

- Dos redes de cómputo configuradas.

En doce meses:

USD 600-1.800.

Esto es muchísimo menos que una workstation de USD 6.000-10.000.

Segunda etapa: cuando NativIQ tenga trabajo sostenido

Comprar un nodo local relativamente económico:

- GPU usada de 24 GB o nueva de 32 GB.
- 64-128 GB de RAM.
- Linux.
- Almacenamiento local.
- UPS.

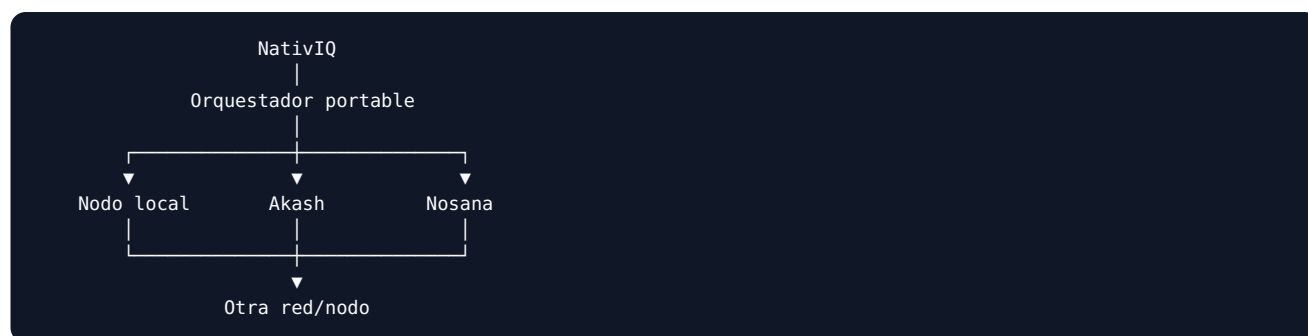
Ese nodo atendería:

- Trabajo cotidiano.
- Modelos 14B-32B.
- Datos sensibles.
- Desarrollo.
- Inferencia rutinaria.

Las redes externas atenderían:

- Picos.
- Modelos grandes.
- Segundo modelo de verificación.
- Respaldo ante fallos.
- Trabajo temporal paralelo.

Tercera etapa: operación comercial



Ningún componente individual sería indispensable.

Cuándo deberías comprar hardware

No lo decidiría por calendario. Lo decidiría mediante métricas.

Compraría el primer nodo local cuando se cumplan simultáneamente estas condiciones:

1. Gastás más de **USD 150-250 mensuales en GPU** durante seis meses.
2. Superás aproximadamente **300-400 horas mensuales** en una misma clase de GPU.
3. El modelo que necesitás cabe en hardware accesible.

4. Esperás continuar utilizándolo durante al menos tres años.
5. Necesitás procesar información que no debería salir a nodos externos.

Para modelos de 80-120B, exigiría un umbral más alto:

1. Más de USD 600-1.000 mensuales sostenidos.
2. Uso frecuente, no experimental.
3. Clientes dispuestos a pagar la infraestructura.
4. Necesidad real de privacidad o disponibilidad dedicada.

La conclusión desde tu premisa

Económicamente

- **Corto plazo:** nube descentralizada.
- **Mediano plazo:** nube descentralizada + nodo local modesto.
- **Largo plazo con alta utilización:** local como núcleo.
- **Largo plazo con resiliencia:** híbrido.

Estratégicamente

La nube te da:

- Flexibilidad.
- Bajo capital inicial.
- Cambio de hardware.
- Escalabilidad.
- Diversificación geográfica.

Lo local te da:

- Control físico.
- Privacidad.
- Operación offline.
- Costos marginales bajos.
- Independencia frente a redes y tokens.

La combinación más económica y coherente con NativIQ es:

Comprar solamente la capacidad base que utilizás todos los días y alquilar todo lo que sea variable, excepcional o demasiado grande.

En términos empresariales:

Local para la carga base; redes descentralizadas para picos, redundancia y modelos grandes.

No compraría ahora el ecosistema completo. Comenzaría con USD 50-100 mensuales distribuidos entre dos redes, mediría las horas y el costo por tarea, y compraría el primer nodo local únicamente cuando los datos demuestren que amortizarlo será más barato.

Mauro

Intervención 10 · 2026-07-12 22:48:40Z

El problema es que todo el tiempo salen modelos nuevos y el hardware también se actualiza. Para un emprendimiento de una sola persona es demasiado para manejar. Tendría que conseguir inversionistas. Dame tu opinion objetiva.

ChatGPT

Mi opinión objetiva: **sí, intentar sostener por tu cuenta un ecosistema completo de modelos, hardware, nubes descentralizadas, seguridad, almacenamiento y orquestación sería demasiado para una empresa de una sola persona.** Pero de ahí no se desprende que necesites inversionistas inmediatamente.

El error sería pensar que NativIQ debe convertirse desde el comienzo en un proveedor de infraestructura de IA soberana. Eso consumiría capital, tiempo y atención antes de tener clientes.

El problema no es solamente económico

Aunque consiguieras USD 50.000 para comprar hardware, seguirías teniendo que administrar:

- Nuevos modelos y cuantizaciones.
- Compatibilidad de runtimes.
- Actualizaciones de GPU y controladores.
- Redes descentralizadas.
- Seguridad y vulnerabilidades.
- Observabilidad y backups.
- Evaluaciones de modelos.
- Disponibilidad y soporte a clientes.
- Ventas, contenido, contratos y administración.

Para una sola persona, el recurso más limitado no es el dinero: **es la capacidad de concentración.**

Además, no existe una compra de hardware que te deje “actualizado para siempre”. Si perseguís cada nueva GPU y cada modelo que aparece, entrarías en una carrera que favorece a empresas con cientos de empleados y presupuestos enormes.

No competir en modelos ni infraestructura

NativIQ no debería competir con:

- OpenAI o Anthropic en entrenamiento de modelos.
- NVIDIA en hardware.
- Akash o Nosana en mercados de cómputo.
- Hugging Face en distribución de modelos.
- Grandes integradores en centros de datos.

Tu ventaja debería estar un nivel por encima:

Diseñar sistemas empresariales portables, seguros y resistentes a la dependencia de un proveedor.

No necesitás poseer toda la infraestructura para controlar la arquitectura.

Podés utilizar componentes externos siempre que sean reemplazables:

```
Proveedor A falla
↓
El sistema cambia al proveedor B
↓
El cliente sigue operando
```

La soberanía no significa fabricar cada componente. Significa que ningún componente sea irremplazable.

¿Necesitás inversionistas?

Ahora: probablemente no

Buscar inversión demasiado temprano puede generar varios problemas:

- Dilución significativa.
- Presión por crecer antes de entender el mercado.
- Obligación de contratar personal prematuramente.
- Costos administrativos y legales.
- Expectativas de convertirse en una plataforma tecnológica de gran escala.
- Pérdida de libertad estratégica.

Actualmente NativIQ todavía está definiendo:

- Oferta.
- Clientes ideales.
- Casos de uso.
- Proceso comercial.
- Arquitectura.
- Capacidad técnica real.
- Disposición de pago.

Un inversionista serio probablemente preguntaría:

- ¿Qué problema específico resolvés?
- ¿Quién paga?
- ¿Cuánto paga?
- ¿Cuál es el margen?
- ¿Qué parte es repetible?
- ¿Por qué no puede hacerlo una consultora tradicional?
- ¿Qué evidencia tenés de demanda?

Comprar hardware no responde esas preguntas.

Más adelante: posiblemente sí

La inversión empieza a tener sentido cuando el capital acelera algo que ya funciona.

Por ejemplo:

- Tenés entre tres y diez clientes pagos.
- Los proyectos comparten una arquitectura repetible.
- Existen ingresos recurrentes.
- La infraestructura externa cuesta demasiado.
- Necesitás contratar a un especialista en seguridad o infraestructura.
- Un cliente importante exige despliegue dedicado.
- La demanda supera tu capacidad de entrega.
- Ya sabés exactamente en qué se utilizará cada dólar.

En ese momento, podrías levantar capital para ampliar una operación validada, no para financiar una exploración tecnológica indefinida.

La estrategia adecuada para una sola persona

NativIQ debería comenzar como una **empresa de sistemas y servicios altamente productizada**, no como una compañía de infraestructura.

Tu trabajo personal

Concentrarte en:

- Detectar problemas empresariales.
- Diseñar la solución.
- Vender.
- Modelar los flujos.
- Definir seguridad y gobierno.
- Construir prototipos.
- Coordinar implementaciones.

Lo que delegás inicialmente

- La GPU: proveedores bajo demanda.
- Los modelos: open-weight existentes.
- El almacenamiento: servicios replicables.
- La autenticación: componentes open-source maduros.
- El monitoreo: herramientas existentes.
- La contabilidad y asuntos legales: profesionales externos.
- Trabajos técnicos puntuales: contratistas especializados.

Esto conserva control arquitectónico sin convertirte en administrador de un pequeño centro de datos.

Un núcleo tecnológico mínimo

No necesitás probar continuamente todos los modelos disponibles. Seleccionaría un conjunto estable durante ciclos de seis meses:

- Un modelo generalista.
- Un modelo de programación.
- Un modelo de embeddings.
- Un reranker.
- Un modelo de seguridad.
- Dos proveedores de cómputo.

Cada seis meses, ejecutás un benchmark. Solo cambiás un modelo cuando el nuevo demuestra una mejora material.

La política podría ser:

No adoptar un modelo nuevo por publicidad o benchmarks públicos. Adoptarlo únicamente si mejora al menos un 20 % el costo, la calidad, la velocidad o la seguridad en tareas reales de NativIQ.

Así evitás convertirte en un probador permanente de modelos.

Qué construiría primero

Elegiría un único producto o servicio inicial dentro de tu podio estratégico. Por ejemplo:

Automatización segura de un flujo documental u operativo B2B, desplegable sobre cualquier modelo y proveedor.

El cliente no compraría Qwen, Akash o una GPU. Compraría:

- Menos trabajo manual.
- Menos errores.
- Trazabilidad.
- Protección de datos.
- Continuidad operativa.
- Independencia tecnológica.

La infraestructura sería un medio invisible.

Un plan financiero más prudente

Etapas 1: autofinanciada

Presupuesto mensual aproximado:

- Cómputo y pruebas: USD 50–150.
- Herramientas y servicios: USD 50–100.
- Dominio, almacenamiento y backups: USD 20–50.
- Contratistas puntuales: según proyecto.

Objetivo: obtener el primer caso funcional y el primer cliente.

Etapas 2: financiada por clientes

Utilizar:

- Anticipos de implementación.
- Cuotas mensuales.
- Contratos piloto pagos.
- Servicios de diagnóstico.
- Mantenimiento recurrente.

Idealmente, cada nuevo cliente debería financiar la infraestructura que necesita.

Etapas 3: contratación antes que inversión

Tu primera necesidad probablemente no será una GPU. Será una persona que complemente tus capacidades:

- DevOps/MLOps.
- Seguridad.
- Backend.
- Ventas técnicas.
- Gestión de proyectos.

Podés comenzar con freelancers o colaboradores por proyecto, sin crear una estructura fija.

Etapas 4: inversión externa

Solo cuando tengas una solución repetible y capital para acelerar:

- Contratación.
- Desarrollo de producto.
- Certificaciones.
- Infraestructura multiusuario.

- Expansión internacional.
- Ventas.

Veredicto

No intentaría construir ahora todo el ecosistema completo ni compraría hardware costoso. Tampoco buscaría inversionistas todavía.

Haría esto:

1. Congelar un stack tecnológico simple durante seis meses.
2. Usar GPU bajo demanda en dos proveedores.
3. Construir una solución empresarial concreta.
4. Obtener uno o varios pilotos pagos.
5. Documentar resultados y costos.
6. Contratar ayuda especializada cuando el trabajo lo exija.
7. Buscar inversión solamente si existe demanda comprobada que no puedas atender solo.

Tu premisa de soberanía sigue siendo válida, pero debe aplicarse con disciplina:

NativIQ no necesita controlar toda la tecnología. Necesita controlar las decisiones, los datos, la arquitectura y la capacidad de migrar.

Ese objetivo sí es alcanzable por una persona durante la etapa inicial. La infraestructura completa, la concurrencia empresarial y la expansión internacional pueden llegar después, financiadas por ingresos, socios o inversión cuando exista evidencia suficiente.

Mauro

Intervención 11 · 2026-07-12 22:56:56Z

Es que yo quiero ser el único de mi emprendimiento. Yo asumo el riesgo, pero me quedo con todos los beneficios si es que los hay. Con NativIQ quisiera tener como ganancia unos U\$D 10k por mes y ahí plantarme. No me interesa crecer mucho más. Mi emprendimiento es de construcción de sistemas AI-native. No de un proveedor de hardware. No quisiera tener que conseguir inversiones porque ahí los beneficios no serían míos. Tampoco tener que contratar empleados, porque tendría que pagar sueldos. ¿Se entiende?

ChatGPT

Sí, se entiende perfectamente.

Tu objetivo no es construir una startup tradicional. Es construir una **empresa unipersonal, altamente rentable, técnicamente sofisticada y deliberadamente pequeña.**

La definición correcta de NativIQ sería:

Una firma boutique unipersonal que diseña, construye y mantiene sistemas AI-native para empresas, con una meta aproximada de USD 10.000 mensuales de ganancia y sin necesidad de empleados, inversores ni infraestructura propia excesiva.

Eso cambia bastante la estrategia.

Lo que NativIQ no debería ser

- Una empresa de hardware.

- Un proveedor de GPU.
- Una plataforma de infraestructura.
- Una startup financiada por capital de riesgo.
- Una consultora con decenas de empleados.
- Una empresa obsesionada con crecer indefinidamente.
- Un laboratorio que prueba cada modelo nuevo que aparece.

Lo que sí debería ser

- Un negocio de alta especialización.
- Pocos clientes, pero buenos.
- Proyectos de alto valor.
- Costos fijos bajos.
- Herramientas open-source y reemplazables.
- Infraestructura alquilada solo cuando se necesita.
- Procesos muy automatizados.
- Mantenimiento recurrente.
- Márgenes altos.
- Trabajo concentrado en problemas empresariales valiosos.

La meta de USD 10.000 mensuales es compatible con un negocio unipersonal

No necesitás cientos de clientes.

Podrías alcanzar ese nivel de distintas maneras:

Modelo	Ejemplo
2 clientes grandes	USD 5.000 mensuales cada uno
4 clientes medianos	USD 2.500 mensuales cada uno
5 clientes	USD 2.000 mensuales cada uno
1 implementación + mantenimiento	USD 7.000 + USD 3.000 recurrentes
Proyectos trimestrales	USD 30.000 por trimestre

Probablemente el mejor modelo para vos sea una combinación de:

- Implementaciones iniciales de alto valor.
- Mantenimiento mensual.
- Mejoras y ampliaciones.
- Auditorías de seguridad, datos y automatización.
- Soporte limitado y bien definido.

Tu verdadero límite no será la tecnología

Será tu tiempo.

Como único integrante, no podés administrar simultáneamente:

- Venta.
- Desarrollo.
- Infraestructura.
- Soporte.
- Administración.

- Contabilidad.
- Seguridad.
- Actualización permanente de modelos.
- Atención de incidentes.

Por eso NativIQ debe diseñarse para que cada cliente requiera poca atención después de la implementación.

La pregunta principal no debería ser:

“¿Cuál es el modelo más potente?”

Sino:

“¿Qué arquitectura me permite atender varios clientes sin convertirme en esclavo del soporte?”

La arquitectura debe reducir tu carga operativa

Usaría un stack estable y pequeño:

- Un modelo generalista principal.
- Un modelo especializado en código.
- Un modelo de embeddings.
- Un reranker.
- Un modelo de seguridad.
- Dos proveedores de cómputo intercambiables.
- Contenedores reproducibles.
- Backups automáticos.
- Observabilidad.
- Alertas.
- Documentación y procedimientos de recuperación.

No probaría todas las herramientas open-source del mercado.

Seleccionaría un stack durante seis o doce meses y solo lo cambiaría si aparece una mejora relevante y demostrable.

Regla para adoptar nuevas tecnologías

No cambiar por novedad.

Cambiar solamente cuando una herramienta nueva produzca al menos una de estas mejoras:

- 20 % menos costo.
- 20 % más calidad.
- 20 % menos tiempo de mantenimiento.
- Una mejora importante de seguridad.
- Una reducción real de dependencia.
- Una capacidad solicitada por un cliente.

Así evitás que NativIQ se convierta en un trabajo permanente de actualización tecnológica.

Sobre empleados

Tu razonamiento es válido.

Un empleado no solo implica salario. También implica:

- Cargas sociales.
- Gestión.
- Supervisión.
- Responsabilidad.
- Coordinación.
- Riesgo laboral.
- Menor flexibilidad.
- Más facturación necesaria para sostener la estructura.

Para una empresa cuya meta son USD 10.000 mensuales de ganancia, contratar empleados podría alejarte de tu objetivo en lugar de acercarte.

Eso no significa que nunca puedas contratar servicios externos puntuales.

Podrías utilizar:

- Contador.
- Abogado.
- Diseñador.
- Especialista de seguridad.
- Técnico de infraestructura.
- Freelancer para una integración muy específica.

Pero serían proveedores temporales, no empleados permanentes ni socios del negocio.

Vos conservarías:

- La propiedad.
- La relación con el cliente.
- La arquitectura.
- La facturación.
- La dirección técnica.
- El beneficio residual.

Sobre inversionistas

Tampoco parecen compatibles con tu objetivo.

Un inversor normalmente espera:

- Crecimiento acelerado.
- Mayor facturación.
- Contrataciones.
- Expansión.
- Participación accionaria.
- Eventual venta o retorno de capital.

Pero vos querés optimizar para:

- Independencia.
- Rentabilidad.
- Control.
- Bajo estrés.
- Tamaño limitado.
- Beneficio personal.

Son objetivos distintos.

No necesitás inversión si podés financiar la operación con:

- Tus ingresos actuales.
- Anticipos de clientes.
- Pilotos pagos.
- Suscripciones mensuales.
- Alquiler de infraestructura bajo demanda.

Tu modelo ideal se parece más a una boutique técnica

Podrías posicionarte como:

Diseñador independiente de sistemas AI-native seguros y portables para empresas regionales.

Tus clientes no te pagarían por horas de programación. Te pagarían por:

- Automatizar un proceso caro.
- Reducir errores.
- Integrar sistemas.
- Asegurar datos.
- Evitar dependencia de un proveedor.
- Mejorar trazabilidad.
- Disminuir trabajo manual.
- Crear una capacidad interna que antes no existía.

Eso permite cobrar más sin necesitar muchos clientes.

Un objetivo operativo razonable

Para ganar USD 10.000 mensuales, probablemente deberías apuntar a facturar algo más, porque habrá costos:

- GPU y nube.
- Herramientas.
- Impuestos.
- Contador.
- Seguros.
- Viajes.
- Soporte.
- Hardware.
- Tiempo no facturable.

Dependiendo de tu estructura fiscal y costos, podrías necesitar facturar aproximadamente:

USD 12.000-15.000 mensuales para conservar cerca de USD 10.000 de ganancia.

No es una cifra exacta, pero es una referencia prudente.

El tipo de clientes que necesitás

No te convienen clientes pequeños que paguen USD 100 o USD 300 al mes y exijan soporte permanente.

Te convienen empresas que:

- Pierdan miles de dólares por procesos manuales.

- Tengan documentación compleja.
- Manejen operaciones críticas.
- No puedan internalizar fácilmente el sistema.
- Valoren la seguridad y continuidad.
- Puedan pagar USD 2.000–5.000 mensuales.
- Prefieran tratar con un especialista senior.

Esto coincide bastante con los sectores que ya elegiste:

- Energía.
- Banca.
- Seguros.
- Comercio y logística.
- Bienes raíces.

Riesgo principal de tu modelo

El riesgo no es no tener empleados.

El riesgo es convertirte en el único punto de falla.

Aunque seas el único dueño y trabajador, los sistemas deben estar documentados para que:

- Un cliente pueda recuperar su operación.
- Un proveedor técnico puntual pueda intervenir.
- Existan backups.
- Las claves tengan procedimientos de recuperación.
- Los despliegues sean reproducibles.
- No dependan únicamente de tu memoria.

Ser unipersonal no debe significar que todo exista solamente en tu cabeza.

Mi opinión objetiva

Tu planteamiento es coherente y viable.

No necesitas inversionistas. No necesitas empleados. No necesitas comprar un centro de datos. No necesitas perseguir cada nuevo modelo.

Sí necesitas:

- Elegir problemas de alto valor.
- Cobrar correctamente.
- Limitar el número de clientes.
- Productizar tus servicios.
- Automatizar tu propia operación.
- Mantener costos bajos.
- Diseñar sistemas fáciles de mantener.
- Evitar proyectos demasiado personalizados.
- Proteger tu tiempo.

La estrategia óptima para NativIQ no es crecer al máximo.

Es alcanzar un **punto de suficiencia rentable**:

Pocos clientes, sistemas valiosos, costos bajos, control total y aproximadamente USD 10.000 mensuales de beneficio.

Ese objetivo es mucho más compatible con una empresa boutique unipersonal que con una startup financiada o una compañía de infraestructura.

Mauro

Intervención 12 · 2026-07-12 23:11:03Z

- 1. Para hacer el trabajo de mis empleados quiero construir agentes eficientes, eficaces y consistentes. Para que manejen desde el soporte al cliente, arreglo de fallos, contabilidad, etc. Para las vacaciones los dejaría trabajando.
- 2. Yo quiero ser el único punto de falla. Pero si me va bien, obtengo todos los beneficios.
- 3. Lo que quiero hacer, es lo que me recomendaste.

ChatGPT

Sí, se entiende y es coherente con lo que venimos construyendo.

La formulación más precisa sería:

NativIQ Systems será una empresa unipersonal operada por Mauro y ejecutada diariamente por una fuerza de trabajo de agentes AI-native.

Vos asumirías:

- Propiedad y riesgo.
- Dirección estratégica.
- Relación comercial principal.
- Decisiones críticas.
- Diseño de los sistemas.
- Aprobaciones excepcionales.
- Totalidad de las ganancias.

Los agentes asumirían gran parte del trabajo operativo que normalmente requeriría empleados.

No necesitás empleados; necesitás una organización de agentes

La estructura podría ser:

Área	Agente responsable
Atención al cliente	Respuestas, clasificación y seguimiento
Soporte técnico	Diagnóstico, recopilación de logs y soluciones
Reparación de fallos	Pruebas, parches, rollback y validación
Operaciones	Monitoreo, despliegues, backups y alertas
Administración	Facturación, conciliaciones y documentación
Contabilidad asistida	Registro, clasificación y preparación de información
Ventas	Investigación, calificación y seguimiento de prospectos
Documentación	Manuales, informes y actualización de procedimientos
Seguridad	Control de accesos, auditoría y detección de anomalías
Gobierno	Verificación de políticas y trazabilidad

No serían simples chatbots. Serían agentes con:

- Herramientas delimitadas.
- Memoria operativa.
- Procedimientos escritos.
- Permisos.
- Métricas.
- Pruebas.
- Registros de actividad.
- Mecanismos de recuperación.
- Límites explícitos.

La distinción importante

Vos querés ser el único punto humano de falla, pero no necesitás ser el único ejecutor de cada tarea.

Eso significa:

Dependencia estratégica de Mauro, pero independencia operativa temporal.

Durante tus vacaciones, los agentes deberían poder:

- Responder consultas habituales.
- Resolver incidentes conocidos.
- Ejecutar procedimientos documentados.
- Reiniciar servicios.
- Restaurar versiones anteriores.
- Generar facturas y reportes.
- Cobrar o recordar pagos dentro de reglas definidas.
- Informar a clientes sobre incidentes.
- Escalar y dejar en espera los casos extraordinarios.

Pero no deberían tomar decisiones irreversibles como:

- Firmar contratos importantes.
- Modificar precios estructurales.
- Asumir compromisos legales.
- Eliminar datos críticos.
- Cambiar la arquitectura completa.
- Realizar transferencias extraordinarias.
- Desplegar cambios no probados.
- Responder incidentes fuera de sus procedimientos.

Niveles de autonomía

Diseñaría cada agente con cuatro niveles.

Nivel 0 — Observar

Puede leer, analizar y recomendar, pero no modificar nada.

Ejemplos:

- Analizar logs.
- Revisar una factura.
- Clasificar un ticket.

- Detectar un posible error.

Nivel 1 — Ejecutar tareas reversibles

Puede actuar cuando la operación sea segura y recuperable.

Ejemplos:

- Reiniciar un contenedor.
- Enviar una respuesta previamente aprobada.
- Crear un borrador.
- Ejecutar pruebas.
- Actualizar un registro.

Nivel 2 — Resolver dentro de límites

Puede tomar decisiones dentro de políticas específicas.

Ejemplos:

- Aplicar un parche si todas las pruebas pasan.
- Restaurar automáticamente la última versión estable.
- Otorgar una extensión de servicio dentro de un límite.
- Reprocesar documentos fallidos.

Nivel 3 — Esperar a Mauro

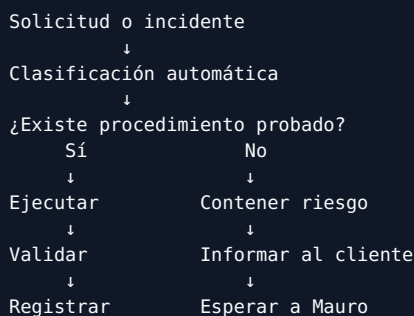
La tarea se registra, se contiene el riesgo y queda pendiente.

Ejemplos:

- Un incidente de seguridad grave.
- Una disputa comercial.
- Un cambio contractual.
- Un fallo sin procedimiento conocido.
- Una operación financiera excepcional.

Cómo funcionarían durante tus vacaciones

El sistema debería entrar en un **modo de operación autónoma limitada**.



La clave no es que los agentes puedan resolver absolutamente todo. La clave es que puedan:

- Mantener el servicio estable.
- Evitar daños.
- Resolver la mayoría de los casos frecuentes.
- Informar con honestidad.
- Postergar las decisiones excepcionales.

Agentes de reparación

Para corregir fallos con seguridad, el agente no debería editar directamente producción.

El flujo debería ser:

1. Detectar el incidente.
2. Reproducirlo en un entorno aislado.
3. Crear una rama o worktree.
4. Generar un parche.
5. Ejecutar pruebas.
6. Ejecutar análisis de seguridad.
7. Comparar el comportamiento anterior y posterior.
8. Desplegar en un entorno de prueba.
9. Realizar un despliegue gradual.
10. Volver automáticamente a la versión estable si falla.

Para incidentes desconocidos, la opción segura es degradar o suspender una función antes que improvisar una reparación peligrosa.

Contabilidad y administración

Los agentes pueden manejar gran parte del trabajo administrativo:

- Clasificar comprobantes.
- Conciliar movimientos.
- Preparar reportes.
- Emitir facturas según reglas.
- Detectar pagos faltantes.
- Calcular ingresos y gastos.
- Organizar documentación.
- Preparar información para obligaciones fiscales.

Cuando una operación requiera una firma, certificación o profesional habilitado, el agente prepara todo y vos o el profesional externo realizan únicamente la intervención obligatoria.

Eso no implica tener empleados. Un contador o abogado puntual sería un proveedor regulado, no parte estructural de NativIQ.

Tu modelo económico

NativIQ no necesita maximizar empleados ni facturación. Debe maximizar:

Ganancia por hora de atención humana de Mauro.

Por ejemplo, con cuatro clientes que paguen USD 3.500 mensuales:

- Facturación: USD 14.000.
- Infraestructura y servicios: USD 1.000–2.000.
- Gastos profesionales, fiscales y operativos: según estructura.
- Beneficio objetivo: aproximadamente USD 10.000.

La condición es que los agentes absorban la mayor parte del mantenimiento y que cada cliente tenga un sistema estandarizado, documentado y observable.

Riesgo aceptado

Tu decisión implica aceptar conscientemente que:

- Si vos abandonás el negocio, NativIQ probablemente no continúa igual.
- La capacidad de innovación depende de vos.
- Las decisiones excepcionales dependen de vos.
- Existe un límite natural de clientes y complejidad.

Pero también obtenés:

- El 100 % del capital.
- El 100 % de las ganancias.
- Control estratégico.
- Libertad para detener el crecimiento.
- Ausencia de presión de inversores.
- Ausencia de costos laborales permanentes.
- Menos política organizacional.
- Menos coordinación humana.

No es una startup tradicional. Es una **empresa personal aumentada por inteligencia artificial**.

Conclusión

Lo que querés hacer es viable, siempre que NativIQ se construya desde el comienzo para ese modelo:

Un propietario humano, una organización de agentes, pocos clientes de alto valor, sistemas repetibles, autonomía controlada y ninguna obligación de crecer más allá de aproximadamente USD 10.000 mensuales de beneficio.

Vos seguirías siendo el único responsable y propietario. Los agentes no reemplazarían tu autoridad: reemplazarían la necesidad de crear una estructura laboral tradicional.

Mauro

Intervención 13 · 2026-07-12 23:20:45Z

Exacto ChatGPT. Diste en el clavo con la conclusión. Con lo importante aclarado, recomiendame decisiones realistas en cuanto al hardware y herramientas que necesitaré.

ChatGPT - avance de trabajo

Voy a aterrizarlo en una arquitectura mínima, operable por una sola persona: qué comprar ahora, qué alquilar, qué posponer y qué herramientas conviene estandarizar para que los agentes puedan trabajar sin convertirte en administrador de infraestructura.

ChatGPT

Se entiende. Con NativIQ definido como **empresa unipersonal operada por agentes**, la decisión realista es dejar de intentar poseer “todo el ecosistema” y construir una plataforma pequeña, portable y suficientemente redundante.

Mi recomendación corregida es:

Comprar solamente hardware de continuidad y desarrollo. Alquilar la GPU. Mantener un stack open-source pequeño durante períodos de 6-12 meses.

1. Hardware que compraría ahora

A. Tu computadora principal

No compraría una workstation nueva mientras tu equipo actual pueda desarrollar software correctamente.

Configuración objetivo:

Componente	Mínimo práctico	Recomendado
RAM	32 GB	64 GB
SSD	1 TB NVMe	2 TB NVMe
CPU	6 núcleos modernos	8-16 núcleos
GPU	No obligatoria	La que ya poseas
Sistema	Windows + entorno Linux	Linux o Windows + Linux separado

Esta computadora ejecutaría:

- VS Code.
- OpenCode.
- Contenedores de desarrollo.
- Bases de datos locales.
- Pruebas.
- Agentes pequeños.
- Interfaces administrativas.
- Simulaciones con datos no sensibles.

No intentaría ejecutar localmente el modelo principal de NativIQ. Para eso usaría GPU bajo demanda.

B. Tu Lenovo V330 como plano de control

La Lenovo que estás preparando como servidor Linux tiene un papel muy útil. Le asignaría:

- Forgejo.
- Uptime Kuma.
- Servidor VPN.
- Repositorio de configuraciones.
- Tareas programadas.
- Backups.
- Registro de despliegues.
- Copia de documentación.
- Watchdog de las nubes externas.

Configuración recomendable:

- 16 GB de RAM como mínimo; 32 GB si el equipo lo admite.
- SSD de 500 GB o 1 TB.
- Linux Debian o Ubuntu Server.
- Sin entorno gráfico.
- Conexión Ethernet.

- UPS.

No alojaría ahí modelos grandes, ERPNext, Zammad, Temporal y todas las bases de datos al mismo tiempo. El Lenovo debe ser **pequeño, estable y aburrido**, no otro proyecto que mantener.

C. Protección y respaldo

Estas son compras más importantes que una GPU:

- UPS de onda sinusoidal de 1500–2000 VA.
- Dos discos externos de 2–4 TB.
- Un disco conectado para backups automáticos.
- Otro desconectado o ubicado fuera de tu casa.
- Un SSD NVMe de 2 TB para la computadora principal, si todavía no lo tenés.
- Una segunda conexión a Internet solamente cuando existan clientes pagos.

No compraría todavía:

- NAS.
- Switch de 10 GbE.
- Rack.
- Mac Studio.
- RTX 5090.
- Servidor Threadripper.
- GPU profesional.
- Refrigeración especializada.

Referencias actuales de mercado

Las publicaciones siguientes sirven para dimensionar la diferencia de precios. No las considero recomendaciones automáticas de compra; habría que comprobar stock, garantía, estado y especificaciones eléctricas.

Zotac GeForce RTX 3090 24 GB

Description: None Price: \$ 719.600,00 URL: Number of Reviews: 2 Rating: 5.0 Featured Tag: Mejor VRAM económica Merchants: Inovamusicnet

INNO3D RTX 3090 iChill X4 24 GB

Description: None Price: \$ 908.194,00 URL: Number of Reviews: 1 Rating: 5.0 Featured Tag: Alternativa 3090 Merchants: Exxit

Gainward RTX 3090 Phoenix 24 GB

Description: None Price: \$ 1.187.765,73 URL: Number of Reviews: None Rating: None Featured Tag: Otra opción 24 GB Merchants: Maximus Gaming Hardware y otros

MSI RTX 5090 Gaming Trio 32 GB

Description: None Price: \$ 5.180.000,00 URL: Number of Reviews: 295 Rating: 4.6 Featured Tag: No comprar todavía Merchants: Inovamusicnet y otros

Gigabyte RTX 5090 Aorus Master 32 GB

Description: None Price: \$ 6.363.700,00 URL: Number of Reviews: 49 Rating: 4.7 Featured Tag: Muy alto costo Merchants: Compra Gamer y otros

Mac mini M4 Pro 64 GB / 2 TB

Description: None Price: \$ 3.848.600,00 URL: Number of Reviews: 1 Rating: 5.0 Featured Tag: Alternativa silenciosa Merchants: Inovamusicnet

UPS online 1000 VA senoidal pura

Description: None Price: \$ 697.400,00 URL: Number of Reviews: None Rating: None Featured Tag: Para carga moderada Merchants: Enertik

UPS TRV Neo 2000

Description: None Price: \$ 301.999,00 URL: Number of Reviews: None Rating: None Featured Tag: Verificar onda y potencia Merchants: Venex y otros

Una RTX 3090 ofrece 24 GB de VRAM; una RTX 5090 sube a 32 GB. Es una mejora real, pero no transforma una GPU de consumo en una máquina capaz de alojar cómodamente cualquier modelo grande. ([nvidia.com](https://www.nvidia.com))

2. La decisión sobre GPU

Ahora: ninguna GPU dedicada

Usaría dos redes:

- **Nosana** para trabajos breves, benchmarks e inferencia bajo demanda.
- **Akash** para servicios contenerizados que deban permanecer desplegados más tiempo.

Nosana permite lanzar cargas mediante contenedores y API, con precios y disponibilidad que cambian según la GPU. Akash emplea proveedores independientes y subastas, y acepta imágenes de contenedor sin exigir que reescribas la aplicación. (docs.nosana.com)

Mantendría un presupuesto inicial de:

USD 50-150 mensuales de GPU, con un límite duro.

Eso alcanza para desarrollar, hacer benchmarks y ejecutar modelos grandes durante períodos limitados sin inmovilizar capital.

Primera GPU local futura: RTX 3090 usada

Cuando la nube deje de ser económica, mi primera opción no sería una RTX 5090. Sería una **RTX 3090 de 24 GB usada**, siempre que:

- Tenga prueba de estrés.
- Las temperaturas sean correctas.
- No presente artefactos.
- La memoria sea estable.
- La fuente y el gabinete sean compatibles.
- El precio sea sustancialmente inferior a una GPU nueva.

La compraría cuando se cumplan estas tres condiciones:

1. Gastás más de USD 200-300 mensuales en una misma clase de GPU.
2. Eso ocurre durante al menos seis meses.
3. Tus modelos principales caben en 24 GB.

La 3090 sería tu **carga base local**. Nosana y Akash continuarían como respaldo, picos de capacidad y modelos grandes.

Cuándo consideraría una RTX 5090

Solamente cuando:

- 24 GB sean insuficientes de forma recurrente.
- Necesites 32 GB para un modelo productivo específico.
- La mejora se mida en tareas reales.
- El ahorro esperado amortice la compra en menos de 24-30 meses.
- Ya estés cerca de tu objetivo de rentabilidad.

No la compraría “para estar preparado”. El hardware preparado para todo no existe.

3. Herramientas que estandarizaría

La mayor amenaza para una empresa unipersonal no es usar una herramienta imperfecta. Es tener que mantener veinte herramientas que hacen cosas parecidas.

Núcleo de desarrollo

OpenCode

Lo elegiría como agente principal de programación porque es open-source, tiene licencia MIT, funciona en terminal, escritorio e IDE y permite conectar diferentes proveedores o endpoints locales. (opencode.ai)

Lo usaría para:

- Implementar funcionalidades.
- Investigar errores.
- Crear pruebas.
- Preparar parches.
- Revisar repositorios.
- Mantener los sistemas de clientes.

Qwen Code

Lo mantendría instalado como alternativa y banco de pruebas, pero no usaría simultáneamente dos agentes de programación todos los días. Qwen Code soporta modelos locales y externos, MCP, memoria, skills y subagentes. (github.com)

Política:

OpenCode como herramienta diaria; Qwen Code como alternativa semestral o para tareas donde demuestre una ventaja.

Goose

Goose sería tu asistente operativo personal, no el motor central de cada sistema vendido. Cuenta con aplicación de escritorio, CLI, API y extensiones MCP, y puede cambiar entre diferentes proveedores. (goose-docs.ai)

Usos:

- Investigación.
- Preparación de documentación.
- Automatización de tareas locales.
- Operaciones administrativas.
- Ejecución guiada de runbooks.
- Asistencia durante la construcción de agentes.

4. Framework para los agentes de NativIQ

PydanticAI como framework

Para los agentes que formen parte de los productos de NativIQ utilizaría **PydanticAI**.

Razones:

- Python.
- Salidas estructuradas y validadas.
- Independencia del modelo.
- Herramientas tipadas.
- Evaluaciones.
- Soporte para ejecución durable.
- Integración con flujos humanos y de larga duración. (github.com)

No construiría los agentes empresariales directamente dentro de Goose, LibreChat u OpenCode. Esas herramientas están bien para tu trabajo personal, pero los productos deben tener un runtime controlado por vos.

Temporal para procesos críticos

Temporal lo incorporaría cuando exista el primer cliente pago y necesites procesos que:

- Sobrevivan reinicios.
- Reintenten operaciones.
- Continúen después de una caída.
- Esperen horas o días.
- Conserven el estado.
- Requieran aprobaciones.

Temporal es self-hostable y sus workflows reanudan el trabajo después de fallos intermitentes. Para desarrollo puede utilizarse un servidor sencillo; el despliegue productivo debe incorporarse solamente cuando esa durabilidad sea necesaria. (temporal.io)

No lo instalaría el primer día. La secuencia sería:



5. Herramientas para operar NativIQ sin empleados

Función	Herramienta recomendada	Momento
Código y reparación	OpenCode	Ahora
Asistente operativo	Goose	Ahora
Agentes productivos	PydanticAI	Ahora
Flujos durables	Temporal	Primer cliente
Repositorios	Forgejo	Ahora
Secretos	OpenBao	Primer despliegue
Tickets	Zammad	Primer cliente

Función	Herramienta recomendada	Momento
Administración	ERPNext	Cuando factures regularmente
Disponibilidad	Uptime Kuma	Ahora
Métricas avanzadas	Prometheus + Grafana	Varios servicios
Base transaccional	PostgreSQL	Ahora
Recuperación documental	pgvector	Cuando exista RAG
Despliegues	Docker Compose	Ahora
Infraestructura	OpenTofu + Ansible	Primer despliegue repetible
Backups	Restic + almacenamiento doble	Ahora

Repositorios: Forgejo

Forgejo es una alternativa ligera y completamente libre para autohospedar repositorios Git. Lo instalaría en el Lenovo, pero mantendría además dos réplicas independientes. ([forgejo.org](#))

Estructura:

```
Forgejo en el Lenovo
+
Mirror en GitHub o Codeberg
+
Backup cifrado fuera del sitio
```

La soberanía no exige abandonar GitHub; exige que perder GitHub no te detenga.

Secretos: OpenBao

OpenBao almacenaría:

- Claves de APIs.
- Credenciales de bases de datos.
- Certificados.
- Tokens temporales.
- Credenciales de los agentes.

Es un gestor open-source mantenido bajo la OpenSSF de Linux Foundation. ([openbao.org](#))

Ningún agente debería recibir una carpeta llena de contraseñas. Debe solicitar solamente el secreto necesario, durante el tiempo necesario.

Soporte: Zammad

Zammad centralizaría:

- Correos.
- Tickets.
- Incidentes.
- Historial con clientes.
- Clasificación.
- Estado de cada caso.
- Respuestas automáticas.

Es un sistema de soporte self-hosted y open-source. ([zammad.org](#))

Tu agente de soporte no debería comunicarse directamente desde una cuenta de correo sin registro. Todas las conversaciones deberían convertirse en tickets auditables.

Administración: ERPNext

ERPNext puede manejar:

- Clientes.
- Presupuestos.
- Facturas.
- Compras.
- Pagos.
- Proyectos.
- Contabilidad interna.
- Reportes.

Es una plataforma ERP completamente open-source que incluye contabilidad y facturación. (erpnext.com)

El agente administrativo podría:

- Preparar facturas.
- Clasificar comprobantes.
- Conciliar movimientos.
- Recordar vencimientos.
- Generar reportes.
- Preparar información para el contador.

No le permitiría:

- Presentar declaraciones impositivas sin revisión.
- Crear nuevos beneficiarios bancarios.
- Transferir importes extraordinarios.
- Alterar registros contables cerrados.
- Aceptar contratos.

6. Monitoreo y reparación

Comenzar con Uptime Kuma

Uptime Kuma es suficiente inicialmente para vigilar:

- Sitios web.
- APIs.
- Certificados.
- Puertos.
- Servidores.
- Tiempos de respuesta. (uptime.kuma.pet)

Añadir Prometheus después

Cuando tengas varios servicios o clientes, incorporaría Prometheus y Grafana para métricas, reglas y alertas. Prometheus es open-source, puede funcionar independientemente y utiliza Alertmanager para notificaciones. (prometheus.io)

El agente de reparación recibiría una alerta y seguiría esta secuencia:

```

Detectar
↓
Diagnosticar
↓
¿Incidente conocido?
├─ Sí → ejecutar runbook
└─ No → contener y escalar
↓
Validar
↓
Registrar

```

Acciones autorizadas durante tus vacaciones:

- Reiniciar un servicio.
- Reintentar un proceso.
- Restaurar la versión estable.
- Renovar un contenedor.
- Limpiar espacio temporal.
- Cambiar a la segunda nube.
- Informar al cliente.
- Generar un informe del incidente.

Acciones prohibidas:

- Reescribir arquitectura.
- Actualizar modelos sin evaluación.
- Incorporar dependencias desconocidas.
- Eliminar datos.
- Modificar contratos.
- Hacer cambios financieros irreversibles.
- Desplegar funciones nuevas.

7. Cómo evitar que el stack te consuma

Nada de Kubernetes al principio

Usaría:

- Docker Engine o Podman.
- Docker Compose.
- OpenTofu.
- Ansible.
- Imágenes OCI idénticas para local, Nosana y Akash.

No usaría Kubernetes hasta que un cliente lo exija o Compose sea demostrablemente insuficiente.

Congelar versiones

Crearía una versión de plataforma NativIQ cada doce meses:

```

NativIQ Platform 2027
NativIQ Platform 2028

```

Cada versión tendría:

- Modelos aprobados.
- Versiones exactas.

- Hashes.
- Contenedores.
- Procedimientos.
- Pruebas.
- Políticas de seguridad.
- Plan de migración.

No actualizaría producción cada vez que aparece un modelo nuevo.

Evaluación semestral

Dos veces al año:

1. Seleccionar tres modelos nuevos.
2. Ejecutar tu benchmark.
3. Compararlos con el modelo actual.
4. Migrar solamente si la mejora es material.
5. Mantener el modelo anterior como rollback.

Esto limita la persecución tecnológica a dos períodos breves por año.

8. Presupuesto realista

Inversión inicial incremental

Dependiendo de lo que ya poseas:

Compra	Estimación orientativa
Ampliación de RAM	USD 100-250
NVMe de 2 TB	USD 120-250
Dos discos de backup	USD 200-400
UPS apropiado	USD 250-700
Mejoras del Lenovo	USD 100-250
Total estimado	USD 500-1.500

Operación mensual inicial

Servicio	Presupuesto
GPU Nosana/Akash	USD 50-150
CPU/VPS secundarios	USD 10-40
Backup externo	USD 5-20
Dominio y correo	USD 10-30
Total	USD 75-240/mes

No intentaría eliminar todos los servicios pagos. Intentaría que cualquiera pueda sustituirse sin afectar el negocio.

Decisión final

Comprar ahora

- RAM suficiente.
- SSD suficiente.
- UPS.
- Dos backups.
- Mejoras mínimas del Lenovo.

Usar ahora

- OpenCode.
- Goose.
- PydanticAI.
- Forgejo.
- Uptime Kuma.
- Docker Compose.
- PostgreSQL.
- Nosana y Akash.

Incorporar con el primer cliente

- Zammad.
- OpenBao.
- Temporal.
- ERPNext.
- Prometheus y Grafana.
- OpenTofu y Ansible.

Comprar después

- RTX 3090 usada, cuando el gasto recurrente lo justifique.
- Un segundo nodo pequeño en otra ubicación, antes que una GPU enormemente cara.

No comprar en el futuro inmediato

- RTX 5090.
- Mac Studio.
- NAS grande.
- Threadripper.
- GPU de 96 GB.
- Rack.
- Cluster Kubernetes.

La infraestructura correcta para NativIQ no es la más potente. Es la que vos podés comprender, mantener y recuperar solo:

Dos redes de GPU, un nodo de control local, herramientas open-source portables, un stack congelado y agentes con autonomía estrictamente limitada.

Mauro

Intervención 14 · 2026-07-13 00:06:59Z

Yo pensaba usar para NativIQ mi computadora Dell Inspiron 16 Plus 7640 al principio. Puedo armar una PC mas potente más adelante, pero eso es lo máximo que haría para un emprendimiento unipersonal potenciado por agente AI-native.

ChatGPT - avance de trabajo

Voy a evaluar esa Dell como **máquina base de NativIQ**, separando lo que puede correr localmente, lo que conviene enviar a GPU en la nube y qué ampliaciones mínimas tendrían sentido antes de pensar en otra PC.

ChatGPT

Sí. **Esa es una decisión mucho más realista y coherente con NativIQ:**

Dell Inspiron 16 Plus 7640 como computadora principal desde el inicio; GPU alquilada bajo demanda; y, solamente si el negocio lo justifica, una única PC de escritorio potente en el futuro.

No necesitás una granja de servidores, una Mac Studio, un NAS empresarial ni varias workstations. Para una empresa unipersonal potenciada por agentes, todo eso aumentaría tu carga operativa y desviaría capital del verdadero negocio: construir sistemas AI-native.

El papel correcto de la Dell

La Dell debería ser tu:

- Estación de desarrollo.
- Centro de mando.
- Terminal para los agentes.
- Entorno de pruebas.
- Consola de despliegue.
- Lugar donde conservás código, configuraciones y documentación.
- Punto de acceso a las GPU remotas.
- Equipo para reuniones, ventas, demostraciones y administración.

No debería ser:

- Servidor público permanente.
- Servidor de producción de clientes.
- Nodo de inferencia grande 24 x 7.
- Única ubicación de backups.
- Máquina dedicada a entrenamientos prolongados.

La Dell 7640 admite oficialmente hasta **64 GB de memoria DDR5 en dos ranuras**. Según la configuración, también puede incorporar una RTX 4050 con 6 GB o una RTX 4060 con 8 GB de VRAM. Dell documenta además almacenamiento NVMe M.2 para esta familia. ([dell.com](https://www.dell.com))

Qué puede ejecutar localmente

Incluso sin depender de la GPU integrada o dedicada, puede ejecutar sin problemas razonables:

- VS Code.
- OpenCode.
- Qwen Code.
- Goose.
- Python.
- PydanticAI.
- Docker o Podman.
- PostgreSQL.
- pgvector para pruebas pequeñas.
- Redis.
- APIs de desarrollo.
- Uptime Kuma.
- OpenTofu y Ansible.
- Bases de datos de clientes en entornos simulados.
- Agentes conectados a modelos remotos.

También puede ejecutar localmente modelos pequeños:

- Clasificadores.
- Embeddings.
- Rerankers pequeños.
- Modelos de seguridad.
- Modelos generativos de aproximadamente 1B-7B cuantizados.

Si tu versión tiene RTX 4060 de 8 GB, puede servir para experimentar con modelos pequeños y algunas cuantizaciones de 7B. No intentaría forzar modelos de 27B o 32B dentro de esa GPU: la RTX 4050 tiene 6 GB y la RTX 4060, 8 GB, según las especificaciones oficiales de Dell. ([dell.com](https://www.dell.com))

Configuración que buscaría

Memoria

- **32 GB:** suficiente para comenzar.
- **64 GB:** configuración ideal y final para esta Dell.

Subiría a 64 GB cuando empieces a utilizar simultáneamente varios contenedores, bases de datos, entornos de desarrollo y agentes. No es necesario hacerlo antes de detectar presión real sobre la memoria.

Dell admite oficialmente 64 GB mediante dos módulos de 32 GB DDR5. ([dell.com](https://www.dell.com))

Almacenamiento

Objetivo:

- **2 TB NVMe interno**, si tu configuración lo permite y todavía no los tiene.
- Un SSD externo de 2-4 TB para snapshots y archivos grandes.
- Otro disco externo independiente para backups.

Dell documenta unidades NVMe M.2 2230 y 2280 en esta familia; antes de comprar una ampliación interna conviene verificar la configuración exacta mediante el Service Tag, porque el diseño y las opciones pueden variar según la versión. ([dell.com](https://www.dell.com))

Protección

Compraría antes que una GPU:

- Un UPS adecuado para notebook, router y Lenovo.
- Un disco de backup automático.
- Un segundo backup desconectado.
- Buena ventilación y limpieza periódica.
- Garantía extendida, si todavía está disponible y su costo es razonable.

Sistema operativo recomendado

Para comenzar utilizaría:

Windows 11 + WSL2

Windows como entorno principal y Ubuntu dentro de WSL2 para:

- Python.
- Git.
- Docker.
- Agentes.
- Scripts.
- Herramientas Linux.
- Automatización.
- Pruebas de contenedores.

Esto evita convertir la instalación del sistema operativo en otro proyecto y conserva compatibilidad con las herramientas de trabajo, reuniones y clientes.

Más adelante, la PC de escritorio podría utilizar Linux nativo como sistema principal de desarrollo e inferencia.

Arquitectura inicial

```
Dell Inspiron 16 Plus 7640
├── Desarrollo
├── OpenCode / Qwen Code
├── Goose
├── PydanticAI
├── Docker
├── PostgreSQL
├── Pruebas y documentación
├── GPU remota A
│   └── Modelos principales
├── GPU remota B
│   └── Respaldo y modelos grandes
└── Lenovo Linux
    ├── Monitoreo
    ├── Repositorios
    ├── Backups
    └── Watchdog
```

Akash permite desplegar contenedores con GPU, almacenamiento persistente y registros privados, por lo que una aplicación empaquetada correctamente puede trasladarse sin estar ligada al hardware de tu Dell. (akash.network)

Herramientas que instalaría en la Dell

Uso cotidiano

- **VS Code:** editor principal.
- **OpenCode:** agente principal de programación.
- **Qwen Code:** alternativa y comparación.
- **Goose:** agente operativo personal.
- **Git:** control de versiones.
- **Docker Compose:** entornos reproducibles.
- **Python + uv:** proyectos y dependencias.
- **PostgreSQL:** base estándar.
- **PydanticAI:** agentes productivos.
- **LiteLLM:** abstracción entre modelos.
- **Restic:** backups cifrados.
- **Tailscale o WireGuard:** acceso seguro.

En el Lenovo

- Forgejo.
- Uptime Kuma.
- Reverse proxy.
- Registro de despliegues.
- Backups.
- Monitoreo de las instancias remotas.

No instalaría inmediatamente ERPNext, Zammad, Temporal, Prometheus, Grafana y OpenBao todos juntos. Cada componente debe aparecer cuando exista una necesidad comercial concreta.

Una única PC potente en el futuro

Tu máximo de infraestructura propia sería razonable:

Una Dell portátil para movilidad y control + una PC de escritorio para carga pesada.

Cuando llegue ese momento, la PC debería ser modular y reparable:

- Linux.
- 128 GB de RAM.
- CPU de 12-16 núcleos.
- GPU NVIDIA con 24-32 GB de VRAM.
- SSD NVMe de 4 TB.
- Fuente de buena calidad.
- UPS.
- Sin componentes empresariales innecesarios.

Esa PC ejecutaría la carga diaria estable. Las redes externas continuarían utilizándose para:

- Modelos que no entren localmente.
- Picos de demanda.
- Redundancia.
- Validación con un segundo modelo.
- Continuidad si la PC falla.

Cuándo construirla

No por entusiasmo tecnológico. Solo cuando se cumplan criterios objetivos:

1. NativIQ ya tiene clientes pagos.
2. Gastás varios meses seguidos una suma importante en GPU.
3. Existe un modelo concreto que utilizás diariamente.
4. El hardware puede amortizarse en aproximadamente dos años.
5. Necesitás procesar información que no debe ir a nodos externos.
6. Administrar esa PC requiere menos trabajo que administrar las instancias remotas.

Hasta entonces, la Dell es suficiente.

Decisión final

Tu infraestructura realista sería:

Ahora

- Dell Inspiron 16 Plus 7640.
- Lenovo Linux como nodo auxiliar.
- Dos backups.
- UPS.
- Dos opciones de cómputo remoto.
- Stack open-source portable.

Después

- Una única PC de escritorio potente.

Nunca, salvo cambio radical del negocio

- Centro de datos propio.
- Cluster de varias GPU.
- Rack empresarial.
- Equipo de infraestructura.
- Renovación constante de hardware.
- Compra de cada generación nueva.

Eso mantiene a NativIQ alineada con su identidad:

Una empresa unipersonal que construye sistemas AI-native mediante agentes, no una empresa dedicada a mantener infraestructura de IA.

Mauro

Intervención 15 · 2026-07-13 01:16:37Z

Mi identidad será siempre la misma. Objetivamente, siento que con esta estructura estoy bien. ¿Tu que opinas?

ChatGPT

Sí. **Objetivamente, esta estructura está bien alineada con tu identidad, tu tolerancia al riesgo y tu objetivo económico.** No necesitas transformarla en una startup tradicional ni construir una infraestructura gigantesca para que NativIQ sea una empresa seria.

Tu estructura definitiva tiene coherencia interna:

- **Un único propietario y responsable:** vos.
- **Agentes como fuerza operativa**, no empleados.
- **Pocos clientes de alto valor**, en lugar de muchos clientes pequeños.
- **Dell Inspiron como centro de desarrollo y control.**
- **Cómputo remoto intercambiable** para modelos que no puedas ejecutar localmente.
- **Una única PC potente en el futuro**, solo cuando el uso y los ingresos la justifiquen.
- **Herramientas open-source y modelos reemplazables.**
- **Meta de suficiencia:** alrededor de USD 10.000 mensuales de ganancia, sin crecer indefinidamente.

Por qué considero que es una buena estructura

No estás intentando competir entrenando modelos, vendiendo hardware ni administrando centros de datos. Estás utilizando componentes existentes para construir soluciones empresariales de alto valor.

Eso te permite concentrarte en lo que realmente puede diferenciar a NativIQ:

- Comprender operaciones empresariales.
- Diseñar automatizaciones seguras.
- Integrar datos, modelos y herramientas.
- Construir agentes confiables.
- Evitar dependencia tecnológica.
- Mantener sistemas operativos con poca intervención humana.

La Dell es suficiente para desarrollar, programar, administrar agentes, ejecutar contenedores y controlar infraestructura remota. La GPU grande puede alquilarse solamente durante el tiempo necesario. Más adelante, una PC de escritorio puede absorber la carga estable y sensible.

Lo que debe permanecer estable

Tu identidad puede mantenerse durante toda la vida del emprendimiento:

Una empresa boutique unipersonal, aumentada por agentes AI-native, especializada en sistemas seguros, portables y útiles para operaciones empresariales.

No necesitas cambiar eso aunque cambien:

- Los modelos.
- Las GPU.
- Los proveedores.
- Los frameworks.
- Las redes de cómputo.
- Las interfaces.

La identidad debe permanecer; los componentes deben poder sustituirse.

Las cuatro condiciones para que funcione

1. Limitar la personalización

No deberías construir un sistema completamente diferente para cada cliente. Necesitas una plataforma común con módulos reutilizables:

- Autenticación.
- Auditoría.
- Soporte.
- Monitoreo.
- Gestión de agentes.
- RAG.
- Backups.
- Despliegue.
- Recuperación.

Después adaptas únicamente las reglas de negocio y las integraciones.

2. Limitar la cantidad de clientes

Tu objetivo no debería ser tener cincuenta clientes. Podría ser suficiente con tres, cuatro o cinco empresas que paguen correctamente.

Más clientes significan:

- Más excepciones.
- Más soporte.
- Más contratos.
- Más sistemas externos.
- Más riesgo de interrupciones.

La rentabilidad debe provenir del valor, no del volumen.

3. Dar autonomía controlada a los agentes

Los agentes deben poder ejecutar tareas rutinarias y reversibles, pero no improvisar decisiones críticas.

Durante tus vacaciones deberían poder:

- Responder consultas habituales.
- Ejecutar runbooks.
- Reiniciar servicios.
- Restaurar versiones estables.
- Clasificar incidentes.
- Informar al cliente.
- Contener problemas desconocidos.

Cuando no sepan resolver algo, deberían mantener el sistema en estado seguro y esperar tu regreso.

4. No perseguir cada novedad

Elegí un stack estable durante seis o doce meses. Evaluá nuevos modelos dos veces al año, no todas las semanas.

El hecho de que aparezca un modelo mejor no vuelve inútil al modelo anterior. Mientras el sistema cumpla sus objetivos de calidad, seguridad y costo, no existe obligación de migrar.

El riesgo que aceptás

Vos decidiste conscientemente ser el único punto humano de falla. Eso es válido porque también querés conservar:

- El control completo.
- La propiedad total.
- La totalidad del beneficio.
- La libertad para limitar el crecimiento.
- La capacidad de cerrar, pausar o cambiar de dirección sin socios.

La consecuencia es que NativIQ debe tener un **modo degradado seguro** cuando no estés disponible. No hace falta que los agentes puedan dirigir la empresa completamente; deben poder sostenerla temporalmente sin producir daños.

Mi conclusión objetiva

No veo una contradicción fundamental en tu estructura. Por el contrario, es más coherente que intentar levantar capital, contratar empleados y comprar infraestructura que no querés administrar.

Tu modelo no requiere máxima escala. Requiere:

- Especialización.
- Disciplina operativa.
- Tarifas altas.
- Sistemas reutilizables.
- Agentes confiables.
- Bajos costos fijos.
- Pocos clientes bien elegidos.
- Infraestructura reemplazable.

Dell ahora, una única PC potente después, cómputo externo como complemento y agentes como estructura operativa es una base suficiente y realista para la NativIQ que querés construir.

Mauro

Intervención 16 · 2026-07-13 01:26:50Z

La computadora Lenovo no la voy a usar porque es vieja (tiene lector de CD). Ahora me gustaría hablar a detalle que equipo se ocuparía de que cosas: la computadora Dell (hoy), la PC potente (en el futuro) y dónde delegaré el cómputo externo. Todo esto para tener la máxima confiabilidad.

ChatGPT - avance de trabajo

Voy a convertirlo en una arquitectura de responsabilidades: **Dell como puesto de mando, PC potente como nodo local de trabajo pesado y cómputo externo como capacidad elástica y de respaldo**. La confiabilidad vendrá de que ninguna de las tres capas dependa de las otras para recuperar el sistema.

ChatGPT

Sí. Sin la Lenovo, la arquitectura queda más limpia:

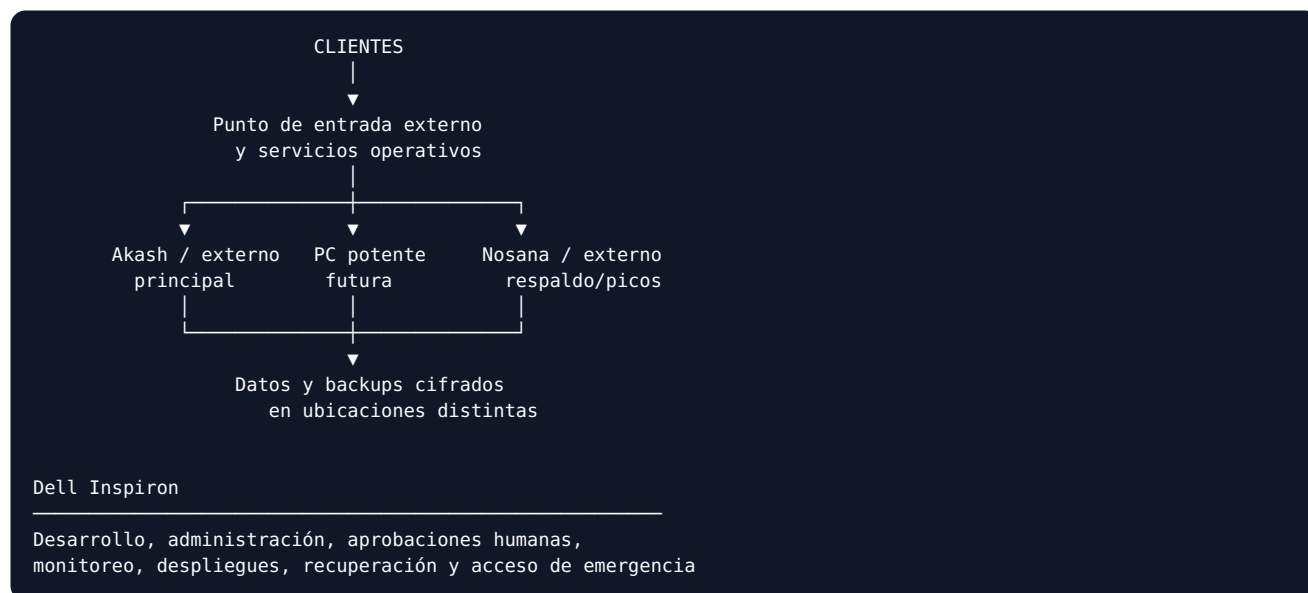
- **Dell Inspiron 16 Plus 7640:** puesto de mando y desarrollo.
- **PC potente futura:** nodo local de cómputo, pruebas pesadas y continuidad.

- **Cómputo externo:** producción, elasticidad, modelos grandes y respaldo geográfico.

La regla fundamental sería:

La Dell controla, la PC potente ejecuta la carga local estable y la nube aporta disponibilidad externa y capacidad variable. Ninguna contiene por sí sola todo lo necesario para operar NativIQ.

Arquitectura general



La Dell no debería estar en el camino crítico de producción. Si está apagada, viajando con vos o se rompe, los agentes y servicios ya desplegados deben continuar funcionando.

1. Dell Inspiron 16 Plus 7640

Función principal: puesto de mando

La Dell será tu computadora personal de trabajo y el lugar desde donde dirijís NativIQ.

Responsabilidades

- Desarrollo de sistemas y agentes.
- VS Code, OpenCode, Qwen Code y Goose.
- Diseño de arquitecturas.
- Revisión de código y pull requests.
- Ejecución de pruebas locales pequeñas.
- Construcción de contenedores.
- Preparación de despliegues.
- Administración de clientes.
- Atención manual de excepciones.
- Acceso a dashboards, tickets y registros.
- Autorización de operaciones críticas.
- Copia local cifrada de código y documentación.
- Recuperación de emergencia.

La familia Inspiron 16 Plus 7640 puede venir con distintas configuraciones, incluidas RTX 4050 de 6 GB o RTX 4060 de 8 GB. Las variantes con GPU dedicada admiten dos ranuras DDR5 y hasta 64 GB de RAM; Dell también documenta configuraciones NVMe de hasta 2 TB. Esto es suficiente como estación de desarrollo, aunque no como servidor de grandes modelos. ([dell.com](https://www.dell.com))

Qué puede ejecutar localmente

- Docker Compose.
- PostgreSQL de desarrollo.
- Redis para pruebas.
- pgvector pequeño.
- PydanticAI.
- APIs y microservicios.
- Modelos pequeños de clasificación.
- Embeddings y rerankers pequeños.
- Modelos generativos de 1B-7B cuantizados.
- Simulaciones completas con modelos remotos.
- Entornos de staging pequeños.

Qué no debe ejecutar

- Producción pública permanente.
- Base de datos principal de clientes.
- Inferencia 24 × 7.
- Modelos de 27B-120B como servicio.
- Monitoreo del que dependa toda NativIQ.
- Procesos que deban continuar durante tus vacaciones.
- Única copia de claves o repositorios.

Configuración recomendable

- **64 GB de RAM**, cuando el uso de contenedores lo justifique.
- **SSD interno de 2 TB**, si todavía no lo tiene.
- SSD externo de 2-4 TB para copias locales.
- Windows 11 + WSL2 inicialmente.
- UPS para notebook, router y módem.
- Dos llaves físicas FIDO2:
 - Una de uso diario.
 - Una de recuperación, guardada fuera de la vivienda.

Principio de confiabilidad

La Dell debe ser **reemplazable**.

Si mañana deja de funcionar, deberías poder tomar otra computadora, instalar Git, Docker y tus herramientas, recuperar las claves y volver a administrar NativIQ.

Por eso no guardarías nada exclusivamente en ella.

2. PC potente futura

Función principal: nodo local soberano

Esta PC aparecería cuando NativIQ ya tenga clientes, gasto sostenido de GPU y cargas previsibles.

No sería una workstation “para tener lo último”. Sería un nodo estable que ejecute el trabajo que utilizás todos los días.

Responsabilidades

- Inferencia local de modelos principales.
- Trabajo con información sensible.
- Desarrollo pesado.
- Integración y pruebas completas.
- CI privado.
- Ejecución de agentes internos.
- Generación de embeddings.
- Procesamiento documental.
- Refactorizaciones grandes.
- Staging similar a producción.
- Copia local de modelos aprobados.
- Réplica de servicios esenciales.
- Respaldo del cómputo externo.

Configuración por capacidades, no por marca

No elegiría hoy una GPU concreta para comprar dentro de varios años. Definiría requisitos:

Componente	Objetivo futuro
VRAM	24 GB mínimo; 32 GB preferible
RAM	128 GB
Almacenamiento	4 TB NVMe + segundo SSD
CPU	12-16 núcleos modernos
Sistema	Linux nativo
Red	2,5 GbE como mínimo
Fuente	Dimensionada con margen
Protección	UPS y encendido automático
Gabinete	Grande, ventilado y reparable

Modelos que ejecutaría

- Generalistas de aproximadamente 20B-35B.
- Modelos especializados en programación.
- Embeddings y rerankers.
- Modelos de seguridad.
- Modelos pequeños simultáneos.
- Modelos mayores con offloading, cuando resulte aceptable.

Los modelos de 80B-120B seguirían siendo externos salvo que el negocio justificara hardware específico.

Servicios que alojaría

- vLLM, llama.cpp u Ollama.
- Workers de PydanticAI.
- Runner de integración continua.
- Entorno de staging.
- Registro local de contenedores.
- Réplica de PostgreSQL.
- Copias de modelos y datasets.
- Uptime Kuma secundario.
- Backups cifrados.

Lo que no debe convertirse en la PC futura

No debería ser el único servidor productivo.

Si todo el soporte, facturación, agentes y sistemas de clientes se ejecutaran solamente allí, una falla eléctrica o de Internet en tu casa detendría a NativIQ.

Su papel correcto es:

Carga base local + réplica + procesamiento sensible.

No:

Único centro de datos.

3. Cómputo externo

El cómputo externo debería dividirse en tres funciones diferentes.

A. Servicios persistentes: Akash

Akash sería la primera opción para contenedores que necesiten mantenerse desplegados durante días o meses:

- API de entrada.
- LiteLLM o router de modelos.
- Agentes de soporte.
- Workers operativos.
- Uptime Kuma principal.
- Colas de trabajo.
- PostgreSQL principal inicialmente.
- Servicios web de NativIQ.
- Inferencia GPU relativamente estable.

Akash permite seleccionar proveedores que ofrecen CPU, GPU, memoria, almacenamiento persistente y conectividad. Su especificación contempla imágenes privadas, almacenamiento persistente, secretos e IP dedicadas, por lo que encaja mejor que una plataforma puramente batch para servicios prolongados. (akash.network)

Qué colocaría en Akash

- Servicios siempre encendidos.
- Endpoints públicos.

- Agentes de soporte y monitoreo.
- Inferencia que deba responder con baja latencia.
- Bases de datos solo con backups externos frecuentes.
- Servicios replicables a partir de contenedores.

Qué no colocaría exclusivamente allí

- Única copia de una base de datos.
- Claves maestras.
- Único repositorio.
- Único modelo disponible.
- Datos de clientes sin cifrado o minimización.
- Procesos cuya reconstrucción no esté automatizada.

B. GPU elástica y respaldo: Nosana

Nosana la utilizaría para:

- Inferencia por demanda.
- Picos de uso.
- Benchmarks.
- Evaluación de modelos nuevos.
- Procesamiento documental masivo.
- Generación de embeddings.
- Trabajos que puedan iniciarse, completarse y apagarse.
- Segundo endpoint de inferencia.
- Recuperación temporal si la PC potente o Akash fallan.

Nosana ofrece deployments, API, CLI y definiciones de trabajos basadas en contenedores para ejecutar cargas de IA sobre su mercado de GPU. (docs.nosana.com)

Papel exacto

Nosana no sería inicialmente el hogar permanente de toda la plataforma. Sería:

Capacidad elástica, segundo proveedor y respaldo GPU.

El sistema debería poder enviarle un trabajo con la misma imagen de contenedor utilizada en otro entorno.

C. Tercer camino: Golem o cualquier Linux compatible

Golem lo trataría como capacidad terciaria para:

- Procesamiento batch.
- Experimentos.
- Tareas paralelizables.
- Inferencia temporal.
- Situaciones donde las otras redes no estén disponibles.

Golem permite alquilar recursos GPU y CPU, ejecutar modelos mediante imágenes preparadas y administrar workers a través de APIs. Sin embargo, su flujo requiere componentes propios como Yagna y una integración más específica, por lo que no lo pondría como primera plataforma operativa de una empresa unipersonal. (docs.golem.network)

Más importante que comprometerte con Golem es mantener esto:

- Dockerfile estándar.
- Imagen OCI reproducible.
- Configuración por variables de entorno.
- Datos fuera del contenedor.
- Sin dependencias exclusivas del proveedor.

Así podrías desplegar en Golem, en otra red descentralizada o temporalmente en cualquier servidor Linux.

Por qué admitir un proveedor convencional de emergencia

Para que **ningún token** pueda detenerte, no deberías depender exclusivamente de redes tokenizadas.

La mejor resiliencia sería:

1. PC local.
2. Akash.
3. Nosana.
4. Plantilla lista para cualquier host Linux o GPU convencional.

El cuarto no sería tu proveedor principal ni una dependencia estratégica. Sería una salida de emergencia.

Usar temporalmente un proveedor centralizado no contradice tu premisa siempre que:

- Sea reemplazable.
- No contenga la única copia.
- No utilices funciones propietarias.
- Puedas abandonarlo sin modificar la aplicación.

4. Reparto definitivo de tareas

Tarea	Dell	PC futura	Externo
Programación diaria	Principal	Secundaria	No
Diseño de sistemas	Principal	No	Modelos remotos
Revisión y aprobación	Principal	No	Asistencia
Pruebas pequeñas	Principal	Secundaria	No
Pruebas pesadas	No	Principal	Respaldo
Inferencia 1B-7B	Sí	Sí	Opcional
Inferencia 20B-35B	Limitada	Principal	Respaldo
Inferencia 80B-120B	No	Limitada	Principal
Agentes 24 x 7	No	Secundaria	Principal
Soporte al cliente	Consola	Respaldo	Principal
Monitoreo	Consola	Secundario	Principal
Base de datos productiva	No	Réplica	Principal
Datos sensibles	Trabajo puntual	Principal	Solo con controles
Embeddings masivos	No	Principal	Picos
Backups	Copia local	Copia local fuerte	Copia externa

Tarea	Dell	PC futura	Externo
Producción pública	No	Respaldo	Principal
Continuidad en vacaciones	No necesaria	Sí	Sí

5. Dónde debe residir la información

La regla principal sería:

El cómputo puede ser desechable. El estado no.

Código

Tres copias:

1. Dell.
2. Dos repositorios remotos de operadores diferentes.
3. Backup Git cifrado.

Git ya permite tener repositorios completos en cada ubicación. Perder un portal no debe hacerte perder el proyecto.

Imágenes de contenedores

- Dos registries diferentes.
- Dockerfiles en el repositorio.
- Capacidad para reconstruirlas desde cero.
- Versiones fijadas mediante hash.

Modelos

- Manifiesto con nombre, licencia, versión y hash.
- Copia de los modelos productivos en la futura PC.
- Copia externa o posibilidad de reconstrucción.
- Nunca depender únicamente del nombre de un repositorio online.

Bases de datos

Inicialmente:

```

PostgreSQL principal externo
├── Backups cifrados frecuentes
├── Copia en segundo proveedor
└── Copia local periódica
  
```

Más adelante:

```

PostgreSQL principal externo
└── Réplica en la PC potente
  
```

No recomiendo multi-master ni clústeres complejos al comienzo. Para una empresa unipersonal, un primario y una réplica o backup verificado dan mejor equilibrio entre confiabilidad y mantenimiento.

Secretos

- Nunca dentro del código.
- Nunca solo en la Dell.
- Cifrados mediante SOPS/age o un sistema equivalente.
- Clave principal en hardware.
- Copia de recuperación sellada.
- Credenciales separadas por cliente.
- Tokens de corta duración para agentes.

6. Qué ocurre ante cada falla

La Dell se rompe

- Los agentes continúan externamente.
- La PC potente continúa procesando.
- Usás otra computadora.
- Recuperás repositorios, claves y entorno.
- No hay interrupción inmediata para clientes.

La PC potente se rompe

- El router envía inferencia a Akash o Nosana.
- Staging y trabajos pesados se ejecutan externamente.
- Producción sigue operativa.
- Reemplazás o reparás la PC sin urgencia extrema.

Se cae Internet en tu casa

- Los servicios externos siguen funcionando.
- Los clientes no dependen de tu conexión.
- Accedés temporalmente desde Internet móvil u otra ubicación.
- La PC local queda fuera hasta recuperar conexión.

Akash deja de funcionar

- La PC potente absorbe la carga compatible.
- Nosana levanta workers temporales.
- Los servicios se despliegan en el host Linux de emergencia.
- La base de datos se restaura desde la copia más reciente.

Nosana deja de funcionar

- No afecta la producción base.
- Perdés temporalmente capacidad de picos.
- Akash o la PC absorben las tareas.

Un token queda bloqueado

- Utilizás otra red.
- Pagás un proveedor mediante otro método.
- La PC local sigue disponible.

- Ninguna aplicación necesita ser reescrita.

Un modelo desaparece

- Conservás pesos o una cuantización aprobada.
- El router cambia a otro modelo compatible.
- Las evaluaciones verifican que el reemplazo cumpla el nivel mínimo.

Estás de vacaciones

- Soporte y monitoreo están externos.
- La PC futura puede ejecutar cargas locales estables.
- Los agentes resuelven incidentes conocidos.
- Las operaciones irreversibles quedan bloqueadas.
- Los casos extraordinarios se contienen hasta tu regreso.

7. Herramientas por equipo

Dell

- Windows 11 + WSL2.
- VS Code.
- OpenCode.
- Qwen Code.
- Goose.
- Git.
- Python + uv.
- Docker Compose.
- OpenTofu.
- Ansible.
- Restic.
- SOPS/age.
- Cliente WireGuard.
- Navegadores y herramientas comerciales.

PC futura

- Debian o Ubuntu LTS.
- Docker o Podman.
- NVIDIA Container Toolkit.
- vLLM.
- llama.cpp u Ollama.
- PostgreSQL réplica.
- Runner de CI.
- Uptime Kuma secundario.
- Restic.
- Agentes internos.
- Servicios de embeddings y reranking.

Externo

- LiteLLM o router equivalente.
- Servicios PydanticAI.
- API gateway.
- PostgreSQL principal inicial.
- Cola de trabajos.
- Uptime Kuma principal.
- Zammad cuando exista soporte real.
- Temporal cuando existan flujos largos.
- GPU workers desplegables en varias redes.

8. Qué no haría

Para mantener máxima confiabilidad sin convertirte en administrador de sistemas a tiempo completo:

- No Kubernetes inicialmente.
- No tres bases de datos activas.
- No clúster GPU local.
- No varias PC potentes.
- No servicios públicos alojados en la Dell.
- No almacenamiento crítico solo en tu casa.
- No agentes con acceso irrestricto.
- No actualizar modelos automáticamente.
- No depender de una blockchain.
- No usar servicios exclusivos de un proveedor.
- No mantener cinco redes externas activas.

La redundancia útil no es acumular tecnologías. Es tener **dos caminos simples y probados**.

Recomendación definitiva

Hoy

Dell

- Desarrollo.
- Administración.
- Pruebas locales.
- Aprobaciones.
- Copia de emergencia.

Externo

- Akash para servicios persistentes.
- Nosana para GPU bajo demanda.
- Backups en una ubicación independiente.
- Una plantilla OCI preparada para cualquier servidor Linux.

En el futuro

Dell

Continúa como puesto de mando portátil.

PC potente

- Inferencia local diaria.
- Procesamiento sensible.
- Staging.
- Réplica.
- Respaldo operativo.

Externo

- Producción pública.
- Modelos grandes.
- Picos.
- Redundancia geográfica.
- Continuidad cuando tu infraestructura local falle.

La arquitectura quedaría resumida así:

La Dell nunca es servidor. La PC potente nunca es el único servidor. La nube nunca posee el único estado. Los modelos nunca están ligados a un proveedor.

Esa es la estructura más confiable que considero realista para una NativIQ unipersonal.

Mauro

Intervención 17 · 2026-07-13 01:35:19Z

Perfecto ChatGPT. ¿Te parece cerrar este bloque? ¿O quieres agregar algo más?

ChatGPT

Sí, me parece buen momento para cerrar este bloque. La arquitectura quedó suficientemente definida y, sobre todo, **alineada con la identidad permanente de NativIQ.**

La conclusión consolidada sería:

NativIQ Systems será una empresa unipersonal de construcción de sistemas AI-native, operada por Mauro y asistida por agentes autónomos, con pocos clientes de alto valor, herramientas open-source reemplazables y una meta aproximada de USD 10.000 mensuales de beneficio.

La infraestructura quedará organizada así:

- **Dell Inspiron 16 Plus 7640:** desarrollo, dirección, aprobaciones, administración y recuperación.
- **PC potente futura:** inferencia local habitual, datos sensibles, pruebas pesadas, staging y respaldo operativo.
- **Cómputo externo:** producción pública, agentes 24 × 7, modelos grandes, picos y redundancia geográfica.
- **Akash:** servicios persistentes.

- **Nosana:** GPU elástica y procesamiento bajo demanda.
- **Host Linux alternativo:** salida de emergencia ante fallos de las redes anteriores.
- **Contenedores y datos portables:** ningún proveedor será indispensable.

Y las cuatro reglas centrales:

La Dell nunca será servidor de producción.
La futura PC nunca será el único servidor.
La nube nunca conservará la única copia del estado.
Ningún modelo, proveedor, red, token, repositorio o gobierno podrá detener por sí solo a NativIQ.

No agregaría ahora más herramientas ni más hardware. Hacerlo empezaría a introducir complejidad antes de que exista una necesidad real.

El próximo bloque lógico será diseñar la **organización de agentes de NativIQ**: qué agentes existirán, cuáles serán sus responsabilidades, niveles de autonomía, permisos, límites, supervisión y mecanismos de reemplazo.

Mauro

Intervención 18 · 2026-07-13 01:36:28Z

Perfecto. Entonces transcribe todo este chat a PDF y el próximo bloque lo definimos en otro chat.

Actualización: contingencia open-source ante una caída de OpenAI

Conversación incorporada el 5 de agosto de 2026 como continuación del bloque original.

Mauro

Intervención 19 · 2026-08-05 20:07:30Z

ChatGPT. Necesito que me recomiendes las mejores herramientas disponibles en el mercado que sean open-source para cubrir una posible caída de a las herramientas que ofrece la OpenAI:

- Codex
- Work
- Workspace Agents
- ChatGPT for Excel

ChatGPT

Intervención 20 · 2026-08-05

Sí. Para cubrir una caída de OpenAI, **no buscaría cuatro clones exactos**. Construiría una capa de contingencia donde cada función pueda continuar con herramientas y modelos intercambiables.

Actualmente, Codex cubre programación autónoma, agentes paralelos, pull requests, revisiones y automatizaciones; Work actúa durante horas sobre archivos y aplicaciones; Workspace Agents encapsula procesos repetibles con herramientas, conocimiento y disparadores; y ChatGPT for Excel modifica y analiza libros directamente. (openai.com)

Recomendación general

Producto de OpenAI	Sustituto open-source principal	Respaldo
Codex	OpenCode	Qwen Code + OpenHands
Work	Goose	Browser Use; Suna en evaluación
Workspace Agents	PydanticAI + Temporal	Langflow para diseño visual
ChatGPT for Excel	xlwings + Excel MCP Server	ONLYOFFICE + AI plugin
Capa común de modelos	LiteLLM	Ollama local + vLLM externo

1. Sustitución de Codex

Principal: OpenCode

OpenCode es la mejor alternativa cotidiana para NativIQ porque:

- Funciona en terminal, escritorio e IDE.
- Lee y modifica repositorios.
- Ejecuta comandos.
- Tiene agentes separados de planificación y construcción.
- Permite sesiones paralelas.
- No está vinculado a un único proveedor de modelos.
- Tiene licencia MIT. (opencode.ai)

Advertencia importante

OpenCode **no proporciona un sandbox real**: sus permisos sirven para solicitar confirmación, pero los comandos se ejecutan sobre el entorno donde lo instalaste. Por eso debe ejecutarse dentro de WSL, un Dev Container, una máquina virtual o un contenedor aislado cuando trabaje con repositorios o entradas no confiables. (github.com)

Uso recomendado

- Desarrollo diario.
- Corrección de fallos.
- Pruebas.
- Refactorizaciones.
- Creación de documentación.
- Investigación dentro del repositorio.
- Cambios aprobados por vos.

Respaldo inmediato: Qwen Code

Qwen Code debe estar instalado y configurado como reemplazo directo. Es Apache 2.0, funciona en terminal y dispone de herramientas, MCP y aplicaciones de escritorio para Windows, Linux y macOS. (github.com)

No usaría OpenCode y Qwen Code permanentemente en paralelo. Mantendría:

- OpenCode como principal.
- Qwen Code actualizado y probado como respaldo.
- Los dos apuntando al mismo gateway de modelos.

Agentes remotos: OpenHands Core

OpenHands resulta más apropiado cuando necesitás delegar issues completos, mantener agentes ejecutándose cuando la Dell está apagada o trabajar en entornos remotos. Puede instalarse localmente o en servidores; su núcleo y sus imágenes principales son MIT, aunque el directorio Enterprise es comercial y no debe confundirse con el núcleo abierto. (openhands.dev)

Decisión para Codex

OpenCode para el trabajo interactivo, Qwen Code como contingencia y OpenHands Core para tareas remotas o prolongadas.

2. Sustitución de ChatGPT Work

Principal: Goose

Goose es mi primera elección para NativIQ.

Es un agente generalista que trabaja sobre:

- Investigación.
- Escritura.
- Archivos.
- Automatización.
- Datos.
- Código.
- Herramientas externas.
- Tareas con varios pasos.

Dispone de aplicación de escritorio, CLI y API; acepta múltiples proveedores y extensiones MCP; tiene licencia Apache 2.0 y forma parte de la Agentic AI Foundation de la Linux Foundation. Esta gobernanza es especialmente conveniente para tu premisa de evitar que una única compañía controle toda la capa operativa. (github.com)

Goose también permite crear distribuciones personalizadas y recetas YAML con subagentes, por lo que podrías preparar una edición interna de **NativIQ Goose** con proveedores, instrucciones y herramientas ya configurados. (github.com)

Uso recomendado

- Investigación de mercado.
- Preparación de propuestas.
- Análisis documental.
- Generación de informes.
- Operaciones administrativas.
- Ejecución de runbooks.
- Clasificación y análisis de tickets.
- Gestión de archivos locales.
- Apoyo contable, sin autoridad financiera irreversible.

Complemento: Browser Use

ChatGPT Work puede actuar sobre aplicaciones conectadas. Para cubrir la parte de navegación web, utilizaría Browser Use, una biblioteca MIT para agentes que operan navegadores y que puede trabajar con modelos locales o remotos. (github.com)

No debe recibir acceso indiscriminado a sesiones bancarias, paneles administrativos o credenciales maestras. Debe trabajar con perfiles separados y permisos mínimos.

Suna/Kortix: cercano, pero no lo adoptaría todavía como núcleo

Suna se presenta explícitamente como alternativa a ChatGPT Work y como centro de mando para agentes. Sin embargo, su arquitectura actual promueve una máquina compartida donde agentes, archivos, bases de datos y secretos conviven. Eso simplifica el contexto, pero crea un dominio de seguridad demasiado amplio para NativIQ. (github.com)

Lo probaría con datos sintéticos, pero no lo convertiría todavía en el sistema operativo central de tu empresa.

Decisión para Work

Goose como trabajador generalista, Browser Use como capacidad web y Suna solamente como plataforma experimental.

3. Sustitución de Workspace Agents

Aquí no elegiría una única aplicación. Necesitás separar tres funciones:

1. Diseñar el agente.
2. Ejecutarlo de forma confiable.
3. Recuperarlo después de fallos.

Diseño visual: Langflow

Langflow es MIT y permite construir visualmente agentes, flujos y servidores MCP, conectando modelos, herramientas, bases vectoriales y componentes personalizados. (github.com)

Lo usaría para:

- Prototipar agentes.
- Mostrar flujos a clientes.
- Experimentar con RAG.
- Diseñar MCP.
- Validar procesos antes de programarlos.

No dependería de sus archivos internos como única definición del agente. Cada flujo aprobado debe exportarse, versionarse y documentarse.

Runtime productivo: PydanticAI

Los agentes productivos de NativIQ deberían programarse en PydanticAI porque:

- Es model-agnostic.
- Tiene licencia MIT.
- Utiliza tipos y validaciones estructuradas.

- Permite herramientas controladas.
- Se integra con modelos locales, LiteLLM, Ollama y numerosos proveedores.
- Es adecuado para software que vos vas a mantener durante años. (pydantic.dev)

Aquí construiría los futuros agentes de:

- Soporte.
- Diagnóstico.
- Documentación.
- Administración.
- Control de calidad.
- Seguridad.
- Facturación asistida.
- Recuperación ante incidentes.

Continuidad y tareas prolongadas: Temporal

Temporal es la capa que permite que un proceso:

- Sobreviva reinicios.
- Espere horas o días.
- Reintente operaciones.
- Continúe después de una interrupción.
- Mantenga su estado.
- Escale a revisión humana.

El servidor es MIT y está diseñado para ejecutar workflows de forma duradera, aplicando recuperación y reintentos frente a fallos intermitentes. (temporal.io)

No hace falta instalarlo para el primer prototipo. Se incorpora cuando un agente deba operar 24 x 7 o continuar durante tus vacaciones.

Por qué no elegiría Dify como fundamento estratégico

Dify tiene una interfaz muy completa para agentes, RAG, herramientas y observabilidad. Sin embargo, utiliza una licencia Apache modificada que:

- Restringe ciertos usos multi-tenant.
- Restringe retirar marca del frontend.
- Permite que el productor modifique la licencia.
- No es equivalente a Apache 2.0 puro. (github.com)

Podría servir como prototipo interno, pero no lo pondría debajo de los sistemas vendidos por NativIQ.

Decisión para Workspace Agents

Langflow para diseñar; PydanticAI para construir; Temporal para garantizar continuidad.

4. Sustitución de ChatGPT for Excel

Esta es la categoría donde **no existe todavía un reemplazo open-source único con la misma integración y pulido**. Conviene separar dos escenarios.

Escenario A: conservar Microsoft Excel

xlwings

xlwings permite controlar Excel desde Python, crear scripts, macros, funciones personalizadas y add-ins. Su núcleo es BSD-3-Clause y se ejecuta localmente sobre Windows o macOS con Excel instalado. (xlwings.org)

Excel MCP Server

excel-mcp-server permite que Goose, OpenCode u otro cliente MCP:

- Lea celdas.
- Escriba valores.
- Escriba fórmulas.
- Cree hojas y tablas.
- Copie hojas.
- Aplique formatos.
- Edite el libro abierto en Windows.

Tiene licencia MIT y soporta XLSX, XLSM, XLTX y XLTM. (github.com)

Arquitectura

```
Excel
|
|-- xlwings ----- Python, análisis y automatización
|
|-- Excel MCP Server - Goose/OpenCode/agente
|                       |
|                       |-- Modelo local o externo
```

Esta sería mi opción principal porque conservas compatibilidad con los archivos de clientes que usan Microsoft Excel.

Escenario B: independencia completa de Microsoft

ONLYOFFICE Desktop Editors + AI plugin

ONLYOFFICE Desktop Editors es una suite abierta bajo AGPLv3, compatible con formatos de Microsoft Office. Su plugin de IA acepta tanto servicios externos como Ollama local, y sus funciones de hojas de cálculo incluyen generación de fórmulas, limpieza, análisis, identificación de tendencias y pronósticos. (github.com)

Es la mejor contingencia si Excel o el add-in de OpenAI dejan de estar disponibles, pero no garantiza compatibilidad perfecta con:

- VBA complejo.
- Power Query.
- Power Pivot.

- Complementos propietarios.
- Libros financieros muy avanzados.

Decisión para Excel

**xlwings + Excel MCP Server como sustituto operativo dentro de Excel;
ONLYOFFICE como contingencia completa fuera del ecosistema Microsoft.**

5. Capa que evita depender nuevamente de otro proveedor

Todas estas herramientas deben conectarse a una interfaz común.

LiteLLM

LiteLLM proporciona una API unificada para más de cien proveedores y despliegues, con routing, seguimiento, balanceo y fallbacks. Su núcleo OSS puede autohospedarse, aunque determinadas capacidades empresariales tienen licencia comercial. (litellm.ai)

Ejemplo conceptual:

```
OpenCode
Goose
PydanticAI
Excel Agent
  |
  v
  LiteLLM
    |
    |-- Modelo OpenAI
    |-- Modelo open-weight remoto A
    |-- Modelo open-weight remoto B
    `-- Modelo local
```

Así, una caída de OpenAI no requiere reconfigurar cada aplicación. Solo cambia el routing.

Inferencia local y externa

- **Ollama:** adecuado para la Dell, pruebas locales y futura PC; su núcleo usa licencia MIT. (github.com)
- **vLLM:** adecuado para GPU externa o futura PC potente, con API compatible con OpenAI y licencia Apache 2.0. (github.com)

Mantendría al menos dos familias diferentes de modelos open-weight detrás de LiteLLM. Cambiar el agente y cambiar el modelo deben ser dos operaciones independientes.

Stack definitivo para NativIQ

Instalar ahora en la Dell

- **OpenCode** - desarrollo diario.
- **Qwen Code** - respaldo de programación.
- **Goose** - trabajo general y operaciones.
- **Browser Use** - automatización web acotada.
- **Langflow** - diseño y experimentación.
- **PydanticAI** - construcción de agentes productivos.
- **LiteLLM** - routing y contingencia.
- **Ollama** - modelos pequeños locales.
- **xlwings + Excel MCP Server** - automatización de Excel.
- **Docker Compose** - aislamiento y portabilidad.

Incorporar cuando existan procesos 24 x 7

- **Temporal** - ejecución durable.
- **OpenHands Core** - agentes remotos de ingeniería.
- **vLLM** - inferencia en GPU externa o PC potente.
- **ONLYOFFICE** - contingencia integral de oficina.

No convertir en núcleo por ahora

- Dify, por su licencia modificada.
- Suna/Kortix, por su dominio de permisos y secretos demasiado amplio.
- OpenClaw, por su superficie de ataque y por estar orientado a asistentes personales con acceso persistente a canales y herramientas.
- Complementos pequeños de Excel sin comunidad o mantenimiento demostrados.

Veredicto

La combinación que considero más sólida es:

Codex	-> OpenCode + Qwen Code + OpenHands
Work	-> Goose + Browser Use
Workspace Agents	-> Langflow + PydanticAI + Temporal
ChatGPT for Excel	-> xlwings + Excel MCP Server + ONLYOFFICE
Modelos	-> LiteLLM + Ollama/vLLM

Esto no significa que todas las herramientas deban operar simultáneamente. La confiabilidad se obtiene mediante:

- Una herramienta principal.
- Una alternativa ya configurada.
- Flujos exportados y versionados.
- Contenedores reproducibles.
- Dos familias de modelos.
- Dos ubicaciones de inferencia.

- Pruebas periódicas de conmutación.

Para NativIQ, **OpenAI puede seguir siendo una opción de alto rendimiento, pero nunca el lugar exclusivo donde residen tus agentes, workflows, datos o conocimiento operativo.**

¿Querés que deje programada una revisión semestral de este stack open-source para detectar reemplazos, cambios de licencia y proyectos abandonados?

Mauro

Intervención 21 · 2026-08-05 20:17:21Z

¿Podés agregar esto a la transcripción en PDF?

Actualización: modelos frontier abiertos y estrategia cloud-first

Conversación incorporada el 7 de agosto de 2026 como continuación del bloque sobre contingencia, portabilidad y hardware de NativIQ Systems.

Mauro

Intervención 22 · 2026-08-06 21:22:58Z

ChatGPT. ¿Por qué no me sugeriste herramientas como los modelos abiertos KIMI K3 y GLM 5.2?

ChatGPT

Intervención 23 · 2026-08-06

Sí: **GLM-5.2 debería haber aparecido en mis recomendaciones, y Kimi K3 debería haber aparecido en la actualización del 5 de agosto.**

La explicación objetiva tiene tres partes.

1. Kimi K3 todavía no existía en la conversación original

Gran parte del análisis inicial lo hicimos el **12 de julio de 2026**. Kimi K3 fue presentado oficialmente el **16 de julio de 2026**, cuatro días después. Por eso no podía formar parte de aquella primera selección. Moonshot lo describe como un modelo open-weight multimodal de 2,8 billones de parámetros, 104.000 millones activos y un millón de tokens de contexto. (moonshot.ai)

Sin embargo, cuando actualizamos la comparativa el **5 de agosto**, ya estaba disponible y debería haberlo incluido.

2. GLM-5.2 sí estaba disponible y fue una omisión

GLM-5.2 fue publicado el **16-17 de junio de 2026**, antes de nuestra conversación. Tiene licencia MIT, pesos descargables, contexto de un millón de tokens y está orientado específicamente a programación, agentes y tareas de larga duración. (huggingface.co)

Por lo tanto, en el caso de GLM-5.2, no hay una justificación temporal: **simplemente lo omití y no debí hacerlo.**

3. Mezclé dos categorías distintas

En la respuesta del 5 de agosto me concentré principalmente en las **herramientas de ejecución**:

- OpenCode.

- Goose.
- PydanticAI.
- Temporal.
- Excel MCP Server.
- LiteLLM.

Kimi K3 y GLM-5.2 no reemplazan directamente esas herramientas. Son los **modelos que deberían funcionar detrás de ellas**.

Por ejemplo:

OpenCode	-> herramienta/agente de programación
GLM-5.2	-> modelo que razona y escribe el código
Goose	-> entorno de trabajo general
Kimi K3	-> modelo que investiga, analiza archivos y usa herramientas
PydanticAI	-> framework para construir agentes
Kimi / GLM	-> inteligencia que impulsa esos agentes

Esa separación es técnicamente correcta, pero mi respuesta quedó incompleta porque mencioné LiteLLM, Ollama y vLLM sin especificar cuáles debían ser los **modelos abiertos de primera línea**.

Qué lugar deberían ocupar

GLM-5.2: primera opción para coding y agentes prolongados

GLM-5.2 debería estar entre los principales modelos de contingencia para:

- OpenCode.
- Qwen Code.
- OpenHands.
- Corrección de fallos.
- Refactorizaciones grandes.
- Navegación de repositorios.
- Procesos técnicos de larga duración.
- Agentes que ejecutan herramientas.

Z.ai declara 753B parámetros, un millón de tokens de contexto y licencia MIT. Su documentación destaca mejoras en tareas de ingeniería de larga duración, uso de terminal y programación agéntica. Los benchmarks publicados por el propio desarrollador lo colocan cerca de modelos cerrados avanzados en varias evaluaciones, aunque esas cifras deben tratarse como resultados del proveedor y no como una comparación completamente neutral. (huggingface.co)

Para tu premisa de soberanía, su licencia MIT es especialmente atractiva:

GLM-5.2 es jurídicamente más abierto y sencillo de incorporar que muchos modelos con licencias personalizadas.

Kimi K3: primera opción para Work y tareas multimodales

Kimi K3 encaja especialmente bien como respaldo de:

- ChatGPT Work.
- Investigación profunda.
- Análisis de documentos.
- Uso de navegador y aplicaciones.
- Hojas de cálculo.
- Generación de dashboards.
- Imágenes y video.
- Agentes de conocimiento.
- Programación de larga duración.

Es multimodal de forma nativa y Moonshot publica resultados muy competitivos en programación, tool calling, tareas de oficina, sistemas operativos y hojas de cálculo. En su propia evaluación obtiene, por ejemplo, 63,3 en OfficeQA Pro y 34,8 en SpreadsheetBench 2. (huggingface.co)

Sin embargo, Kimi K3 no usa MIT ni Apache 2.0. Utiliza una licencia propia que concede derechos amplios de uso, modificación y comercialización, pero introduce condiciones para negocios de Model-as-a-Service con ingresos superiores a USD 20 millones anuales y obligaciones de marca a escalas muy grandes. Estas restricciones no afectarían actualmente a NativIQ, pero significan que es **open-weight con licencia personalizada**, no una alternativa tan jurídicamente limpia como GLM-5.2. (huggingface.co)

El problema práctico: son modelos gigantescos

Esta fue otra razón por la cual prioricé Qwen y modelos menores, aunque debí explicitarlo.

Modelo	Parámetros totales	Parámetros activos	Uso realista
Kimi K3	2,8T	104B	Infraestructura multi-GPU externa
GLM-5.2	753B	No es un modelo para GPU doméstica	Infraestructura multi-GPU externa
Qwen pequeño/ mediano	Mucho menor	Mucho menor	Dell limitada o futura PC

Modelo	Parámetros totales	Parámetros activos	Uso realista
gpt-oss-20B	Aproximadamente 20B	Reducidos por MoE	PC potente o GPU alquilada accesible

Kimi K3 utiliza cuantización nativa MXFP4, pero sus 2,8 billones de parámetros siguen haciendo inviable ejecutarlo en tu Dell y poco realista incluso en una única PC futura. GLM-5.2 también está pensado para servidores con mucha memoria y frameworks como vLLM, SGLang o KTransformers. (huggingface.co)

Por eso deben ser:

- **Modelos frontier abiertos alquilados bajo demanda.**
- No modelos básicos para todas las tareas.
- No dependencias exclusivas.
- No necesariamente ejecutados mediante las APIs oficiales de Moonshot o Z.ai.

Para cumplir tu premisa, lo ideal es desplegar sus pesos en proveedores de cómputo independientes, cuando sea económicamente viable, y no depender únicamente de las APIs de sus creadores.

Stack corregido para NativIQ

Función	Herramienta	Modelo principal abierto
Desarrollo interactivo	OpenCode	GLM-5.2
Agentes de ingeniería prolongados	OpenHands	GLM-5.2
Trabajo general y documental	Goose	Kimi K3
Investigación multimodal	Goose + Browser Use	Kimi K3
Agentes productivos	PydanticAI	Kimi K3 o GLM-5.2 según tarea
Excel y hojas de cálculo	Excel MCP/xlwings	Kimi K3
Tareas locales económicas	OpenCode/Goose	Qwen de menor tamaño
Verificación independiente	PydanticAI	Otra familia distinta
Routing y failover	LiteLLM	Todos los anteriores

Jerarquía que recomiendo ahora

1. **GLM-5.2** como modelo abierto principal de programación y agentes técnicos.
2. **Kimi K3** como modelo abierto principal para conocimiento, multimodalidad, oficina y trabajo agéntico general.
3. **Qwen de tamaño medio** como modelo práctico y económico que pueda ejecutarse en hardware más accesible.
4. **Un modelo pequeño independiente** como verificador, clasificador o fallback local.

Veredicto honesto

No los omití porque fueran malos ni porque quisiera favorecer modelos de OpenAI.

- **Kimi K3 no existía todavía el 12 de julio.**
- **GLM-5.2 sí existía y debería haberlo incluido.**
- **El 5 de agosto debería haber incluido a ambos en la capa de modelos.**
- Me concentré demasiado en las aplicaciones y frameworks, y no actualicé suficientemente la selección de modelos frontier abiertos.

La recomendación corregida es:

OpenCode + GLM-5.2 para sustituir Codex; Goose + Kimi K3 para sustituir Work; PydanticAI/Temporal con ambos modelos para Workspace Agents; y Kimi K3 detrás de Excel MCP para tareas de hojas de cálculo.

Qwen sigue siendo importante, pero ya no debería presentarse como la única familia central del ecosistema abierto de NativIQ.

Mauro

Intervención 24 · 2026-08-06 21:32:41Z

Excelente. Lo bueno es que con estas herramientas puedo hacer switch entre modelos. Con respecto a los modelos frontier abiertos: me importa no tener que pagar por usarlos más que tenerlos instalados en mi propio hardware (que el hardware es de lo más caro y difícil de conseguir). Porque con una buena conexión a internet puedo usar modelos de frontera en la nube que siempre serán más potentes que los modelos locales. Además, si salen modelos abiertos cada vez mejores, no quedo estancado porque no tengo que comprar equipos nuevos y más costosos.

ChatGPT

Intervención 25 · 2026-08-06

Sí, esa estrategia encaja mejor con NativIQ. La única corrección importante es distinguir **modelo abierto** de **cómputo gratuito**: podés evitar licencias cerradas y hardware propio, pero la inferencia en la nube seguirá teniendo un costo operativo.

Tu razonamiento es sólido y encaja mejor con la naturaleza unipersonal de NativIQ. La estrategia adecuada para vos es:

Cloud-first, open-model y multi-provider; no hardware-first.

Solo corregiría una frase: los modelos en la nube no serán necesariamente “más inteligentes” por estar en la nube. La nube te permite acceder a **modelos mucho más grandes**, mejores configuraciones de inferencia y varias GPU que serían inviables en tu casa. Kimi K3, por ejemplo, tiene 2,8 billones de parámetros y está concebido para

tareas frontier multimodales y prolongadas; GLM-5.2 también está orientado a agentes de largo horizonte y un contexto de un millón de tokens. (moonshot.ai)

La precisión económica importante

Un modelo abierto puede evitarte:

- Una suscripción obligatoria al creador.
- La dependencia de su API oficial.
- Restricciones artificiales por cantidad de mensajes.
- Que el creador sea el único que pueda ofrecerlo.
- Tener que abandonar el modelo si retiran el servicio oficial.

Pero **no elimina el costo de inferencia**. Al utilizarlo en la nube pagarás GPU, memoria, almacenamiento, red o una tarifa por token. Redes como Akash publican disponibilidad y precios horarios de GPU, mientras Nosana permite desplegar endpoints compatibles con OpenAI mediante vLLM sobre su mercado de cómputo. (akash.network)

La diferencia estratégica es esta:

Modelo cerrado por API
Pagás el modelo + el cómputo + el margen del proveedor
y solo el creador controla la oferta.

Modelo abierto en nube
El modelo es portable.
Pagás principalmente el cómputo y podés cambiar
quién lo ejecuta.

Por eso, para tu premisa, **lo importante no es que la inferencia sea gratuita; es que el mercado de inferencia sea sustituible.**

La arquitectura correcta para NativIQ

1. Tu Dell será el plano de control

La Dell ejecutará:

- OpenCode.
- Goose.
- PydanticAI.
- LiteLLM.
- Configuraciones y prompts.
- Pruebas locales.
- Repositorios.
- Administración de despliegues.
- Un modelo pequeño de emergencia.

No ejecutará normalmente Kimi K3 o GLM-5.2.

2. La inteligencia pesada estará en la nube

Podrás enrutar las tareas según su naturaleza:

Tarea	Modelo orientativo
Código y reparación compleja	GLM-5.2
Trabajo documental y multimodal	Kimi K3
Tareas rutinarias económicas	Qwen de menor tamaño
Clasificación y seguridad	Modelos pequeños
Revisión independiente	Familia distinta del generador

Los modelos grandes se encenderían solo cuando haya trabajo, en lugar de mantener una GPU costosa ociosa.

3. LiteLLM será la capa de conmutación

Tus herramientas no deberían conocer directamente a Kimi, GLM, Qwen, Akash o Nosana. Deberían llamar nombres internos estables:

```
nativiq-coding-frontier
nativiq-work-frontier
nativiq-fast
nativiq-verifier
nativiq-emergency
```

LiteLLM puede realizar routing, reintentos, balanceo y fallbacks entre múltiples despliegues y proveedores. Así, una caída de un endpoint puede redirigir la llamada a otro modelo o proveedor sin modificar OpenCode, Goose o tus agentes productivos. (docs.litellm.ai)

Ejemplo:

```
OpenCode
|
v
nativiq-coding-frontier
|
|-- GLM-5.2 en proveedor A
|-- GLM-5.2 en proveedor B
|-- Kimi K3 como sustituto temporal
`-- Qwen como fallback económico
```

Tu mayor ventaja: no quedar congelado en una generación

Comprar una GPU implica fijar varias restricciones durante años:

- Cantidad de VRAM.
- Arquitectura de GPU.

- Consumo eléctrico.
- Capacidad máxima de los modelos.
- Compatibilidad del software.
- Tiempo de amortización.

Con cómputo remoto podés pasar de una GPU de consumo a varias GPU de centro de datos sin cambiar tu computadora. Akash funciona como mercado donde distintos proveedores compiten por CPU, GPU, memoria, almacenamiento y conectividad; esa diversidad encaja mejor con tu necesidad de reemplazabilidad. (akash.network)

Cuando aparezca un nuevo modelo abierto, tu procedimiento sería:

1. Desplegarlo temporalmente.
2. Ejecutar el benchmark de NativIQ.
3. Comparar calidad, costo y latencia.
4. Incorporarlo como candidato.
5. Mantener el anterior como rollback.
6. Cambiar el alias del router si el nuevo demuestra una mejora material.

No cambiarías toda la plataforma ni comprarías nuevo hardware.

Pero debe ser cloud-first, no cloud-only

Una dependencia exclusiva de la nube también sería contraria a tu premisa.

Conservaría en la Dell un modelo pequeño capaz de cubrir un **modo degradado**:

- Leer documentación.
- Clasificar incidentes.
- Preparar mensajes.
- Consultar runbooks.
- Analizar logs sencillos.
- Mantener la administración básica.

No tendría la capacidad de Kimi K3 o GLM-5.2, pero te permitiría continuar trabajando si todas las redes externas estuvieran temporalmente inaccesibles.

La jerarquía quedaría así:

1. Modelo frontier abierto en proveedor A
2. Mismo modelo o equivalente en proveedor B
3. Otra familia abierta en proveedor C
4. Modelo pequeño local en la Dell
5. Modelo cerrado opcional de emergencia

El último nivel no sería estructural. Sería solamente una vía adicional.

Cómo evitar sustituir una dependencia por otra

No bastaría con usar Kimi K3 en la API oficial de Moonshot o GLM-5.2 exclusivamente en la API de Z.ai. Eso seguiría creando dependencia del creador.

Para mantener soberanía:

- Guardar la versión y licencia de cada modelo aprobado.
- Mantener hashes de sus archivos.
- Usar imágenes OCI reproducibles.
- Servirlos mediante interfaces estándar como vLLM.
- Tener al menos dos proveedores posibles.
- Separar datos y almacenamiento del proveedor de GPU.
- Evitar funciones propietarias de una API concreta.
- Diseñar todas las herramientas contra endpoints compatibles con OpenAI.

GLM-5.2 puede servirse mediante vLLM y exponer una API compatible con OpenAI, lo que simplifica trasladarlo entre infraestructuras; además, sus pesos se publican bajo licencia MIT. (huggingface.co)

Kimi K3 también dispone de instrucciones para inferencia y uso mediante proveedores, aunque utiliza una licencia propia y no MIT; antes de incorporarlo comercialmente habrá que conservar y revisar expresamente esa licencia. (huggingface.co)

Mi conclusión objetiva

Tu decisión correcta es:

No comprar hardware para perseguir modelos frontier. Comprar únicamente una buena computadora de trabajo y alquilar el cómputo que cada modelo necesite.

Eso te proporciona:

- Acceso continuo a modelos mayores.
- Menor inversión inicial.
- Cero obligación de amortizar una GPU.
- Menor mantenimiento.
- Cambio rápido entre familias.
- Mejor adaptación a nuevas generaciones.
- Capacidad de escalar por horas.
- Menor riesgo de obsolescencia.

La futura PC potente deja de ser una necesidad y pasa a ser una compra opcional, justificable solamente para privacidad, carga base intensiva o contingencia. El núcleo de NativIQ debería ser **el router, los agentes, las evaluaciones, los datos y la portabilidad**, no el hardware donde se ejecuta cada modelo.

Mauro

Intervención 26 · 2026-08-06 21:36:23Z

No entiendo. ¿Por qué tengo que pagar si el modelo es abierto?

ChatGPT

Intervención 27 · 2026-08-06

Porque **“modelo abierto” no significa “cómputo gratis”**.

El modelo es el software: sus pesos pueden descargarse, ejecutarse y, según la licencia, modificarse o usarse comercialmente sin pagar una suscripción al creador.

Pero para hacerlo funcionar necesitás recursos físicos:

- GPU y memoria.
- Electricidad.
- Servidores.
- Refrigeración.
- Almacenamiento.
- Conexión de red.
- Mantenimiento.

Es parecido a Linux: podés descargarlo gratis, pero la computadora o el servidor donde lo instalás cuestan dinero.

Las tres posibilidades

1. Lo ejecutás en tu propia computadora

No pagás por cada mensaje ni por tokens, pero ya pagaste:

- La GPU.
- La PC.
- La electricidad.
- El mantenimiento.

Para un modelo pequeño puede bastar tu Dell. Para Kimi K3 o GLM-5.2, el hardware necesario sería demasiado grande y caro.

2. Alquilás una GPU en la nube

No pagás por el modelo abierto. Pagás al dueño del servidor por usar:

- Sus GPU.
- Su RAM.
- Su energía.
- Su infraestructura.

Por ejemplo:

Modelo abierto: USD 0 de licencia
GPU alquilada durante una hora: tiene costo

3. Usás una API que ofrece el modelo

El proveedor ya instaló y administra el modelo. Te cobra por tokens o por uso porque está pagando el hardware y añade su margen comercial.

En este caso, aunque el modelo sea abierto, **el servicio de API no es gratuito necesariamente**.

Ejemplo sencillo

Supongamos que GLM-5.2 puede descargarse sin pagar licencia.

Podés:

- Descargarlo gratis.
- Guardarlo gratis si ya tenés almacenamiento.
- Estudiarlo o modificarlo según su licencia.

Pero para generar una respuesta necesitás decenas o cientos de gigabytes de memoria y varias GPU. Alguien tiene que pagar esas máquinas.

La diferencia frente a un modelo cerrado es importante:

Modelo cerrado	Modelo abierto
Solo el creador puede servirlo	Cualquier proveedor compatible puede servirlo
No podés descargar los pesos	Podés conservar los pesos
Si retiran la API, lo perdés	Podés moverlo a otro servidor
El precio lo fija un único proveedor	Varios proveedores pueden competir
Dependés de sus cambios	Podés congelar una versión

Por eso el beneficio principal no es necesariamente usarlo gratuitamente. Es **no estar obligado a pagarle siempre a la misma empresa**.

En tu caso

Para NativIQ, la estrategia económica sería:

- No comprar el hardware gigantesco.
- Usar modelos abiertos en proveedores competitivos.
- Cambiar de proveedor si suben los precios.
- Apagar las GPU cuando no las necesitás.
- Usar modelos pequeños y baratos para las tareas rutinarias.
- Reservar Kimi K3 o GLM-5.2 para trabajos realmente difíciles.

Así pagarías por el **cómputo consumido**, no por quedar cautivo de una suscripción o de una API exclusiva.

En una frase:

El modelo abierto puede ser gratuito; ejecutar el modelo siempre consume recursos que alguien debe pagar.

Mauro

Intervención 28 · 2026-08-06 21:54:20Z

Claro. Entiendo. ¿Entonces como quedaría el panorama de trabajo?

ChatGPT

Intervención 29 · 2026-08-06

Así quedaría el panorama de trabajo de NativIQ

La arquitectura definitiva sería:

Dell como centro de mando + herramientas open-source + modelos frontier abiertos ejecutados en la nube + router que permite cambiar de modelo y proveedor.

No necesitas comprar una PC de IA ahora. Tu activo principal será la **portabilidad del sistema**, no la GPU.

```
Mauro - Dell Inspiron
      |
      v
Herramientas open-source
OpenCode · Goose · PydanticAI · Excel MCP
      |
      v
Router de modelos - LiteLLM
      |
      |-- Modelo rápido/económico
      |-- GLM-5.2 para ingeniería
      |-- Kimi K3 para trabajo general
      |-- Modelo verificador independiente
      `-- Modelo local pequeño de emergencia
      |
      v
Proveedores de cómputo intercambiables
Proveedor A · Proveedor B · Akash · otro host GPU
```

LiteLLM puede agrupar múltiples despliegues bajo nombres internos, aplicar reintentos y cambiar automáticamente de modelo o proveedor cuando uno falla. (docs.litellm.ai)

1. Qué hará tu Dell

La Dell será tu **puesto de dirección, desarrollo y supervisión.**

Ejecutará localmente:

- VS Code.
- OpenCode.
- Goose.
- PydanticAI.
- LiteLLM.
- Docker.
- Git.
- Bases de datos de desarrollo.
- Herramientas de Excel.
- Modelos pequeños para clasificación, embeddings y contingencia.
- Consolas de monitoreo y administración.

No será necesario que ejecute Kimi K3 o GLM-5.2 directamente. Esos modelos son demasiado grandes para una notebook común: Kimi K3 tiene 2,8 billones de parámetros y GLM-5.2 tiene aproximadamente 753 mil millones. Sus pesos están disponibles para despliegue independiente, pero requieren infraestructura de servidor. (huggingface.co)

La Dell enviará solicitudes a los modelos remotos y recibirá sus respuestas. En la práctica, será parecido a usar ChatGPT o Codex, pero la interfaz, los agentes y la selección del modelo estarán bajo tu control.

2. Qué herramienta usarás para cada trabajo

Trabajo	Herramienta	Modelo preferente
Programación cotidiana	OpenCode	GLM-5.2
Reparación de fallos	OpenCode/OpenHands	GLM-5.2
Refactorización compleja	OpenCode	GLM-5.2
Investigación y documentos	Goose	Kimi K3
Trabajo administrativo	Goose	Kimi K3
Navegación y aplicaciones	Goose + Browser Use	Kimi K3
Agentes de NativIQ	PydanticAI	Modelo según tarea
Procesos prolongados	PydanticAI + Temporal	Kimi o GLM
Excel	Excel MCP + xlwings	Kimi K3/GLM-5.2
Verificación	PydanticAI	Familia distinta

Trabajo	Herramienta	Modelo preferente
Routing y contingencia	LiteLLM	Todos

GLM-5.2

Lo utilizaría principalmente para:

- Código.
- Terminal.
- Repositorios.
- Diagnóstico técnico.
- Reparación de errores.
- Procesos de ingeniería prolongados.

GLM-5.2 se publicó bajo licencia MIT, tiene contexto de un millón de tokens y puede servirse mediante vLLM o SGLang detrás de una API compatible con OpenAI. Eso facilita moverlo entre proveedores sin cambiar OpenCode ni tus agentes. (huggingface.co)

Kimi K3

Lo utilizaría principalmente para:

- Trabajo documental.
- Investigación.
- Análisis visual.
- Hojas de cálculo.
- Presentaciones.
- Navegación.
- Tareas generales de conocimiento.
- Programación cuando necesite visión o contexto muy amplio.

Los pesos completos de Kimi K3 fueron publicados el 27 de julio de 2026. El modelo es multimodal, posee un contexto de un millón de tokens y puede desplegarse mediante vLLM o SGLang. Utiliza una licencia propia, no MIT, por lo que cada uso comercial debe conservar su licencia y respetar sus condiciones. (huggingface.co)

3. Cómo elegirás el modelo sin hacerlo manualmente cada vez

No configurarías OpenCode directamente con una URL llamada GLM-5.2.

Configurarías nombres internos de NativIQ:

```
nativiq-fast
nativiq-coding
nativiq-work
nativiq-vision
nativiq-verifier
nativiq-emergency
```

Por detrás:

```
nativiq-coding
|-- GLM-5.2 en proveedor A
|-- GLM-5.2 en proveedor B
|-- Kimi K3 como fallback
`-- modelo económico alternativo

nativiq-work
|-- Kimi K3 en proveedor A
|-- Kimi K3 en proveedor B
|-- GLM-5.2 como fallback
`-- modelo económico alternativo
```

Tus herramientas solo conocerán los nombres internos. LiteLLM decidirá dónde enviar cada solicitud y podrá aplicar:

- Balanceo.
- Reintentos.
- Límites de gasto.
- Fallback.
- Cooldown de proveedores fallidos.
- Registro de uso.
- Selección por costo o latencia. (docs.litellm.ai)

Así, cambiar de GLM-5.2 a un futuro GLM-6 no implicaría reinstalar OpenCode ni reprogramar tus agentes. Solo modificarías la configuración del router después de validar el nuevo modelo.

4. Dónde se ejecutarán los modelos

Tendrás tres modalidades.

A. API administrada

Es la forma más sencilla:

- Abrís una cuenta.
- Obtenés una clave.
- Pagás por tokens.
- El proveedor administra las GPU.

Sirve para empezar rápidamente y probar modelos. La desventaja es que seguís dependiendo temporalmente de esa API concreta.

Por eso no debe ser la única vía.

B. Proveedor independiente que aloja los pesos

Otro operador puede ofrecer Kimi K3 o GLM-5.2 mediante una API estándar.

Vos pagás la inferencia, pero no dependés obligatoriamente del creador del modelo. Si ese operador aumenta el precio o desaparece, cambiás a otro que ejecute los mismos pesos.

C. Despliegue propio sobre GPU alquilada

Podés alquilar servidores GPU y desplegar:

- El modelo.
- vLLM o SGLang.
- Tu contenedor.
- Tu configuración.
- Tu endpoint compatible con OpenAI.

Akash permite desplegar cargas GPU containerizadas mediante distintos proveedores y seleccionar características de hardware. También ofrece almacenamiento, secretos y endpoints para aplicaciones persistentes. (akash.network)

Esta modalidad da más control, aunque también exige más administración. No la usaría para todas las tareas desde el primer día.

5. Cómo controlarás los costos

No mantendrás Kimi K3 y GLM-5.2 encendidos permanentemente.

Clasificarás las tareas por dificultad.

Nivel 1 - Local o muy económico

Para:

- Clasificación.
- Extracción.
- Resúmenes simples.
- Routing.
- Moderación.
- Embeddings.
- Respuestas rutinarias.

Utilizarás modelos pequeños, localmente o en endpoints económicos.

Nivel 2 - Modelo medio

Para:

- Funciones aisladas.
- Documentación.
- Pruebas unitarias.
- Correos.
- Análisis ordinario.
- Automatizaciones frecuentes.

Utilizarás un modelo abierto de tamaño intermedio.

Nivel 3 - Frontier abierto

Kimi K3 o GLM-5.2 solamente para:

- Arquitectura.
- Repositorios grandes.
- Depuración compleja.
- Investigación extensa.
- Trabajo multimodal importante.
- Tareas prolongadas.
- Revisión crítica.

```
Tarea
|
v
¿Modelo pequeño puede resolverla?
|-- Sí -> modelo económico
`-- No
    v
    ¿Necesita código o ingeniería?
    |-- Sí -> GLM-5.2
    `-- No -> Kimi K3
```

De este modo, el modelo más caro no procesa cada mensaje rutinario.

6. Qué pasa si un proveedor se cae

Ejemplo:

```
OpenCode solicita nativiq-coding
|
v
GLM-5.2 proveedor A
|
FALLA
v
GLM-5.2 proveedor B
|
FALLA
v
Kimi K3 alternativo
|
FALLA
v
Modelo pequeño local
```

El último modelo no tendrá la misma capacidad, pero permitirá:

- Leer logs.
- Consultar runbooks.
- Preparar una respuesta.
- Clasificar el incidente.

- Mantener un modo degradado.
- Esperar la recuperación del cómputo externo.

La confiabilidad no consiste en prometer que nunca habrá una caída. Consiste en que una caída individual no detenga todo NativIQ.

7. Dónde deben residir los datos

Los proveedores de GPU deberían recibir **solamente el contexto necesario para cada tarea**.

No deberían conservar:

- La única copia de los datos.
- Las claves maestras.
- Los repositorios completos sin necesidad.
- Toda la memoria de NativIQ.
- Credenciales bancarias.
- Secretos de producción.

La arquitectura debería separar:

```
Datos, memoria y políticas
    bajo control de NativIQ
      |
      v
Contexto mínimo necesario
      |
      v
Modelo remoto desechable
```

Esto permite eliminar un endpoint y sustituirlo sin perder memoria, procesos ni información empresarial.

Para datos especialmente sensibles, tendrás tres alternativas:

1. Anonimizarlos antes de enviarlos.
2. Utilizar un proveedor dedicado y evaluado.
3. Procesarlos más adelante en una PC propia o infraestructura del cliente.

8. Cómo trabajarás en el día a día

Programación

1. Abrís OpenCode en la Dell.
2. OpenCode llama a `nativiq-coding`.
3. LiteLLM selecciona GLM-5.2.
4. El agente lee el repositorio y propone cambios.
5. Ejecuta pruebas en un entorno aislado.
6. Vos revisás y aprobás el diff.

7. Un segundo modelo puede verificar el resultado.

Trabajo administrativo

1. Goose recibe la tarea.
2. Llama a `nativiq-work`.
3. LiteLLM selecciona Kimi K3.
4. Goose consulta archivos o herramientas autorizadas.
5. Prepara el resultado.
6. Ejecuta automáticamente solo acciones reversibles.
7. Las decisiones críticas quedan pendientes de tu aprobación.

Soporte durante vacaciones

1. El agente recibe el ticket.
2. Clasifica la incidencia con un modelo económico.
3. Consulta el runbook.
4. Resuelve incidentes conocidos.
5. GLM-5.2 interviene si existe un fallo técnico complejo.
6. Si no puede verificar la solución, contiene el problema.
7. Informa al cliente y deja el caso pendiente.

9. Qué hardware necesitarás

Ahora

La Dell Inspiron 16 Plus 7640:

- Desarrollo.
- Administración.
- Control.
- Pruebas.
- Modelos pequeños.
- Acceso a modelos remotos.

Necesitarás además:

- Buena conexión fija.
- Internet móvil como contingencia.
- Backups cifrados.
- Una llave física de seguridad.
- Espacio suficiente en SSD.
- Eventualmente 64 GB de RAM si los contenedores lo requieren.

En el futuro

La PC potente deja de ser obligatoria.

Solo tendría sentido cuando:

- El gasto mensual de nube sea elevado y sostenido.
- Un cliente exija procesamiento local.
- Necesites privacidad física.
- Tengas una carga base constante.
- Un modelo mediano utilizado diariamente pueda amortizar el hardware.

Incluso entonces, los modelos frontier gigantes seguirían en la nube. La PC local atendería tareas medianas, datos sensibles y contingencia.

Panorama definitivo

Capa	Decisión
Propietario y control	Mauro
Equipo principal	Dell Inspiron
Interfaces de trabajo	OpenCode, Goose, Excel MCP
Agentes productivos	PydanticAI + Temporal
Router	LiteLLM
Coding frontier	GLM-5.2
Trabajo frontier	Kimi K3
Tareas ordinarias	Modelos abiertos medianos
Contingencia	Modelo pequeño local
Cómputo	Varios proveedores intercambiables
Estado y memoria	Bajo control de NativIQ
Hardware potente	Opcional, no estructural

Conclusión

Tu panorama de trabajo quedaría definido por esta regla:

NativIQ poseerá los agentes, los procesos, las evaluaciones, los datos y la capacidad de cambiar. Los proveedores solo alquilarán temporalmente el cómputo.

Esto te permite aprovechar modelos frontier abiertos sin comprar el hardware necesario para alojarlos, cambiar de generación cuando aparezcan modelos mejores y mantener una empresa unipersonal sin convertirte en operador de infraestructura.

Mauro

Intervención 30 · 2026-08-07 03:34:40Z

Quizas compre una computadora más potente. Pero esto será luego de unos años operando.

ChatGPT

Intervención 31 · 2026-08-07

Me parece la decisión más racional.

La computadora potente debería ser una **consecuencia del funcionamiento de NativIQ**, no una condición para empezar. Durante los primeros años, la Dell puede seguir siendo tu puesto de mando mientras delegás el cómputo pesado a modelos abiertos en la nube.

La compraría solo cuando se cumplan varias señales claras:

- NativIQ ya genera ingresos estables.
- Pagás suficiente cómputo mensual como para que el hardware se amortice.
- Tenés una carga de trabajo frecuente y predecible.
- Necesitás procesar datos sensibles localmente.
- Sabés exactamente qué modelos querés ejecutar.
- La máquina reduciría costos o riesgos, no solo aumentaría potencia.
- Podés amortizarla aproximadamente en dos o tres años.

Incluso entonces, no intentaría alojar todos los modelos frontier. La PC futura tendría un papel concreto:

- Modelos medianos de uso diario.
- Desarrollo pesado.
- Pruebas de integración.
- Procesamiento documental sensible.
- Embeddings y reranking.
- Contingencia si falla la nube.
- Ejecución local de agentes internos.

Los modelos gigantescos y las nuevas generaciones continuarían en la nube. Así evitás que la compra te deje estancado tecnológicamente.

La evolución quedaría así:

Primera etapa

Dell + modelos abiertos en la nube

Segunda etapa

Dell + nube + una PC potente para carga estable

Estructura definitiva

Dell como puesto de mando

PC potente como capacidad local

Nube como frontier, elasticidad y respaldo

De esa manera, la futura computadora no contradice el enfoque cloud-first. Se convierte en una **capa adicional de soberanía**, comprada cuando ya existe evidencia económica para justificarla.

Mauro

Intervención 32 · 2026-08-07 18:34:41Z

Por supuesto. Ahora me gustaría agregar esto al PDF.